

Unmasking the Internet: A Survey of Fine-Grained Network Traffic Analysis

Yebo Feng, Jun Li, Jelena Mirkovic, Cong Wu, Chong Wang, Hao Ren, Jiahua Xu, and Yang Liu

Abstract—Fine-grained traffic analysis (FGTA), as an advanced form of traffic analysis (TA), aims to analyze network traffic to deduce fine-grained information on or above the application layer, such as application-layer activities, fine-grained user behaviors, or message content, even in the presence of traffic encryption or traffic obfuscation. Different from traditional TA, FGTA approaches are usually based on complicated processing pipelines or sophisticated data mining techniques such as deep learning or high-dimensional clustering, enabling them to discover subtle differences between different network traffic groups. Nowadays, with the increasingly complex Internet architecture, the increasingly frequent transmission of user data, and the widespread use of traffic encryption, FGTA is becoming an essential tool for both network administrators and attackers to gain different levels of visibility over the network. It plays a critical role in intrusion and anomaly detection, quality of experience investigation, user activity inference, website fingerprinting, location estimation, etc. To help scholars and developers research and advance this technology, in this survey paper, we examine the literature that deals with FGTA, investigating the frontier developments in this domain. By comprehensively surveying different approaches toward FGTA, we introduce their input traffic data, elaborate on their operating principles by different use cases, indicate their limitations and countermeasures, and raise several promising future research avenues.

Index Terms—Network traffic, traffic analysis, traffic classification, traffic monitoring, fine-grained traffic analysis, intrusion detection, user behavior identification.

I. INTRODUCTION

IN the context of Internet, protocols and applications are usually built upon hierarchical models [1] (e.g., TCP/IP and OSI), where the communication functions of a telecommunication or computing system are categorized into several abstraction layers. Higher layers only encapsulate high-level methods, protocols, and specifications, operating with the support of lower layers [2]. With such design, programmers can easily develop interoperable Internet applications regardless of diverse underlying protocols and technologies. However, this convention also makes cross-layered network analysis feasible. As developers of higher layer applications

usually only take higher-layer measures (e.g., encryption, anonymization, etc.) to preserve the user privacy regardless of leaving traceable patterns on lower layers, analyzers can capture network features from the lower layers to infer higher-layer knowledge in communication [3], even in the presence of message encryption. Such a process is called traffic analysis (TA), a technique widely used in today's Internet.

TA has been studied for decades, with myriad systems, tools, and algorithms [4]–[10] developed to serve different types of purposes, such as traffic measurement, traffic engineering, anomaly detection, and network surveillance. In early development of TA, traditional TA approaches were mainly designed for basic network traffic measurement/forecast [11]–[13], anomaly detection [14], and coarse-grained traffic classification [15]. These approaches are usually rule-based, statistics-based, sketch-based [16], [17] or clustering-based, can separate traffic of different network protocols or conduct basic modeling of traffic flow changes. Later, with the advancement of the Internet and the growing complexity of network traffic, a deeper level of network visibility became necessary. For instance, application-layer visibility is crucial for measuring application usage, conducting application security inspections, and monitoring fine-grained network events. Similarly, visibility above the application layer is vital for modeling user behavior, analyzing user experiences, and performing content analysis. As a result, researchers and developers are evolving traditional TA into the more sophisticated fine-grained traffic analysis (FGTA), enabling the extraction of more granular insights from network traffic data. In this paper, we define FGTA as a type of advanced TA techniques that focus on deducing information on or above the application layer from network traffic data. These processes utilize only link-layer or network-layer traffic data and are applicable regardless of whether the traffic data is encrypted.

As a subset of TA, FGTA is mainly different from traditional TA in the following ways:

- The most essential difference is the output of analysis. Traditional TA can coarsely distinguish or model traffic from different types of protocols (e.g., HTTPS vs. SMTP), communication methods (e.g., VPN vs. Tor), or networking models (e.g., peer-to-peer vs client-server), generating coarse-grained traffic statistics, traffic flow models, or traffic classification results. However, FGTA aims to analyze traffic at a finer granularity, providing fine-grained analysis results on or above the application layer, such as traffic from different application-layer activities (e.g., Twitter posting vs. Twitter reading), different groups of application users (e.g., online social network

Yebo Feng, Cong Wu, Chong Wang, Hao Ren, and Yang Liu are with the College of Computing and Data Science (CCDS), Nanyang Technological University, Singapore, 639798 (E-mail: {yebo.feng, cong.wu, chong.wang, yangliu}@ntu.edu.sg and hao.ren@ieee.org).

Jun Li is with the Computer Science Department, University of Oregon, Eugene, OR, 97403, USA (E-mail: lijun@cs.uoregon.edu).

Jelena Mirkovic is with the Information Sciences Institute, University of Southern California, Marina del Rey, CA, 90292, USA (E-mail: mirkovic@isi.edu).

Jiahua Xu is with the Computer Science Department, University College London, UK (Email: jiahua.xu@ucl.ac.uk).

Corresponding author: Hao Ren.

(OSN) bots vs. normal users), or different user content (e.g., the visiting of a specific website).

- The analysis pipelines of traditional TA and FGTA are often, but not always, distinct. FGTA, aiming at generating more granular information, usually have more complicated and sophisticated analysis pipelines. For example, some FGTA approaches takes traditional TA as a prerequisite step to “preprocess” the traffic before the final inference, such as a FGTA approach that tries to identify the web page the user is visiting needs to first leverage traditional TA to extract all the web browsing traffic.
- As for analysis algorithms, compared to traditional TA approaches, most FGTA approaches depend on more sophisticated modeling, classification, or prediction methods, such as deep machine learning or high-dimensional clustering, to tackle the challenging fine-grained analysis tasks. On the other hand, traditional TA, dealing with more straightforward tasks, can utilize a number of different analytical methods, such as rule-based, statistics-based, or soft-computing-based approaches.

With the increasingly complex Internet architecture, increasingly frequent transmission of user data, and the widespread use of traffic encryption [6], FGTA is becoming a more and more important research topic. Compared with traditional TA, FGTA can reveal more information from network traffic and can achieve high efficacy even in various complicated network environments [18]. Besides, as network traffic data become more easily accessible than before, the applicable scenarios of FGTA are more extensive compared with directly analyzing traffic content. Furthermore, FGTA is efficient and portable in discovering application-layer knowledge [19]–[21]. By analyzing a small amount of metadata or statistical information of traffic, FGTA can obtain almost the same level of visibility as decoding large amount of message content. Therefore, FGTA has a wide range of usage scenarios. As for network managements, FGTA can help measure application usage [22], detect complicated network intrusions or anomalies [23], investigate edge user experience [24], etc. As for the attacker side, FGTA can help eavesdrop on private information of users [25], model user behaviors [26], estimate user locations [27], etc. Studying FGTA is essential for comprehensive network inspection, safeguarding information transmission, and precise network configuration.

In this paper, we conduct a comprehensive examination of over 300 pieces of literature related to FGTA. Our selection criteria encompassed sources from academic journals, academic conference proceedings, workshop papers, industry reports, preprint papers, etc. We prioritized academic publications that have made significant theoretical and practical contributions to the field of FGTA, published in reputable venues (e.g., ACM Computing Surveys, ACM CCS, Usenix Security, NDSS, ACM IMC, etc.), or have been widely cited. We search the literature from popular academic databases, such as IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, etc., and search engines, such as Google Scholar, Microsoft Academic, Google, etc. We employ a diverse set

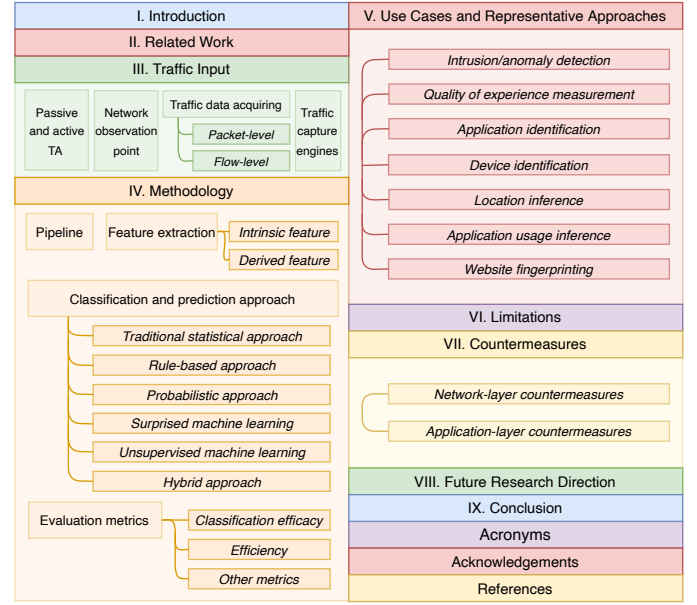


Fig. 1: The paper organization and snapshots of proposed taxonomies.

of keywords and keyword combinations to broadly identify potentially relevant literature¹. Subsequently, we manually review the content of these works to ensure their relevance to FGTA. This survey primarily focuses on literature published within the last two decades, specifically from 2004 to 2024. This time frame was deliberately chosen to capture both the foundational theories and the latest advancements in FGTA. Our analysis of the literature is conducted in a two-stage process: firstly, an initial review is conducted to assess the relevance, quality, and alignment of the sources with the objectives of our survey; secondly, we perform an in-depth review of these sources to identify their key contributions, methodologies, limitations, experimental setups, results, etc. The methodology we adopted aims to strike a balance between comprehensiveness and depth. By doing so, we ensure that our survey not only covers the most pertinent literature in the field of FGTA but also provides a thorough analysis of the most significant contributions within this domain.

The rest of this paper is organized as follows. We first introduce related work and compare existing surveys to our work in Section II. We then discuss the input data of FGTA (i.e., network traffic data) and its collection in real-world environments in Section III. Next, in Section IV, we discuss and summarize the methodologies of FGTA, including their operating pipelines, feature extraction methods, classification approaches, and evaluation metrics. Besides, we elaborate on frontier developments of FGTA by their use cases in Section V. We then point out the limitations of existing FGTA in Section VI and introduce the countermeasures in Section VII. In addition, based on our observations and reflections on

¹The keywords include but are not limited to: fine-grained traffic analysis, fine-grained network traffic classification, traffic-based user behavior inference, application-layer behavior modeling with network traffic, website fingerprinting, user location estimation with network traffic, detailed network traffic classification, etc.

TABLE I: Overview of related literature (○: not included; ◐: partially included; ●: included).

Ref.	Year	Summary	Focus	Subjects covered			
				General TA	FGTA	Traffic capture	Counter-measure
This paper	2025	A survey of FGTA, which aims to analyze network traffic to deduce information related to high-layer activities, fine-grained user behaviors, or application-layer message content.	FGTA	◐	●	●	●
[17]	2023	A survey of sketch-based traffic analysis using sliding windows, including their fundamental principles, primary use cases, advantages, and limitations.	Sketch-based TA	◐	○	◐	○
[28]	2023	A review of the literature on network traffic prediction, including experiments based on real data sets to compare the various approaches directly in terms of fitting quality and computational costs.	Network traffic prediction	◐	○	◐	○
[18]	2022	A recent survey on achievements in machine learning-powered encrypted traffic analysis, including the workflow, feature extraction, and algorithms.	Machine-learning-based encrypted TA	◐	◐	◐	◐
[29]	2022	A survey that consists of an analysis of IoT traffic data acquisition approaches, a classification of public datasets, a literature evaluation of IoT traffic processing, and a comparison of ML approaches for IoT device classification.	Machine-learning-based IoT TA	○	◐	●	○
[30]	2022	A survey of deep neural network (DNN) architectures commonly used in the traffic flow prediction literatures, categorizes and describes the literatures themselves, and presents an overview of the commonalities and differences among different works.	DNN-based traffic prediction	◐	○	◐	○
[6]	2021	A survey of literature that deals with network traffic analysis and inspection after the ascent of encryption in communication channels.	Encrypted TA	◐	◐	○	●
[31]	2021	An extensive analysis of the communications channels of 32 IoT consumer devices, including traffic measurement and modelling.	IoT TA	◐	◐	◐	○
[32]	2020	A survey that looks at the emerging trends of network traffic classification in IoT and the utilization of traffic classification in its applications. It also compares the legacy of traffic classification methods.	IoT TA	●	◐	◐	○
[5]	2019	A survey that mainly focuses on approaches and technologies to manage the big traffic data, additionally briefly discussing big data analytics (e.g., machine learning) for the sake of TA.	General TA	●	◐	●	○
[33]	2018	A review of works that contributed to the network traffic analysis targeting mobile devices, including a systematic classification of such works according to their goal, traffic capture point, and targeted platforms.	Mobile device TA	◐	◐	◐	●
[7]	2018	A systematic review based on the steps to achieve traffic classification by using machine learning techniques, including their workflow, feature extraction, deployment, etc.	Machine-learning-based TA	◐	◐	○	○
[34]	2016	An examination of the literature on analyses of mobile traffic collected by operators within their network infrastructure.	Mobile device TA	◐	◐	●	○
[35]	2015	A survey of approaches for classification and analysis of encrypted traffic, including widespread encryption protocols and payload and feature-based classification approaches.	Encrypted TA	◐	◐	○	○
[36]	2014	A survey in which a complete and thorough analysis of the most important opensource deep packet inspection modules is performed.	Payload-based TA	◐	○	◐	○
[37]	2013	A survey of peer-to-peer traffic detection and classification, with an extended review of the related literature.	P2P TA	◐	○	○	○
[38]	2009	A survey explains the main techniques and problems known in the field of IP traffic analysis and focuses on application detection.	General TA	●	○	●	○
[39]	2009	A report attempts to provide an overview of some of the widely used network traffic models, highlighting the core features of the model and traffic characteristics they capture best.	General TA	◐	○	○	○

this field, we propose some avenues for future research in Section VIII, thereby helping future academics and developers to advance FGTA. In the end, we conclude this paper in Section IX. To our best knowledge, this paper is the first survey paper that focuses on FGTA and compares the state-of-the-art approaches in this field. Figure 1 illustrates the organization of this survey paper and give snapshots of the proposed taxonomies.

II. RELATED WORK

In this section, we introduce related work on TA and compare our paper with them. Table I summarizes the most related and representative ones.

Our work differs from existing survey works regarding TA in the following aspects:

- we have a clear and focused survey topic: the whole paper focuses on FGTA, which aims to analyze network traffic to only deduce *fine-grained outputs*, such as information

related to high-layer application activities, fine-grained user behaviors, or application-layer message content. Notably, no other related survey papers, including [17], [18], and [29], have FGTA as their primary focus;

- to investigate FGTA comprehensively, our paper utilizes multiple methodologies, including literature survey, summarization, and taxonomization, to cover different related subjects, such as traffic capture, application identification, website fingerprinting, countermeasures, etc. Most existing related survey papers only cover a subset of the aforementioned topics;
- although studied for many years, TA is still iterating rapidly and continuously, especially for FGTA. Compared with other earlier literature, this paper sorts out and examine the most recent development of FGTA at the time of writing this paper.

In the early development of TA (i.e., before 2010), the survey papers in this field mainly focus on coarse-grained traditional TA [38]–[40], including protocol-level traffic classification, TA approaches based on deep packet inspection (DPI), distinguishing server or peer-to-peer nodes from clients, and coarse grained application identifications.

Later, due to increasingly diverse web-applications and widespread use of traffic encryption, there is an increasing need for more sophisticated TA approaches to monitor and analyze the modern networks. Meanwhile, the evolution of classification algorithms and easy access to big data also effectively stimulate the development of TA. Therefore, survey papers began to examine works that leverage big data [5], machine learning [7], or efficient data structures [17] to tackle TA.

On the other hand, the network is also becoming more and more specialized, which has spawned many TA approaches with specific design goals. To track such a trend, many of the recent survey papers only investigate a certain type of TA approaches, such as TA for Internet of things (IoT) devices [31], [32], encrypted TA [6], [18], TA for mobile devices [33], [34], among others. Similar to these papers, our work focuses on a new and specific topic—FGTA, which means our paper only focus on the TA approaches that input conventional network traffic data but generate fine-grained inference outputs (e.g., fine-grained user behaviors and application message content). This direction has not been systematically studied before.

III. TRAFFIC INPUT

Similar to traditional TA, FGTA approaches utilize the same type of network traffic data from some vantage points in the network as input to synthesize knowledge. Network traffic data refers to the information exchanged between devices on a computer network. Such data can be in diverse formats and include a wide range of information, such as communication logs, packet headers, and payload. The network traffic data is the inference object for all TA approaches. In this section, we introduce different types of traffic capture engines by the way they collect network traffic data, compare different types of network traffic data, and survey their usabilities. We also introduce these traffic capture engines' deployments and application scenarios in FGTA.

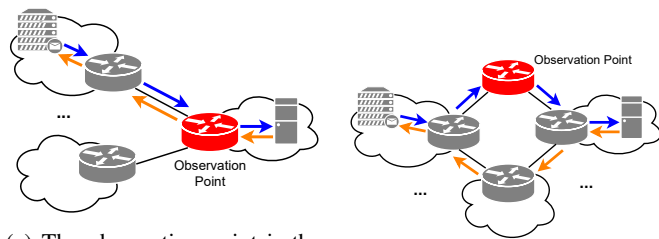
A. Passive and Active TA

TA can be generally classified into passive and active approaches based on the way they collect network traffic data [41], [42].

Passive TA approaches involves monitoring and analyzing network traffic without altering or injecting any data into the network [42], [43]. It relies solely on the observation of existing traffic flows. Typically, passive TA approaches consist of capturing packets, logging traffic patterns, and analyzing these logs to infer information. Such approach offer several advantages. First of all, passive TA approaches are non-intrusive. Since they do not interfere with the network, they are less likely to be detected by users or security mechanisms. Furthermore, passive TA approaches do not add additional load to the network, making them suitable for continuous monitoring. However, passive TA approaches may not capture all relevant information, particularly in cases of encrypted or obfuscated traffic. Additionally, as they rely on existing traffic flows, passive TA approaches cannot proactively test or measure specific aspects of network performance or behavior, making them inherently reactive.

Active TA approaches involve actively injecting data into the network to provoke responses, which are then analyzed to gain insights into the network's behavior and performance [44], [45]. Active TA approaches may include techniques such as sending probe packets [46], watermarking existing traffic [47]–[49], performing packet timing analysis, or conducting specific tests to measure latency, throughput, and other metrics. Compare to passive TA approaches, active TA approaches offer several advantages. First, active TA approaches can provide more detailed and precise information, especially about specific network conditions and device status. Besides, By generating traffic, active TA approaches can test scenarios and conditions that may not naturally occur, allowing for more thorough analysis. However, the act of injecting traffic can be detected by network users and security mechanisms, potentially leading to countermeasures [50]. Active TA approaches also add additional load to the network, which can affect performance and may not be suitable for continuous monitoring. Furthermore, active TA approaches require additional network infrastructure and resources to operate, such as a traffic encoder to inject or watermark traffic flows and a traffic decoder to identify marked traffic flows. This makes active TA approaches less applicable in real-world scenarios.

In the context of FGTA, passive TA approaches are more commonly used due to their non-intrusive nature, ability to continuously delve into network traffic data, and their focus on user-behavior-centric analysis models. In contrast, active TA approaches are rarely employed in FGTA as they are designed for measuring specific network conditions, endpoint device status, or the accessibility of network services, and focus less on undisturbed user behavior, which is the core of FGTA. Therefore, throughout this paper, we primarily focus on passive TA approaches. In this section, we specifically discuss the network traffic input by passive TA approaches rather than active TA approaches.



(a) The observation point is the gateway of the network. The traffic capture engine can collect bidirectional traffic data.

(b) The observation point is in the network. The captured traffic can be asymmetric.

Fig. 2: Network visibility with different observation locations.

B. Network Observation Point

The observation point of the traffic capture engine will significantly impact the integrity of the captured data and the network visibility. Different observation points are suitable for different types of TA tasks.

The optimal observation point for most FGTA tasks is at the network gateway (as shown in Figure 2a), as it provides full visibility into communication sessions between two nodes. The gateway allows for the capture of complete inbound and outbound traffic, which is essential for FGTA. Although such a bidirectional traffic dataset is suitable to infer the interactions between the observed network and rest of the Internet, analyzers cannot learn about the traffic in the rest of the Internet according to this dataset.

Sometimes, the observation point can be in the middle of the network (illustrated in Figure 2b), especially when the traffic capture engine is deployed by an ISP or IXP. In this case, the capture engine is able to collect a large amount of traffic that pass by it. However, it also raises the following concerns:

- Due to asymmetric packet routing [51], in-network observation point sometime may only capture traffic in one direction (illustrated in Figure 2b).
- It cannot guarantee the integrity of captured traffic for a long period because of the deployment of various traffic engineering techniques [52], [53]. The routing path for any packet can be dynamic in today's networks.

Therefore, in-network observation points may be more suitable for traditional TA tasks such as Internet measurement and network-layer anomaly detection. As for FGTA, many approaches (e.g., user behavior inference, website fingerprinting) prefer to use the gateway-based observation point to capture more complete traffic data from endpoints. However, wherever the observation point is located, it is difficult to capture all the relevant traffic in the network.

To capture comprehensive traffic data from the network with complex topology, we can deploy multiple observation points at different vantage points if conditions permit. By using a pool of metering processes to collect network packets at multiple observation points, optionally filter them and aggregate information about these packets, a traffic exporter can gather each of the observation points together into an observation domain and sends this information to a traffic capture engine [54]. Then we can obtain relatively comprehensive network traffic data without redundancy. Another benefit of deploying

multiple observation points is that it allows distributed or cooperative TA, where multiple analyzers can collaborate to analyze traffic data and synthesize more comprehensive knowledge [55]–[57]. For example, analyzers can choose to upload only extracted features or intermediate results to a central server to reduce the burden of data transmission or enhance the privacy of the data [58]; researchers can also leverage federated learning techniques to train a global model without sharing sensitive traffic data [59]–[62]. However, these approaches are costly to implement and often impractical due to real-world constraints, making them seldom used in practice, particularly in the context of FGTA.

C. Traffic Data Acquiring

Since the birth of the Internet, various traffic capture engines have been developed to log traffic information. TA approaches can further leverage these “log information” to measure network events, detect anomalies, and analyze network behaviors. Based on different information captured, these traffic capture engines can be classified into either packet-level or flow-level [4].

1) *Packet-level capture*: Packet-level capture is widely used in local networks and endpoint devices. As its name states, it copies or makes a snapshot of all the network packets that pass by the network interface and forwards the collected data to a collector. The agent that takes charge of the capture is called a packet-level traffic capture engine or a “sniffer”, which can be either software-based (e.g., Snoop [63], Wireshark [64], etc.) or hardware-based (e.g., Sniffer InfiniStream [65]). It can be as simple as an IP table rule on a route that copies all the traffic to a cloud disk besides normal forwarding.

Packet-level capture can collect raw network traffic, containing both packet headers and packet payloads. Theoretically, it can support all types of FGTA tasks by logging all the information flowing through the network, making it an ideal input source for FGTA. However, in most cases, packet-level traffic capture might not be the right solution to deploy for the following reasons:

- Packet-level traffic capture is expensive, not only because the interface needs to copy all the packets that pass by it, but also because the interface needs to forward all the captured traffic to an analysis node through a link. All these operations will double the workload of the network interface and occupy a considerable amount of link bandwidth. Packet-level traffic capture is therefore not scalable.
- The information contained in packet-level traffic data is sometimes an “overkill” for TA, as many TA approaches only require statistical information from the packet headers to complete the analysis. Moreover, user messages, website content, and video streaming are usually contained in packet payloads in encrypted forms, making most information captured in packet-level traffic meaningless for all TA approaches.
- Packet-level traffic may contain sensitive information (i.e., payload) of users. Thus, network service providers are cautious about capturing and analyzing such data.

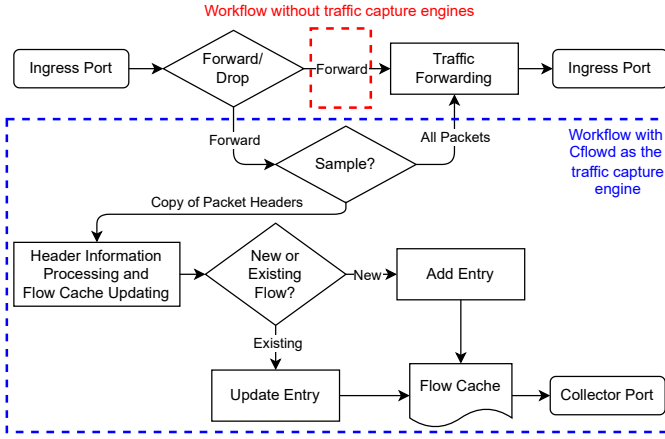


Fig. 3: Workflow of a network interface when Cflowd serves as the traffic captured engine.

2) *Flow-level capture*: To address the aforementioned issues of packet-level traffic captures and make traffic capturing affordable, scalable, and practical for network service providers, researchers and developers have proposed myriad flow-level traffic capture engines.

In flow-level traffic capture systems, the capture engines no longer copy or make snapshots of each packet, instead, they first aggregate relevant packets into a flow and then capture metadata or statistical information to represent that flow. Here, the concept of flow has been around for a long time. Typically, a flow can be identified by either a 5-tuple (i.e., source IP address, source TCP/UDP port, destination IP address, destination TCP/UDP port, and IP protocol) or a 3-tuple (i.e., source IP address, destination IP address, and IP protocol) [66]. However, with the development of flow capture engines, researchers have proposed many other formal and informal definitions of network traffic flows (e.g., RFC 2722 [67], RFC 3697 [68], RFC 3917 [69], etc.). In this paper, we define a network traffic flow as a sequence of relevant network packets from a source to a destination for the same application. In most instances, the network system will process packets within a flow in the same manner. Besides, each application-layer behavior will generate one or multiple flows in both directions.

By capturing traffic at flow-level, traffic capture engines no longer suffer from high system overhead and high bandwidth usage. Figure 3 illustrates the workflows of a network interface with and without Cflowd as the flow-level traffic captured engine [70]. Unlike packet-level traffic capture that will copy and forward any packet entirely to the collector port, flow-level traffic capture only copies information from headers to assemble traffic flows. The volume of data to process is then largely reduced in such a procedure. According to existing literature [71], NetFlow, the most frequently used flow-level traffic capture engine, only creates 1-1.5% of throughput (without sampling) on the interface it is exported on [72]. With a great deal of data reduction, network administrators can store, process, inspect and analyze large amounts of network data efficiently. Furthermore, when combining this procedure

with packet sampling, it becomes feasible to capture and store traffic flows at an ISP or IXP scale, thereby extending the usage scenarios of TA. As we can see from a study, NetFlow only occupies around 15% of the router/switch's CPU load when capturing sampled network traffic [73]. Compared with packet-level traffic capture that sometimes may double the system overhead and link usage, flow-level traffic capture is a huge improvement regarding efficiency and deployability.

However, flow-level traffic capture has a notable drawback—it reduces the visibility of network traffic by providing only metadata and aggregated statistical information rather than details of individual packets. This limitation is particularly problematic for FGTA, as many approaches rely on inter-packet information, such as packet interval times. To address this issue, the lifecycle of each flow in traffic capture engines can be shortened, allowing flows to be generated more frequently and thereby improving network visibility.

D. Widely used traffic capture engines

Here, we introduce widely-used traffic capture engines in academia and industry (Table II shows comparisons of them).

1) *Packet-level traffic capture engines*: Back in the early days of Internet, developers had realized the importance of capturing network packets for troubleshooting. Thus, Tcpdump [80], a software-based packet-level traffic capture engine (sniffer), was proposed in 1988. It allows users to store and display TCP/IP and other packets being transmitted or received over a network. Nowadays, Tcpdump has been ported to several operating systems (e.g., Unix with libpcap library, Windows with WinPcap) and is still frequently used in network studies. Similar software-based sniffers were also proposed to meet different needs. For example, Snoop [63], a simple packet capture tool that is bundled on Solaris operating system; Wireshark [64], a free packet capture and analysis software that not only supports multiple operating systems (e.g., Linux, Solaris, Windows, FreeBSD, Mac OS, etc.), but also comes with a user-friendly interface; PF_RING [81], a high speed packet capture library that can turn a commodity PC into an efficient and cheap network measurement box suitable for both packet capture and TA. As for routers and switches, traffic mirroring [83]–[85] is also well-studied, with many software or hardware-based approaches [65], [86] proposed to support real-time packet capture for enterprise-level networks.

However, as capturing the entire packet is expensive and sometimes impractical, people began to make a snapshot of each packet rather than storing it entirely. The most frequently-used approach is sFlow [79], an industrial method (defined in RFC 3176 [79]) originally developed by InMon Inc., to capture packet-level snapshot from switches and routers. Compared to previous packet-level traffic capture engines, sFlow offers several features that make it an ideal input for most FGTA approaches:

- Without capturing the entire packet, sFlow can just copy the first N bytes of a packet to save computing and transmission resource. This is especially useful for TA tasks as packet payloads are useless in such scenarios

TABLE II: Comparisons of selected widely-used traffic capture engines (●: fully supported; ◐: partially supported; ○: not supported.).

Traffic Capture Engine	Data Captured	Granularity	Open or Proprietary	Layer (OSI)	Hardware Acceleration	Sampling
SNMP [74]	High-level statistical information about the interface.	Flow-level (aggregated)	Open	2, 3	○	○
IPFIX [75]	Metadata and statistical information about the flow.	Flow-level	Open	3, 4	●	●
NetFlow v9 [76]	Metadata and statistical information about the flow.	Flow-level	Proprietary	3, 4	●	●
NetFlow v5 [77]	Metadata and statistical information about the flow.	Flow-level	Proprietary	3, 4	●	●
Argus [78]	Metadata and statistical information about the flow.	Flow-level	Open	2, 3, 4	●	○
sFlow [79]	Complete packet headers and partial packet payloads.	Packet-level	Partially Open	2 - 7	●	●
Tcpdump [80]	Network information passing through the observation point.	Packet-level	Open	2 - 7	○	○
Wireshark [64]	Network information passing through the observation point.	Packet-level	Open	2 - 7	○	○
PF_RING [81]	Network information passing through the observation point.	Packet-level	Open	2 - 7	●	◐
Netmap [82]	Network information in the memory of the observation point.	Packet-level	Open	2 - 7	●	◐

but the entire packet headers are still preserved for fine-grained analysis.

- As an industrial standard, sFlow is compatible on many different platforms of network switches and routers and utilizes a dedicated chip built into the devices to operate, which removes the burden of the CPU and memory of the router or switch when capturing the traffic.
- By introducing time-based or packet-based sampling techniques, sFlow can capture traffic on all interfaces simultaneously at wire speed.

Therefore, sFlow can reach a good balance between data integrity and velocity for FGTA—being able to capture all the packet headers and simultaneously create less burden on the router or switch.

2) *Flow-level traffic capture engines*: Flow-level traffic capture engines also have a long history. Back in 1984, the Audit Record Generation and Utilization System (Argus flow [78]) was proposed as the first implementation of network flow monitoring, and is still an ongoing open source network flow monitor project now. Argus can monitor all network traffic, including Internet Protocol (IP) traffic, data plane, control plane and management plane. It captures much of the packet dynamics and semantics in each flow, providing reachability, availability, connectivity, duration, rate, load, delay metrics for all network flows. It also captures most attributes that are available from the packet headers [87]. Later, in 1988, Simple Network Management Protocol (SNMP) [74] was proposed as a component of the Internet Protocol Suite as defined by the Internet Engineering Task Force (IETF). Unlike Argus flow that provides rich information about ongoing traffic, SNMP only provides statistical information per interface, such as link utilization, interface bandwidth, and some other information if the device provides. SNMP is thus less applicable in TA compared with Argus, especially in the domain of FGTA.

With rapid development and popularization of the Internet, the industry had realized the importance of flow-level traffic capture engine and many solutions were proposed. The most typical example is NetFlow [76], so far the most widely-used flow-level capture engine with many TA approaches built upon. Just like Argus, NetFlow uses a flow record to represent a set of packets. However, unlike Argus, which is a

bidirectional monitoring approach, NetFlow is a unidirectional flow monitor, reporting flow information of each direction of conversations independently. This feature allows NetFlow to have a finer granularity than Argus. Since NetFlow was developed by Cisco, it is bundled with most Cisco routers and switches, making it the object of imitation of the entire industry. Following NetFlow, many similar systems were proposed by both research institutions and commercial companies, such as Cflowd [70], J-Flow [88], NetStream [89], Remote Network Monitoring (RMON) [90], etc. NetFlow itself also has evolved into different variations. The most famous one is Internet Protocol Flow Information Export (IPFIX) [75], an IETF protocol built upon NetFlow v9.

The most recent development of traffic capture and traffic handling have been mainly focusing on the velocity issue. Researchers have proposed multiple approaches to capture large volume of network traffic at line speed without having any effect on data plane. For example, Netmap [82] a memory-based framework that enables commodity operating systems to handle millions of packets per seconds without the support of custom hardware; eXpress Data Path (XDP) [91], a fast programmable packet processing approach based on the operating system kernel, supports high speed packet logging and processing; hXDP [92], an efficient software network packet processing approach written in extended Berkeley Packet Filter (eBPF) on Field Programmable Gate Arrays (FPGA) network interface controllers (NICs); NetSeer [93], a flow event telemetry (FET) monitor, which aims to discover and record all performance-critical events on the programmable data plane. However, these approaches do not alter the FGTA pipeline; they simply enhance the speed of infrastructure for capturing and processing network traffic, without modifying the captured data itself.

IV. METHODOLOGY

In this section, we delve into the methodology of FGTA and explore this field from the perspectives of data processing pipelines, feature extraction approaches, classification & prediction approaches, and evaluation metrics. These components are integral to the success of FGTA and play crucial roles in achieving accurate results.

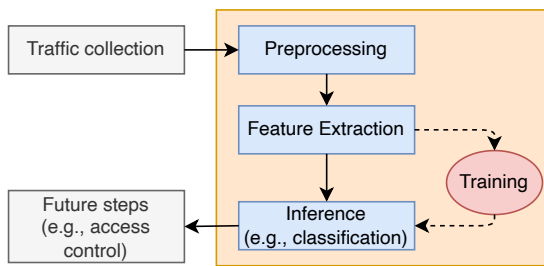


Fig. 4: The most widely-used processing pipeline for FGTA, which consists of three key steps: preprocessing, feature extraction, and classification.

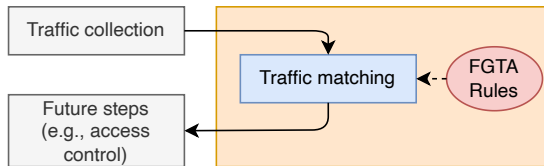


Fig. 5: Example of a simplified FGTA data processing pipeline, which is used when the target traffic pattern is distinct or well-defined.

A. Pipeline

The process of generating fine-grained analysis results from raw network traffic collected from network infrastructures typically involves several necessary steps. These data processing procedures are known as the FGTA pipeline. Different FGTA approaches may have different pipelines, with different steps and different orders. In this subsection, we discuss three types of FGTA pipelines (illustrated in Figure 5, 4, and 6).

Figure 4 depicts a commonly adopted pipeline for FGTA. This pipeline is also prevalent in traditional TA methodologies as it offers a complete and versatile framework for the processing of network traffic data. Regardless of whether the input traffic is in flow-level or packet-level format, it usually cannot be directly processed by analysis algorithms. Therefore, similar to traditional TA pipeline, the first step of this FGTA pipeline is usually to preprocess the raw traffic data. The preprocessing step typically involves the following tasks:

- **Data decoding:** the raw network traffic data is usually encoded in a format that is not easily processable (e.g., binary format or encrypted form). This task converts the raw traffic data into a readable and processable form.
- **Data cleaning:** the raw traffic data may contain some noise, invalid data, or control messages. This task extracts only the valuable data for further analysis.
- **Data refactoring:** this task refactors the raw network traffic data and make it suitable for the subsequent analysis or maintenance. For example, indexing the raw traffic data by socket pairs, or converting the flow records to a B tree structure [94].
- **Other tasks necessary for subsequent steps:** depending on different FGTA pipelines, there may be other tasks necessary for subsequent steps. For example, extracting marked packets from the raw traffic data, anonymizing the raw traffic data for General Data Protection Regulation

(GDPR) compliance [95], or compressing the data for efficient storage.

After the preprocessing step, FGTA approaches usually move to feature extraction, which refers to the process of selecting and transforming raw network traffic data into a set of relevant features that are suitable for machine learning, inference, or other analysis steps. For both traditional TA and FGTA approaches, the feature extraction is a particularly important step for representing the ongoing network events and achieving accurate results. However, compared with traditional TA approaches, FGTA approaches may require more sophisticated feature extraction steps as they often need to extract more detailed information from the raw traffic data. We further discuss more details about feature extraction in Section IV-B.

After relevant features are extracted, FGTA approaches are typically ready for inference. The inference goals of these approaches can vary, including identifying specific network events, classifying traffic flows based on different application behaviors, detecting network anomalies, or predicting specific future network traffic. We further discuss the use cases of FGTA approaches in Section V. The inference results of FGTA approaches can be used for a variety of purposes, including network monitoring, access control, device management, data center protection, etc. However, regardless of the inference goal, the inference step of FGTA always operates in the form of fine-grained classification or prediction. For example, classifying outlier traffic flows from normal traffic flows (i.e., anomaly detection), or classifying traffic flows according to different applications (i.e., application identification). Therefore, we use the term *classification and prediction* to refer to the inference step of FGTA approaches. Compared traditional TA approaches, FGTA approaches may require more sophisticated classification and prediction approaches as they often need to classify or predict more detailed information from extracted features. These advancements may include more sophisticated machine learning algorithms, more sophisticated statistical modeling, or more complicated rule-based matching. Section IV-C discusses the classification and prediction approaches used in FGTA.

The previously mentioned pipeline outlines the general steps for FGTA approaches. However, depending on the specific goals, system design, and operational environments, the FGTA pipeline can be simplified or extended, with specific steps omitted or added.

Figure 5 illustrates a simplified FGTA pipeline, where the raw network traffic data is directly used for rule-based traffic pattern matching. A short data processing pipeline is very efficient to operate and can still generate accurate results if the pre-defined matching rules are simple and effective. It is useful when target traffic pattern is distinct or well-defined (i.e., location inference [27]). This simplified pipeline is widely-used in traditional TA approaches due to the simplicity of the target traffic pattern. For example, sketch-based approaches utilize probabilistic data structures (e.g., hash-based methods) to directly match and measure incoming traffic's statistics with low overheads and high throughput [96]–[101]. However, these approaches can only coarsely distinguish or measure traffic from different types of devices, protocols, or distinguishable

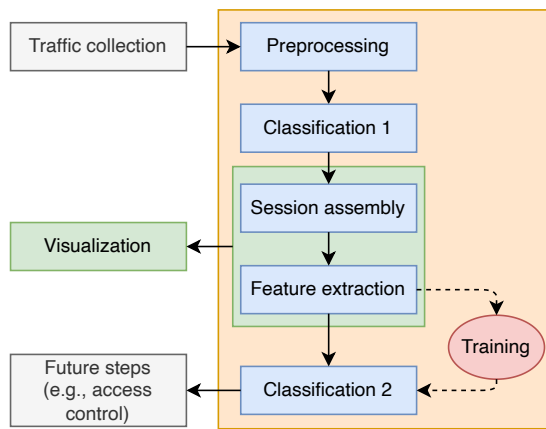


Fig. 6: Example of a more complicated FGTA data processing pipeline, which is used when the target traffic pattern is not directly distinguishable.

applications, serving as a tool for large-scale network measurement or a prerequisite step for more sophisticated FGTA approaches. To generate fine-grained, application-layer inference results, such a simplified pipeline is inadequate. Additional data processing steps are needed to more thoroughly analyze the traffic data for digging the hidden fine-grained information.

To infer high-level, fine-grained information from content-agnostic network traffic data, many FGTA approaches tend to employ more complicated pipelines to mine hidden and hard-to-dig knowledge. These complicated pipelines are not widely seen in traditional TA approaches due to the simplicity of the target traffic pattern and added computational complexity. Figure 6 illustrates such an example. Many application usage inference approaches apply similar pipelines [26], [102], [103] because they need to extract features and classify traffic for multiple times at different phases to derive detailed user behavior information of specific applications. The sample pipeline include two different analysis phases, with one for narrowing down the analysis scope and the other for generating fine-grained analysis results. More importantly, this pipeline re-assemble traffic flows into sessions (some papers may call them transactions [21] or bursts [22], [104]) before extracting features for the final classification. This step is very helpful for digging fine-grained behavior information from the traffic data because the target network behavior or event usually consist of multiple packets or traffic flows. Simply analyzing the network traffic flow by flow or packet by packet may not be able to capture the whole picture of the ongoing network events. Therefore, session assembly is used to aggregate adjacent, relevant, or similar traffic data into an analysis unit, which is a more representative data structure to present the ongoing network events and makes it easier for FGTA approaches to infer fine-grained application-layer information. Figure 7 illustrates an example of session assembly [103], where flow records are divided into flow points and then aggregated into sessions according to the traffic density. Based on current literature, the following approaches are commonly used for session assembly:

- **Time-based session assembly:** this approach aggregates

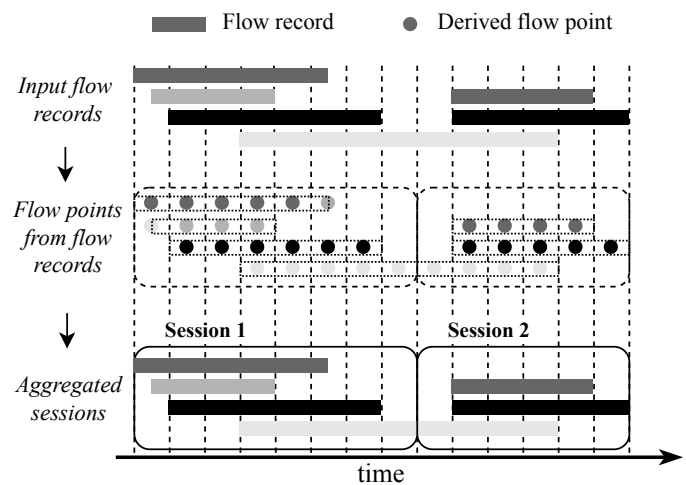


Fig. 7: Example of a session assembly procedure, where relevant flow records are aggregated into a traffic session to represent a network event.

traffic flows into sessions based on the timing or the intervals of ongoing network traffic.

- **Clustering-based session assembly:** this approach utilize clustering algorithms to group traffic flows into sessions.
- **Index-based session assembly:** this approach aggregates traffic flows into sessions by specific indexes (e.g., socket pair, packet ID ranges, time to live (TTL), etc).
- **Rule-based session assembly:** this approach aggregates traffic flows based on pre-defined rules (e.g., rules on the hash value of packet payload, rules on TCP flags, etc.).

The session assembly step is rare in traditional TA approaches. After sessions are assembled, representative features can be properly extracted and forwarded to next steps for fine-grained classifications or predictions.

B. Feature extraction

Feature extraction is a term refers to the process of selecting and generating relevant features from the raw data in order to create a representation that can be used for machine learning, statistical modeling, or other analysis procedures [105]. It is an essential step for both traditional TA approaches and FGTA approaches. In the context of TA, feature extraction involves inspecting network traffic data to identify relevant features that can be used for the corresponding classification tasks. This process typically involves techniques such as packet inspection, data fusion, and statistical modeling to identify and derive patterns or characteristics in the data that are relevant to the specific inference goal. Recently, the rise of deep learning techniques has enabled numerous approaches to automatically extract or select features from preprocessed data inputs or extensive feature sets [106]–[108]. However, considerations of efficiency, efficacy, explainability, and the complexity of network traffic data mean that many FGTA approaches continue to depend on meticulously crafted feature extraction techniques to generate features for subsequent analytical steps. The resulting set of features will be used as

TABLE III: Examples for intrinsic and derived features for FGTA.

Category	Example	
Intrinsic feature	Flow-level	Packet-level
	Flow size, number of packets, AS number, protocol type, flow duration, etc.	TCP flag, ToS, packet size, packet interval, first n bytes of the payload, etc.
Derived feature	Flow/packet-based	Session-based
	Interval deviation, size deviation, interval distribution, inbound/outbound packet ratio, packet similarity, etc.	Session duration, density distribution, session image, normalized session vector, round-way communication number, etc.

input to the classification or prediction models to generate the analysis output. In contrast to traditional TA methods, FGTA often necessitates more advanced and intricate feature extraction techniques. This is because FGTA aims to deduce detailed application-layer information from raw traffic data, requiring a more insightful and informative representation of the network traffic data.

Due to the nature of network traffic collection, all the extracted features can be categorized into two types: intrinsic features and derived features. Intrinsic features are directly contained in the raw network traffic data, such as packet length, packet header fields, etc. The process of fetching intrinsic features is simple and straightforward. The analysis system can directly select, slice, or generate intrinsic features from the raw data, requiring little to no additional processing. On the other hand, derived features are not directly contained in the raw network traffic data. They are generated by applying some data processing techniques to the raw data, such as statistical modeling, feature transformation, information assembly, data fusion, etc. Typically, traditional TA methods more commonly utilize intrinsic features, while FGTA approaches tend to rely on derived features. This preference is attributed to their differing analysis objectives, granularity, and efficiency goals.

Table III lists some typical examples of intrinsic and derived features. Different features are suitable for different FGTA tasks. Typically, some relatively easy FGTA tasks may only require intrinsic features to operate. For example, some application identification or anomaly detection approaches can generate accurate results by directly inputting intrinsic features. Because the network traffic of such applications or anomalies can already be distinguishable by intrinsic features [20], [109], [110]. However, some more complex FGTA tasks may require derived features for finer granularity analysis, especially for tasks that infer detailed, application-layer user behaviors [22], [103], [111]. The target network traffic of these FGTA tasks is less distinguishable and may only show obvious patterns with sophisticated feature engineering techniques. We discuss more details about suitable features for different FGTA tasks in Section V.

Although derived features are more powerful than intrinsic features in mining fine-grained information from network traffic data, they are also more complex and expensive to generate. One may need to apply sophisticated data processing techniques, such as traffic buffering, data fusion, session

assembly, statistical modeling, etc., to fetch these derived features. Such processes are time-consuming and may require significant computational resources. As time-sensitive tasks, it is vital for FGTA procedures to be efficient and scalable, thereby outputting analysis results in a timely manner. Thus, carefully selecting and generating necessary features is a critical step for all FGTA approaches.

C. Classification and prediction approach

The key step of FGTA is to classify the target network traffic from others or to predict the target network traffic's behavior. Designing a proper classification or prediction approach determines the efficacy and performance of FGTA approaches. In many cases, constructing a classification or prediction model requires labeled data. In the context of FGTA, labeled data refers to network traffic data that has been manually labeled or annotated with ground truth information. This ground truth information typically includes information such as the application type, user behavior type, or whether the traffic is generated by malicious behavior or not. Obtaining labeled data can be a challenging and resource-intensive process. It typically requires a significant amount of manual effort and expertise to accurately label network traffic data. Researchers may be able to automate the labeling process with the help of other state-of-the-art classification approaches, but the accuracy of labels may not be ideal [112]. On the other hand, some classification and prediction approaches can be trained without labeled data or with other forms of prior knowledge. In the remaining of this subsection, we discuss the classification and prediction approaches that are commonly used in FGTA approaches (summarized in Table IV).

1) *Traditional statistical approach*: Traditional statistical approaches leverage statistical properties, statistical models or some other mathematical methods to identify subtle differences or patterns in different groups of network traffic [123]. Typical examples of statistical approaches include distribution fitting [113], logistic regression [114], linear regression [23], etc. Traditional statistical approaches are widely used in traditional TA tasks because they are explainable, easy to implement, usually efficient to operate, and good at tackling relatively easy tasks. However, as FGTA tasks becoming more and more challenging, traditional statistical approaches are not sufficient to identify subtle differences in network traffic. Thus, traditional statistical approaches are gradually replaced by more sophisticated classification approaches, such as machine learning approaches. Still, traditional statistical approaches are commonly used in feature extraction, pre-analysis, and pre-classification. For example, many FGTA approaches use traditional statistical approaches to narrow down the analysis scope before fine-grained analysis, thereby reducing the computational complexity of the subsequent procedures [113].

2) *Rule-based approach*: Rule-based approaches are based on a set of pre-defined rules that are manually designed by experts to locate the target network traffic group [124]. Before defining the classification rules, the experts usually need a thorough understanding of the target network traffic and the network environment. Typical examples of classification rules

TABLE IV: Summary of widely-used classification and prediction approaches in FGTA.

Category	Description	Representative algorithms/approaches	Pros	Cons	Use in FGTA	Reference
Traditional statistical approach	Leverage statistical properties or statistical models for FGTA tasks.	Distribution fitting, regression, variance matching, etc.	Explainable, easy to implement, and efficient to handle large amounts of network traffic.	Poor efficacy especially when the FGTA task is challenging.	Limited	[23], [113], [114]
Rule-based approach	Utilize a set of pre-defined rules to locate the target network traffic group. For FGTA tasks, the rules can be complicated.	Session signatures, traffic thresholds, predefined packet header fields, etc.	Explainable, easy to implement, controllable, and efficient to handle large amounts of network traffic.	It is usually challenging to define rules for FGTA tasks. Poor efficacy and poor flexibility.	Limited	[19], [24], [115]
Probabilistic approach	Approaches based on probability theories to analyze network traffic at a fine granularity.	Bayesian classifier, Markov model, HMM, etc.	Flexibility, adaptability, and ease of use.	Complexity, sensitivity to assumptions, limited accuracy, and relatively poor explainability.	Popular	[109], [116], [117]
Supervised machine learning	ML methods that rely on labeled network traffic data to learn knowledge, which can be used for network traffic classifications or predictions.	KNN, SVM, LSTM, transformer, few shot learning, etc.	Great efficacy, ease of use, and good flexibility.	Limited explainability, overfitting, dependency on high-quality labeled data, limited scalability, and requiring relatively long training time.	Most popular	[118]–[120]
Unsupervised machine learning	ML methods that do not require labeled training data and can discover patterns and relationships in the network traffic data on its own.	K-means, PCA, DBSCAN, etc.	Discovering unknown patterns in network traffic data, flexibility, no labels needed, and no training time.	Limited result interpretability, limited efficacy, poor scalability in inference, and overfitting.	Popular	[103], [121], [122]
Hybrid approach	Combine multiple different approaches for better performance in FGTA.	Ensemble model, semi-supervised machine learning, combining statistical approaches with rule-based approaches, etc.	Inherits advantages of multiple classification or prediction approaches.	Complicated to design, and computationally expensive.	Popular	[20], [113], [122]

include session signatures [24], traffic thresholds [115], pre-defined packet header fields [19], etc. Similar to traditional statistical approaches, rule-based approaches are explainable, easy to implement, and efficient to operate, thereby being widely used in traditional TA tasks. However, in the era of FGTA, the analysis tasks are in finer granularity and becoming more and more challenging. Thus, the pre-defined rule sets are becoming larger, more complex, making them more difficult to define, verify, and maintain. Moreover, the rule-based approaches are not able to adapt to the dynamic network environment, which is a common feature of modern networks. Therefore, in recent trends, rule-based approaches are less used in FGTA tasks. But they are still powerful tools in some specific FGTA tasks, pre-classification, and accelerating the analysis process.

3) *Probabilistic approach*: Probabilistic approaches are based on probability theory and statistical inference to identify the target network traffic group [125]. They model the traffic data probabilistically for classification tasks. For instance, typical probabilistic approaches like Bayesian classifier [126], Markov model [127], or hidden Markov model (HMM) [128] are widely used to model network traffic first. These models can then be utilized to identify traffic patterns of specific applications, protocols, anomaly, or behaviors. Benefitting from the following advantages, a variety of FGTA approaches have been proposed based on probabilistic approaches to tackle different FGTA tasks [109], [116], [117]:

- Flexibility: probabilistic approaches can be used to model a wide range of traffic patterns and behaviors. Besides, they can tolerate noise and uncertainty in the data, making them powerful tools for analyzing complex and heterogeneous traffic data.
- Adaptability: probabilistic approaches can be easily

adapted to changes in traffic patterns over time, allowing them to detect new or previously unseen threats.

- Ease of use: with the support of various libraries, probabilistic approaches are relatively easy to implement and does not require extensive domain knowledge or expertise.

However, probabilistic approaches feature the following disadvantages, resulting limited performance and application especially in complicated FGTA tasks (e.g., user behavior inference):

- Complexity: probabilistic approaches are usually computationally expensive, especially when the network traffic data is large and complex.
- Sensitivity to assumptions: probabilistic approaches are sensitive to the assumptions (labels) made during model training or development, and incorrect assumptions can lead to inaccurate results.
- Limited accuracy: probabilistic approaches may not achieve the highest accuracy compared to other more advanced methods, such as deep learning, in some scenarios. Also, they are relatively weak in handling nondiscrete data.
- Explainability: probabilistic approaches may not provide as much interpretability as other methods, making it difficult to understand how the models arrived at their conclusions.

4) *Supervised machine learning*: Supervised machine learning is a widely used machine learning method that can be applied to almost any FGTA tasks with reliable prior knowledge [18]. In supervised machine learning, a classifier is trained using a labeled training dataset that includes known classification labels. The trained classifier is then used to classify or detect anomalies in new traffic data. Supervised ma-

chine learning approaches typically involve two main phases: training and inference. During the training phase, the classifier is trained on the labeled dataset (i.e., labeled network traffic) to learn the relationship between the input features and the classification labels. The inference phase involves using the trained classifier to infer the classification labels of ongoing network traffic.

With decades of development, researchers have proposed a variety of supervised machine learning approaches [129], from traditional machine learning methods, such as k-nearest neighbor (KNN), decision tree, Support Vector Machine (SVM), to advanced deep learning methods [130], [131], such as multi-layer perceptron (MLP), recurrent neural network (RNN), long short-term memory (LSTM). Recently, there has been a notable surge in applying state-of-the-art machine learning techniques to FGTA. These include few-shot learning [132]–[135], which achieves commendable accuracy with minimal training network traffic data; transformers [136], [137], known for their superior pattern recognition capabilities and scalability in training; transfer learning [138], [139], which utilizes knowledge from other tasks or domains to enhance FGTA performance with limited training traffic data; and online learning [140], a dynamic approach where the model continually updates and refines its parameters with incoming network traffic streams, enabling real-time adaptation to evolving network traffic patterns. Each of the proposed supervised machine learning approaches has its own advantages and disadvantages, making them suitable for different FGTA tasks. Selecting the most suitable supervised machine learning approach is the key to designing an effective ML-based FGTA approach. We discuss more details about the ML algorithm select by use case in Section V.

Overall, due to the following advantages, supervised machine learning approaches are the most widely used approaches in FGTA [23], [118]–[120]:

- High accuracy: supervised machine learning can achieve high accuracy in FGTA, especially when compared to other methods.
- Ease of use: on the one hand, supervised machine learning approaches are relatively easy to implement, with supports of various libraries and tools [141]–[143]. On the other hand, they do not require extensive domain knowledge or expertise to manually identify distinguishable rules or patterns.
- Flexibility: supervised machine learning approaches can be used to model a wide range of traffic patterns and behaviors.

However, supervised machine learning approaches also have many shortcomings that limit their performance and use cases in FGTA tasks:

- Limited explainability: many supervised machine learning algorithms, such as DNN, can be difficult to interpret, which can limit their usefulness in some applications, especially in anomaly or attack detection.
- Overfitting: supervised machine learning models can overfit to the training data, which can result in poor performance on new data.

- Dependency on labeled data: supervised machine learning requires high-quality labeled training data, which can be time-consuming and expensive to collect, making them less effective than unsupervised or semi-supervised methods in some cases.
- Limited scalability: many supervised machine learning approaches may not scale well to extremely large or complex datasets. Both the training and inference phases may be computationally expensive.

5) *Unsupervised machine learning*: Unlike supervised machine learning, unsupervised machine learning is a machine learning method that does not require labeled training data and can discover patterns and relationships in the data on its own [144]. Unsupervised machine learning algorithms typically involve clustering [145] or dimensionality reduction [146] techniques that can help identify similarities and differences between traffic flows. These algorithms do not directly output labeled classification results, but can be used to group similar traffic flows together or identify anomalous traffic flows that do not fit into any of the existing clusters. Widely used unsupervised machine learning algorithms include K-means [147], DBSCAN, principal component analysis (PCA) [148], hierarchical clustering [149], etc.

Unsupervised machine learning algorithms feature the following advantages in FGTA:

- Discovering unknown patterns: unsupervised machine learning approaches can identify previously unknown patterns and behaviors in the traffic data, which can be useful for detecting new or emerging threats.
- Flexibility: unsupervised machine learning approaches can be more flexible and adaptable than supervised machine learning as they do not require labeled data, making them easy to work with a wide variety of traffic datasets.
- No training time: unsupervised machine learning approaches usually takes zero training time, making them more efficient than supervised machine learning approaches regarding model development.

They also inevitably have the following disadvantages:

- Limited result interpretability: interpreting the results of the clustering or dimensionality reduction algorithms used in unsupervised machine learning can be difficult without prior domain knowledge.
- Limited accuracy: unsupervised machine learning may not achieve the same level of accuracy as supervised machine learning, especially when dealing with complex or noisy traffic datasets.
- Scalability in inference: although taking no time for training, some unsupervised machine learning algorithms are computationally expensive in the inference phase, which can limit their scalability.
- Overfitting: unsupervised machine learning models can also suffer from overfitting or underfitting, which can result in poor performance on certain datasets.

In conclusion, unsupervised machine learning is a powerful tool for FGTA, but it may have limitations in terms of result interpretability and accuracy. Due to such natures, unsupervised

TABLE V: Widely used metrics that indicate the classification efficacy for FGTA, where TP denotes the number of true positives, TN denotes the number of true negatives, FP denotes the number of false positives, and FN denotes the number of false negatives.

Metric	Description	Calculation
TPR	The probability that an actual positive will test positive, the key metric that indicates the sensitivity or true positive rate of an analysis, reflects the FGTA approach's ability to correctly identify those with the condition.	$\frac{TP}{TP+FN}$
TNR	The probability that an actual negative will test negative.	$\frac{TN}{TN+FP}$
PPV	The probability that an item with a positive test result is truly positive.	$\frac{TP}{TP+FP}$
NPV	The probability that an item with a negative test result is truly negative.	$\frac{TN}{TN+FN}$
FNR	The probability of positives which yield negative outcomes, an important metric especially in anomaly or attack detection.	$\frac{FN}{FN+TP}$
FPR	The probability of negatives which yield positive outcomes, one of the most important metrics that indicates the usability of the FGTA approach. A high false positive rate (FPR) can lead to a large number of false alarms, forcing network administrators to ignore the analysis results.	$\frac{FP}{FP+TN}$
FDR	The probability that an item with a positive test result is truly negative.	$\frac{FP}{FP+TP}$
FOR	The probability that an item with a negative test result is truly positive.	$\frac{FN}{FN+TN}$
F1	The harmonic mean of precision (PPV) and recall (TPR), indicating a balance between the accuracy of positive predictions and the completeness of positive case identification.	$\frac{2TP}{2TP+FP+FN}$
ACC	How close a given set of analysis results are to their true value, the most widely used metric that indicates the overall reliability of the FGTA results in reflecting the actual situation.	$\frac{TP+TN}{TP+TN+FP+FN}$
ROC	A graph showing the performance of a classification model at all classification thresholds, key to understanding and adjusting the trade-off between true positive rate and false positive rate to reach the best analysis performance for the FGTA approach.	Through TPR and FPR.
AUC	The area under the entire ROC curve.	Through ROC.

machine learning approaches are not widely used in tasks such as anomaly detection and attack detection [103], [121].

6) *Hybrid approach*: Hybrid approaches combine the advantages of multiple classification approaches to achieve better adaptability, explainability, or performance. People can use different approaches as different procedures in the FGTA pipeline, enhancing the feature extraction or pre-classification phase, or simply use an ensemble classification model to increase the robustness. For example, combining supervised and unsupervised machine learning approaches for semi-supervised traffic classification [20], combining statistical approaches with machine learning approaches better FGTA performance [113], or utilizing a variety of approaches in the FGTA pipeline for more comprehensive attack coverage [122].

Although hybrid approaches can usually achieve better performance, they are more complex to design and assemble. Besides, hybrid approaches are usually more computationally expensive than using single approaches.

D. Evaluation metrics

In FGTA, evaluation metrics are important measures of the performance, efficiency, and usability of proposed approaches. In this section, we discuss some commonly used evaluation metrics.

1) *Classification efficacy*: The most important evaluation metrics for FGTA are to indicate the classification efficacy. They measure the accuracy of the proposed approach in classifying the target traffic flows. The classification efficacy is well-defined in the domain of data mining [150]. It can be measured by true positive rate (TPR), true negative rate (TNR), positive predictive value (PPV), negative predictive value (NPV), false negative rate (FNR), FPR, false discovery rate (FDR), false omission rate (FOR), F1 score (F1), accuracy (ACC), receiver operating characteristic (ROC), area under the curve (AUC), etc. Table V lists the calculations and descriptions of these metrics. For specific FGTA tasks, some metrics may be more important than others. For example, in anomaly or attack detections, FPR and FNR are more important than PPV and NPV because FPR determines the false alarm rate, reflecting the usability of proposed approaches, and FNR determines the miss rate, reflecting the detection effectiveness of proposed approaches. FGTA is usually a high-throughput analysis task. Given its substantial volume of input, even a relatively small FPR or FNR can be magnified, resulting in numerous false alarms or overlooked detections. This situation places network administrators in a challenging position, where they might have to disregard the analysis results or risk allowing excessive malicious traffic through the network. While in tasks such as webpage fingerprinting, PPV and NPV are more important than FPR and FNR because PPV and NPV are more relevant to the classification efficacy.

2) *Efficiency*: Efficiency is another important evaluation metric for FGTA, which is usually measured by the time cost of finishing analyzing a certain amount of traffic flows by the proposed approach. In addition to accuracy and other performance metrics, the time cost can have a significant impact on the proposed approach's practicality and applicability. In real-world scenarios, the majority of FGTA tasks are performed on a large amount of traffic flows in real time. Therefore, the bottom line of these FGTA tasks is to reach the line speed in processing traffic flows. Here, the line speed refers to the maximum speed at which a FGTA approach can process incoming traffic data without dropping or losing packets. It is typically measured in terms of bits per second (bps) or packets per second (pps).

To increase efficiency, many FGTA approaches optimize the feature extraction procedure, classification algorithms, or the general pipeline. Some approaches also choose to design dedicated hardware architectures to accelerate the FGTA process. For example, FlowLens [151] utilizes programmable switch to support ML-based flow classification at hardware level, making ML-based flow classification efficient enough to catch up with line-speed traffic. We discuss more details about this issue in Section V.

3) *Other metrics*: According to different use cases, FGTA approaches may need to consider many other metrics, such as the memory cost, the storage cost, compatibility, stability, etc. For example, FGTA-based intrusion detection approaches may need to consider the explainability of the outputs to help network administrators understand the logic behind the detection results, thereby facilitating the network security management with confidence [152]. The explainability here

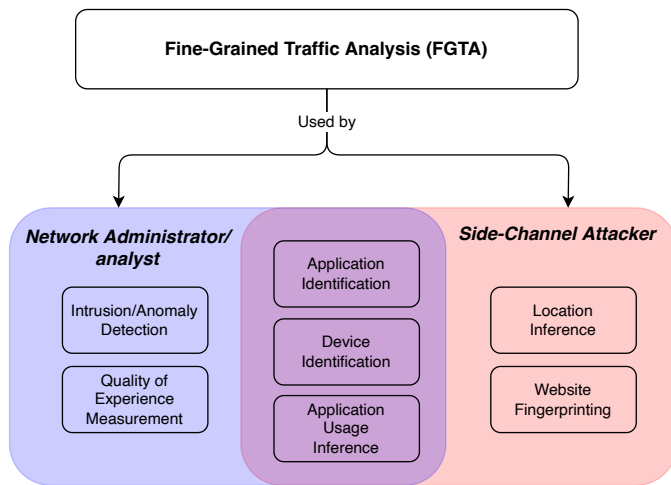


Fig. 8: A taxonomy for FGTA by use case.

not only refers to the reason why the FGTA approach generates a certain output but also refers to the mitigation costs, network situations, confidence level, etc. In addition, some FGTA approaches may need to consider the privacy of the users for compliance with certain regulations (e.g., GDPR [95] and California Consumer Privacy Act (CCPA) [153]). In such cases, they may not be able to collect, store, or process certain types of network traffic data directly, such as the payload of network packets. Thus, developers need to select suitable metrics for specific use cases.

V. USE CASES AND REPRESENTATIVE APPROACHES

As discussed in Section I, FGTA has a wide range of uses. FGTA can be leveraged by both attackers and network administrators, for both illegal purposes and social good. Attackers can leverage FGTA to eavesdrop on user activities online [114], [118], [154], stealing sensitive information and raising significant privacy concerns. By analyzing even encrypted traffic patterns, they can infer user behaviors and preferences, posing threats to both individual and organizational security. This highlights the need for effective countermeasures to prevent the misuse of FGTA technologies. On the other hand, people can leverage FGTA to detect network anomalies [103], [155], [156] or better manage the network [24], [157]. This has also been widely practiced in the industry. For example, Kentik [158] provides a network traffic intelligence platform that can help network administrators detect network anomalies, optimize networks, and perform fine-grain network troubleshooting. Palo Alto Networks [159] provides a network traffic analysis platform that can help network administrators detect intrusions in fine granularity.

According to their use cases, we propose a taxonomy for FGTA approaches (illustrated in Figure 8). In the remainder of this section, we review representative FGTA approaches across different categories, examining their goals, operation mechanisms, performance, and key differences. We have also summarized some of the comparisons in Table VI, VIII, IX, and X. Notably, since most existing works do not provide code

for testing, we cannot directly compare their detailed performance. Therefore, system overheads are estimated based on the complexity of their pipelines, feature extraction methods, and algorithms. Similarly, their real-time analysis capabilities are estimated based on their operational mechanisms and the processing time data listed in the papers (if available). For papers that do not explicitly mention the time needed for analysis, we estimate the time based on their input data, methodologies, or other related data. Thus, the real-time analysis capabilities listed in the tables are only rough estimates based on specific settings in the papers, which may vary in different environments. In the comparisons, approaches that can deliver results within a minute with sufficient computational resources are considered real-time; those requiring certain amount of flow-level data or traffic session completion are deemed partially real-time (e.g., 1-10 minutes); and approaches needing a long time (e.g., over 10 minutes) to collect enough data or the entire dataset for offline analysis (e.g., clustering-based approaches) are classified as not real-time.

A. Attack/Anomaly Detection

TA approaches are widely used by both the industry and academia to detect anomalies or attacks. With more than two decades of research, we have seen a myriad of solutions (e.g., [160]–[163]) targeting at different types of threats. However, as network attacks become increasingly sophisticated, detection approaches based on traditional TA are gradually becoming insufficient to address modern threats, particularly those occurring at or above the application layer. For example, traditional TA approaches may excel at detecting conventional DDoS attacks but often fall short in identifying application-layer anomalies, such as online social network bots or data exfiltration. Therefore, researchers began adopting FGTA to model endpoints' or users' behaviors to detect such attacks/anomalies. In this subsection, we elaborate on FGTA-based attack/anomaly detection approaches, introducing their applicable scenarios and operation mechanisms (Table VI shows an overview).

1) *Intrusion detection*: Many FGTA approaches focus on detecting complicated intrusions in the network by examining the characteristics of the underlying network traffic. Most of them apply machine learning models to perform the detection.

Amoli et al. [121] leveraged an unsupervised machine learning model (i.e., density-based spatial clustering of applications with noise (DBSCAN)) to distinguish subtle differences between historic traffic and intrusion traffic. Their approach is able to detect zero-day and complex attacks without much prior knowledge of these attacks. Papadogiannaki et al. [174] generated traffic signatures from packet metadata sequences and then used these to detect intrusions in the UNSW-NB15 dataset [175].

Many researchers also focus on utilizing supervised deep learning models to detect intrusions. Tang et al. [110] extracted six basic features from traffic flows and trained a DNN model with the NSL-KDD dataset to detect intrusions. Shone et al. [122] first leveraged nonsymmetric deep autoencoder (NDAE) for unsupervised feature learning. Then, they implemented stacked NDAEs with GPU-based architectures for

TABLE VI: Comparisons of selected attack/anomaly detection approaches (○: not supported; ◐: partially supported; ●: supported).

Category	Approach	Year	Goal of Analysis	Feature	Method	System Overhead	Real-Time Analysis	Real-World Evaluation
Intrusion Detection	Amoli et al. [121]	2016	Mail-bomb, SSH-processable, botmaster, etc.	Flow-level traffic feature such as duration, number of packets, smallest packet size, largest packet size, etc.	DBSCAN	Medium	○ (~10 mins)	○
	Tang et al. [110]	2016	R2L, U2R, Probe, DoS	Duration, protocol type, src bytes, dst bytes, count, srv count	DNN	High	◐ (~1-5 mins)	○
	Shone et al. [122]	2018	R2L, U2R, Probe, DoS, guess password, portsweep, buffer overflow, etc.	Features extracted with NDAE	NDAE for unsupervised feature learning, stacked NDAEs for detection	High	◐ (~1-5 mins)	○
	Mirsky et al. [155]	2018	Video injection, ARP MitM, OS scan, etc.	Damped incremental statistics and 23 other features from packet-level data	Kitsune's core algorithm (KitNET), a type of autoencoders	High	● (0.1-10 secs)	●
	Han et al. [164]	2023	Normality shift detection. Unauthenticated OSPF, P2P traffic, MS SQL Stack BO, log anomaly, advanced persistent threat (APT), etc.	Features appeared in the datasets	Detect shift statistically through hypothesis testing; tackle normality shifts by optimizations and restricting model parameter updating.	High	●	●
	Ullah et al. [165]	2024	Infiltration, brute force, DoS, etc.	Features extracted from network packets and selected by the CNN model.	Transformer-based transfer learning, Synthetic Minority Oversampling Technique (SMOT), convolutional neural network (CNN), and LSTM.	High	◐ (~1-5 mins)	○
	Feng et al. [152]	2024	Explainable and adaptive DDoS detection	Different feature sets for different types of DDoS, mainly statistical features.	KNN with k-dimensional tree and grid optimization	Low	● (<10 secs)	●
Malware Detection	Shabtai et al. [166]	2014	Malicious attacks or masquerading/injected mobile applications	2 best feature subsets selected from 20 manually defined feature subsets of various sizes	Linear regression, decision table, SVM for regression, Gaussian processes for regression, isotonic regression, and decision/regression tree	Medium	◐ (mostly <10 mins)	◐
	Wang et al. [167]	2016	Android malware such as plankton, FakeInstall, FakeRun, MobileTx, etc.	Six TCP flow features and four HTTP request features	C4.5 decision tree	Low	●	○
	Lashkari et al. [168]	2017	Malicious and masquerading applications such as Airpush, Kemoge, AVpass, FakeAV, etc.	24 features extracted from both packet and flow-level traffic	Random forest, KNN, decision tree, random tree, and regression	Medium	◐ (~1-5 mins)	◐
	Anderson et al. [23]	2017	Detecting malicious, encrypted malware network traffic	22 and 319 data features in the standard and enhanced feature set extracted from NetFlow and IPFIX data	Linear regression, logistic regression, decision tree, random forest, SVM, and MLP	High	◐ (<5 mins)	●
	Piskozub et al. [169]	2021	Malware detection and classification, including adware, ransomware, trojan, virus, and worm.	Flow duration, round-trip time, IP protocol used, connection towards local or public IP, destination port, packets sent, bytes sent, packets received, bytes received, sent packet payload entropy, and received packet payload entropy.	A combination of denoising autoencoders and DNN classifiers	High	◐ (~1-5 mins)	◐
Data Exfiltration Detection	Ren et al. [170]	2016	Cross-platform information leak identification	Raw network packets with payload	Decision tree, AdaBoost, bagging, blending, and Naïve Bayes	Medium	● (<1 min)	●
	Continella et al. [171]	2017	PII leakage detection, even in the presence of obfuscation techniques	Raw network packets with payload	Behavior modeling and differential analysis	Low	○ (~98 mins)	◐
	Rosner et al. [156]	2019	Information leaks in TLS-encrypted network traffic	A feature space that includes observations about individual packets and sequences of packets; additional features from the phase detection and the full original traces.	Trace alignment, phase detection, feature selection, feature probability distribution estimation and entropy computation	Low	◐	○
	Willems et al. [172]	2023	Data exfiltration as occurring in ransomware attacks	Average packet count and request entropy per session, average session duration, payload size, time between sequential sessions, and weights.	The anomaly detector is composed of an ensemble layer with multiple autoencoders and a Threshold Checker.	High	◐	○
Others	Feng et al. [103]	2021	Online social network bot detection	Traffic fingerprint images converted from NetFlow data	DBSCAN, CNN	High	◐ (~1-5 mins)	●
	Coulter et al. [173]	2019	A data-driven cyber security system that can identify high-level application-layer attacks or anomalies such as Twitter spam	Statistical features extracted from the network traffic and content (optional)	A variety of classification approaches	High	●	○
	Feng et al. [113]	2022	Cryptojacking activity	Packet size, timing, direction, and protocol from sFlow data	LSTM	High	◐ (≥5 mins)	○

quick and accurate intrusion detection on labeled datasets (i.e., KDD Cup '99 and NSL-KDD). Mirsky et al. [155] monitored the statistical patterns of network traffic and designed an ensemble of neural networks called autoencoders to collectively differentiate between normal and abnormal traffic patterns. Their approach is able to detect various attacks (e.g., video injection, ARP MitM, OS scan, etc.). Besides, unlike many other approaches that are only evaluated in closed-world environments, this approach was tested with a real-world test bed.

2) *Malware detection*: Today's malware is becoming more and more challenging to be detected by traditional TA due to traffic hiding and the increasing adoption of traffic encryption. FGTA could be an ideal tool to detect such malware.

Shabtai et al. [166] proposed a framework for malware detection on Android platforms. It can identify attacks or masquerading applications installed on a mobile device and injected applications with malicious code by semi-supervised machine-learning methods. Wang et al. [167] leveraged a machine learning algorithm (i.e., C4.5 decision tree) in analyzing mobile traffic, which is capable of identifying Android malware with high accuracy—more than 98%.

Later, some researchers collectively evaluated the efficacy of different machine learning models in detecting malware. Lashkari et al. [168] detected malicious and masquerading applications with five different classifiers—random forest, KNN, decision tree, random tree, and regression. They found that these models can achieve similar performances in malware detection. Besides, they published a labeled dataset that contains both benign Android applications and injected applications' network traffic. Anderson et al. [23] designed and carried out experiments that show how six machine learning algorithms (e.g., linear regression, logistic regression, decision tree, random forest, SVM, and MLP) perform when confronted with real network data. They found the random forest ensemble classifier to be the most robust for the domain of malware detection.

3) *Data exfiltration detection*: FGTA can also be used in detecting data exfiltration, thereby protecting personal sensitive data from leakage. Different from directly detecting anomalies or attacks, approaches in this domain usually profile user behaviors or model normal application usage to identify abnormal data transfer.

Wei et al. [176] proposed ProfileDroid, which is the first approach to profile mobile application at four layers: (a) static, or application specification, (b) user interaction, (c) operating system, and (d) network. At network-layer, this approach can capture essential characteristics of application communications, including but not limited to the ratio of incoming traffic and outgoing traffic, number of distinct traffic sources, traffic intensity, the percentage of HTTP and HTTPS traffic, etc. The profiling information can help identify inconsistencies and surprising behaviors, thereby detecting data exfiltration. A similar work is TaintDroid [177]. It leverages dynamic information-flow tracking to identify private data leaks of Android applications. The authors indicated that network traffic is useful to help monitor the behavior of popular third-party Android applications and discover potential misuse cases

of user private information across applications. These two approaches do not only leverage network traffic, but their ideas inspired a lot of subsequent work in this domain.

Later, researchers began to investigate purely using network traffic to profile application usage and report possible data exfiltration. Razaghpanah et al. [178] monitored network communications on mobile phones from user-space. The proposed approach facilitates user-friendly, large-scale deployment of mobile traffic measurements and services to illuminate mobile application performance, privacy and security. Song et al. [179] proposed a VPN-based approach to detect sensitive information leakage. Le et al. [180] proposed AntMonitor, which passively monitors and collects packet-level measurements from Android devices to provide a fine-grained analysis. By inspecting the network traffic data, it can provide users with control over how their data is shared by applications. Ren et al. [170] proposed ReCon, a cross-platform system that reveals personally identifiable information (PII) leaks by inspecting network packets and gives users control over them without requiring any special privileges or custom operating system (OS). The authors leveraged the Weka data mining tool [181] to train classifiers that predict PII leaks. Continella et al. [171] proposed an approach to privacy leak detection that is resilient to obfuscation techniques (e.g., encoding, formatting, encryption). To achieve the goal, the authors first established a baseline of the network behavior of applications, and then utilized black-box differential analysis on application usages.

However, the aforementioned approaches still require inspections on traffic content to detect data exfiltration. The ideal FGTA-based solution should be content-agnostic. In 2019, Rosner et al. [156] presented a black-box approach for detecting and quantifying side-channel information leaks in TLS-encrypted network traffic. Given a user-supplied profiling-input suite in which some aspect of the inputs is marked as secret, it combines network trace alignment, phase detection, feature selection, feature probability distribution estimation and entropy computation to quantify the amount of information leakage that is due to network traffic.

4) *Others*: A few research works have been focusing on using FGTA to detect other types of application-layer anomalies. By harnessing the power of machine learning on big data, such approaches can model fine-grained application-layer anomalies only with flow-level traffic or packet headers. For example, BotFlowMon [21], [103] detects online social network bot traffic by converting NetFlow records to images and training a CNN-based classification model; Coulter et al. [173] proposed a data-driven cyber security system that can detect Twitter spam or other high-level application-layer anomalies through machine-learning-based flow analysis; Feng et al. [113], [182] detects cryptojacking traffic by inferring the hash rate stability with cryptomining traffic in sFlow format. Table VI lists their analysis features and methodologies.

B. Fine-Grained Quality of Experience Investigation

Fine-grained quality of experience (QoE) measures how the specific web service is experienced by individual users at the

edge of the network [183], thereby providing a more user-centric perspective on network performance for network troubleshooting, configuration, and optimization. It is an essential approach for internet service provider (ISP)s to provide high-quality services to the network users.

In traditional TA, three techniques servers similar purposes: quality of service (QoS) [184]–[186], network traffic prediction [28], [30], [139], and traditional QoE. However, they address the network performance issue from different angles. QoS refers to the network parameter settings configured by service providers to deliver various levels of service to their customers, which focuses on the network service quality from the perspectives of overall network rather than individual users. Network traffic predictions aim to predict the future network traffic flow based on historical data, thereby helping network administrators to allocate network resources efficiently or configure the network in a more suitable way. Network traffic predictions can be achieved through machine learning [187]–[189], statistical models [190], SARIMA models [191], etc. We can see that both QoS and network traffic prediction are more network-centric and focus on aggregated user traffic, while fine-grained QoE emphasizes individual users' detailed network experiences, focusing on user-centric, application-layer information extraction.

On the other hand, plenty of works have been proposed to conduct QoE investigations with traditional TA (e.g., [183], [192], [193]). They can roughly classify network traffic into several groups (e.g., video, voice, data transfer, etc.) using statistical, DPI-based, or rule-based approaches and measure the service experience according to some metrics. However, such approaches may not be able to tackle today's increasingly complicated network traffic, since different types of traffic may be encrypted by different protocols (e.g., HTTPS, Quick UDP Internet Connection (QUIC)) and sent from different devices (e.g., IoT, smartphone, server) by different applications. Besides, service providers may want to conduct more granular management of network traffic. For example, residential areas' network administrators may want to increase the priority of video streaming traffic related to YouTube for certain users; network administrators of companies may want to ensure the quality of online meeting traffic for some offices. Therefore, over the past ten years, researchers have started leveraging FGTA to infer detailed user information at or above the application layer, enabling QoE investigations with greater precision and finer granularity.

Usually, fine-grained QoE investigations are performed in two steps:

- 1) Extract the target traffic using traffic classification.
- 2) Measure the extracted traffic to check if it meets certain criteria.

Some approaches may combine these two steps into one and directly identify potential QoS/QoE problems. Table VII shows a comparison of some selected QoE methods.

In 2016, Dimopoulos et al. [194] proposed a random-forest-based detection model to identify QoE issues related to YouTube video streaming. By selecting three sets of features with information gain, the proposed model is able to directly detect different levels of QoE degradation that is caused by

TABLE VII: Comparisons of selected fine-grained QoE investigation approaches.

Approach	Goal	Method	Feature
[194]	Identify QoE degradation in YouTube	Random forest	Three feature sets selected by information gain.
[195]	Estimate QoE in YouTube	Random forest, J48, Naïve Bayes, OneR, and SMO	Five hand-crafted feature sets
[196]	Estimate video streaming QoE over HTTPS and QUIC protocols	Decision tree	A packet-level feature set extracted from network and transport-layers
[157]	Estimate QoE in YouTube	Random forest and linear regression	Three feature sets (inbound, outbound, and inbound + outbound)
[24]	Estimate mobile ABR video adaptation behavior over HTTPS and QUIC protocols	Traffic fingerprinting with chunk sizes	Packet size and timing

three key influence factors (i.e., stalling, the average video quality, and the quality variations). The authors demonstrated that it can detect QoE problems with an accuracy of 92% by evaluating this approach using collected traffic. At the same year, Orsolic et al. [195] also studied using different machine learning algorithms (i.e., random forest, J48, Naïve Bayes, OneR, and Sequential Minimal Optimization (SMO)) to detect YouTube QoE issues under different bandwidth scenarios. In 2019, Khokhar et al. [157] proposed the first work that not only can identify YouTube QoE issues related to objective factors (e.g., startup delay, stalling, resolution change, etc.), but also can identify QoE issues related to the subjective Mean Opinion Score (MOS).

Mazhar et al. [196] further extends QoE investigation to all encrypted video streaming traffic (transferred over HTTPS or QUIC) by using a classification model trained by a decision tree. They demonstrated that their approach is able to achieve a 90% classification accuracy for HTTPS and an 85% classification accuracy for QUIC. Xu et al. [24] infers mobile Adaptive Bitrate (ABR) video adaptation behavior using packet size and timing information in encrypted environments.

C. Website Fingerprinting

Website fingerprinting (WFP) is used to identify what web page the user is visiting, even in the presence of traffic encryption or encrypted tunnels established by Tor [197], [198], Shadowsocks (i.e., a popular secure socks5 proxy) [199], VPN, etc. It is a FGTA technique that widely-used by attackers to eavesdrop on user activities or content online. In this subsection, we survey and compare well-known WFP approaches (Table VIII), elaborating on the history of WFP and investigating its capability.

1) *Early development of WFP*: WFP has a long history. The early WFP attacks simply focused on using data sizes to infer the URL the user is visiting through encrypted SSL connections. Back in 1998, Mistry et al. [200] demonstrated that the size of HTML files is a critical feature to specific web pages.

TABLE VIII: Comparisons of selected WFP approaches (○: not supported; ◐: partially supported; ●: supported).

Approach	Year	Method	Feature	System Overhead	Real-Time Analysis	Effectiveness			
						HTTP/1.1	VPN	Tor	Multi-tab
Mistry et al. [200]	1998	Size matching	Size of HTML file	Low	● (few milliseconds)	○	○	○	○
Sun et al. [201]	2002	Similarity score calculation (Jaccard's coefficient)	HTTP object count, sizes, etc.	Low	● (few milliseconds)	◐	○	○	○
Bissias et al. [115]	2005	Cross correlation of two value sequences	Packet size and inter-arrival time distributions	Low	◐	●	◐	○	○
Liberatore et al. [202]	2006	Similarity score calculation (Jaccard's coefficient)	Direction and length for each packet	Low	●	●	◐	○	○
Herrmann et al. [203]	2009	Multinomial Naïve Bayes	Frequency distribution of the IP packet size	Low	● (~10 secs)	●	●	○	○
Panchenko et al. [204]	2011	SVM	Volume, time, and direction of the traffic	Medium	● (few milliseconds)	●	●	◐	○
Cai et al. [116]	2012	Damerau-Levenshtein distance and Hidden Markov Model	Packet size, time, and direction	Medium	●	●	●	●	○
Wang et al. [205]	2014	KNN	A large feature set generated from packet-level traffic	Medium	● (~0.1 secs)	●	●	●	○
Hayes et al. [206]	2016	Random decision forests	Features selected by gini coefficient	Low	● (~0.1 secs)	●	●	●	◐
Rimmer et al. [207]	2017	SDAE, CNN, and LSTM	Automatically learned feature sets from packet-level network traffic	High	● (<5 secs)	●	●	●	○
Sirinam et al. [208]	2018	CNN	Packet-level traffic data	High	●	●	●	●	○
Sirinam et al. [209]	2019	N-shot learning with triplet networks	Selected by a neural-network-based feature selector	High	●	●	●	●	○
Yin et al. [25]	2021	Split point finding and BalanceCascade-XGBoost	Packet size, time, and direction	Low	● (~2-6 secs)	●	●	●	●
Wang et al. [210]	2022	DNN ensemble	Total number of packet, incoming packets, outgoing packets, and the ratio of quintuplets, along with the DAE feature vector.	High	◐ (~5 mins)	●	●	●	○
Deng et al. [211]	2023	A multi-classifier framework based on a novel transformer model	CNN-based local feature extraction	High	● (0.26-0.42 secs)	●	●	●	●

They proposed an attack that simply uses the transmitted data volumes to identify certain websites. Although this attack is not feasible anymore after the launch of connection pipelining and connection parallelization by HTTP 1.1 (RFC 2616 [212]), this research inspired many other WFP studies in the next two decades. In 2002, Hintz [213] defined “fingerprints” of websites as the histograms of transferred files’ sizes. He recorded some website fingerprints and successfully recognize some websites transferred through HTTPS with these fingerprints. However, Hintz’s WFP attack only works for a small number of websites. Later, Sun et al. [201] extends size-based WFP to thousands of websites. They proposed a WFP approach based on Jaccard’s coefficient, which can correctly identify 75% of the websites in their collected dataset. However, a common drawback of file-based attacks is that they cannot tackle traffic hidden in encrypted tunneling protocols (e.g., VPN, OpenSSH), not to mention Tor.

2) *Defeat encrypted tunnel*: To extend WFP to handle encrypted tunneling protocols, multiple “more advanced” WFP approaches had been proposed. Both Bissias et al. [115] and Liberatore et al. [202] proposed improved forms of WFP. Rather than using the data size as the feature, they extract sets of traffic patterns from encrypted IP packet headers, such as packet inter-arrival time, size, etc. These approaches have some efficacy in identifying websites transferred by encrypted tunneling services. However, the accuracies of page identification is still not ideal in reality. In 2009, by using packet-level features, Herrmann et al. [203] proposed a multinomial Naïve Bayes classifier that can identify up to 97% of web requests on a sample of 775 sites and over 300,000 real-world traffic dumps recorded over a two-month period. The authors demonstrate that this approach is effective in tackling website traffic in encrypted tunnels. Lu et al. [214] pointed out that packet ordering information, though noisy, can be

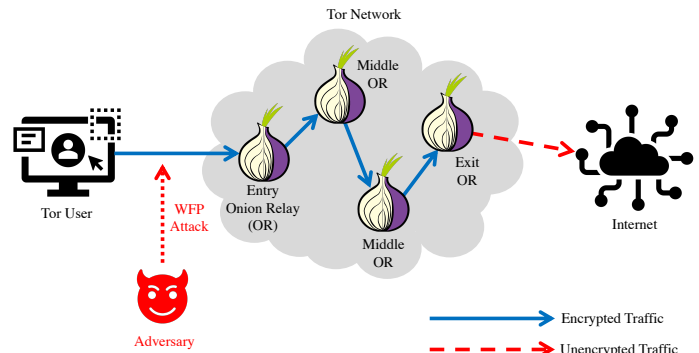


Fig. 9: Threat model for WFP attacks over Tor network.

utilized to enhance website fingerprinting. In addition, the ordering information is effective for WFP even under traffic morphing. By calculating the Levenshtein distance between different network traffic, their approach can perform WFP over OpenSSH for 2,000 profiled websites. The identification accuracy of the proposed scheme reaches 81%, which is 11% better than the approach proposed by Liberatore et al. [202].

3) *WFP in Tor era*: To safeguard personal information and avoid Internet censorship in an increasingly dangerous network environment, many people began to use The Onion Router (Tor), a free and open-source software for enabling anonymous communication, to visit the Internet. Different from traditional encrypted tunneling protocols, Tor reroutes Internet traffic through a worldwide, volunteer overlay network, consisting of more than six thousand relays [215], for concealing a user’s actual location and Internet usage from anyone conducting network surveillance or TA.

Figure 9 illustrates the operation model of Tor. To protect user’s identity, each Tor user creates an encrypted virtual

tunnel to its destination through a chain of several volunteer nodes—onion relays (ORs). According to their positions in the virtual tunnel, ORs can be classified into entry OR, middle OR, and exit OR. Each of the ORs only knows its predecessor and its successor [216]. When forwarding network traffic, the user's network packets will be encrypted in multiple layers and each of the ORs can only decrypt one layer of encryption. Thus, Tor ensures that none of the ORs in the circuit knows the user and its destination at the same time. Besides, to prevent TA, the user data is encapsulated in chunks of a fixed size, called cells, before transmission [217]. The WFP attacks above are thus ineffective against Tor network, as they rely heavily on packet-size-related features.

Indeed, it is almost impossible to find any useful knowledge inside a Tor network. However, the virtual tunnel between the Tor user and the entry OR does provide attackers with an interface and makes WFP possible (illustrated in Figure 9).

In 2011, Panchenko et al. [204] are the first to demonstrate that it is feasible to use WFP to identify web pages visited by Tor users. They trained a SVM classifier with features extracted from volume, time, and direction of network packets, with a classification accuracy of 55% when testing with their web page dataset. Panchenko et al. are also the first to evaluate their WFP attack in a real-world setting. The result shows that their approach is able to achieve a true positive rate of up to 73% and a false positive rate of 0.05%. Based on this work, a significant amount of improved WFP approaches were proposed to use different algorithms and features (e.g., VNG++ [218], Hidden Markov Models [116], Levenshtein-like distance [219], etc.) to tackle web page identification in Tor. In 2014, Wang et al. [205] proposed a KNN WFP classifier and applied it on a large feature set with weight adjustment. Their approach achieved an accuracy of 91% in a closed-world setting and a true positive rate of 85% for a false positive rate of 0.6% when testing with more than 5,000 background pages in a real-world setting.

Nevertheless, these WFP approaches still have some obvious flaws according to an evaluation made by Juarez et al. [220]:

- Previous WFP attacks assume single-tab browsing behavior of users. However, multi-tab browsing is widely used in reality.
- WFP attacks highly depend on the coverage of training dataset, but existing datasets cannot include web page traffic from all versions of Tor browser, user habits, or user locations.
- Previous WFP attacks cannot detect dynamic or personalized web pages, as they traffic of these pages is polytropic.
- Many countermeasures for WFP have been proposed (which will be discussed later in Section VII), making many of the previous WFP attacks non-effective.

To further increase the success rate of WFP attacks and defeat countermeasures, researchers began to collect more comprehensive training datasets, use more complicated feature sets, and apply more sophisticated classification algorithms for WFP.

Wang et al. [219] described how they collect the training dataset in a much more thorough manner than previous works. They gathered the data in different Tor settings and with different defense approaches. Later, Panchenko et al. [221] collected the first Internet-scale WFP dataset to develop and evaluate WFP comprehensively. Based on the dataset, they proposed CUMUL, a web page classifier that has a higher recognition rate and a smaller computational overhead than previous approaches. They also demonstrated that although CUMUL is more efficient and superior in terms of detection accuracy, still, it cannot scale when applied in realistic settings. As for WFP feature set, Cai et al. [222] systematically analyzed previous WFP approaches to understand which traffic features convey the most information; Hayes et al. [206] utilized the gini coefficient index to select a feature set and designed a random decision forests classifier based upon them; Wang et al. [223] evaluated the classification accuracy of each feature category by using KNN.

In the recent five years, the development of WFP has been focusing on conducting attacks in the presence of effective countermeasures, with little encrypted data, or under complicated circumstances [25], [210], [211]. Many of recent approaches also investigated the applicability of deep learning techniques in WFP. Rimmer et al. [207] trained three classification model with Stacked Denoising Autoencoder (SDAE), CNN, and Long Short-Term Memory (LSTM) respectively. These deep learning models are capable of automatically learning the best features to conduct WFP. The authors further demonstrated that automatically-created features are more effective especially in tackling constantly changing web content. In 2018, Sirinam et al. [208] presents a very powerful WFP attack—Deep Fingerprinting (DF). By employing a CNN model with a sophisticated architecture design, the authors claim that this attack can defeat many WFP countermeasures (e.g., WTF-PAD [224] and Walkie-Talkie [223]) and works well in very complicated real-world scenarios (95% accuracy for 20,000 URLs in a real-world setting). Sirinam et al. [209] further proposed an approach based on N-shot learning with triplet networks in 2019, which can achieve decent efficacy with relatively small training data. Besides these approaches, Abe et al. [225] also applied SDAE in WFP; Bhat et al. [226] leveraged ResNets [227], a CNN architecture, to reach high success rates in WFP; Oh et al. [228] used unsupervised DNN to generate low-dimensional features and trained different machine learning classification models based upon them. In 2021, Wang et al. [229] leveraged adversarial domain adaption (a transfer learning technique) to achieve high WFP accuracy with little encrypted data; Yin et al. [25] proposed a WFP attack that is able to identify websites in multi-tab environments, which means it can achieve usable accuracies regardless of the number of simultaneously opened web pages; Hoang et al. [230] found that even in the presence of domain name encryption technologies or content delivery network (CDN), WFP based on IP addresses is still feasible. They exploited the complex structure of most websites, which load resources from several domains besides their primary one, and further applied the generated domain fingerprints to conduct WFP at large.

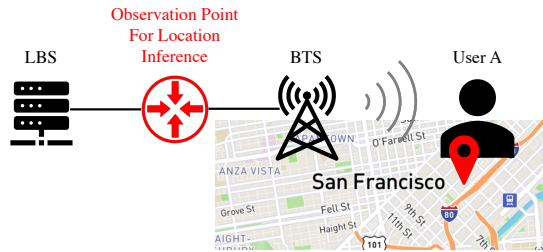


Fig. 10: Operation model for geographical location inference.

D. Location Inference

Location inference is a widely studied topic by computer scientists. We have seen myriad works focusing on using social network information [231], [232], smartphone accelerometer [233], image content [234], etc. to infer users' locations. Over the past decade, some researchers have begun utilizing FGTA for location inference, which involves extracting and analyzing location-related features from network traffic generated by relevant applications. The location we discuss here can be either a geographical location or a contextual location, the later one means the type of location the user is sending packets from, such as an airport, a campus, or a residential building. This subsection examines inference approaches for these two types of locations.

1) *Contextual location inference*: The intuition behind contextual location inference with FGTA is straightforward—users from different types of locations tend to generate different traffic because they need to use different web applications at different locations. Besides, different locations (e.g., campus, company, residential area) may process network traffic in different manners. Contextual location inference using FGTA aims to measure and analyze sets of network traffic and infer where these sets of traffic are coming from.

Back in 2009, Trestian et al. [235] conducted a detailed study on applications accessed by users at different locations. They demonstrated that users are more likely to show interest in a particular class of applications than others at certain locations, which is irrespective of the time of day. They indicated that we can further use the traffic generated by these applications to identify the type of locations (e.g., work versus home). In 2014, Das et al. [154], [236] collected around 100 GBs of real-world network traffic from more 1700 users at different types of locations (e.g., cafeteria/restaurant, university campus, airport/travel, etc.). By measuring and analyzing this dataset, Das et al. selected sets of features for packet-level, flow-level traffic and built a decision-tree-based classification model to predict contextual location with an overall accuracy of 87%. Later, a few similar works also demonstrated that mobile traffic from different cellular towers [237]–[239] tends to have different characteristics.

The drawback of contextual location inference is that it only works on a group of network traffic sending from many endpoints. It cannot infer a device's contextual location by only analyzing its own network traffic.

2) *Geographical location inference*: Purely using network traffic to infer a user's geographical location seems impossible.

However, in 2015, Ateniese et al. [27] demonstrated that it is actually feasible under certain assumptions.

Nowadays, location-based applications (LBA), such as Facebook, Yelp, Google Map, etc., are widely used. These LBAs obtain user locations through location-based services (LBS). LBS providers usually use a base transceiver station (BTS) to locate a user and send real-time location information to the user. Ateniese et al. proposed an approach (illustrated in Figure 10) that simply monitors the traffic between the BTS and the LBS to identify user locations. They found that different locations will trigger LBS packets of different sizes. An adversary can potentially create a location knowledge base of different locations' packet sizes and their corresponding timestamps to conduct geographical location inference. Still, this approach has many limitations (e.g., low accuracy, difficult to build the location knowledge base at large, etc.). This work is more about demonstrating the feasibility of geographical location inference with FGTA than launching a full-fledged, ready-to-use approach.

E. Device/OS Identification

Traditional TA has long been employed to identify user devices or the OSes running on them. For instance, in 2003, Lippmann et al. [240] focused on analyzing TCP and IP packet metadata to distinguish different OSes. However, these traditional approaches typically provided identification at a coarse granularity, such as determining the device type (e.g., smartphone, PC, server) or OS type (e.g., Windows, Linux, Android). As the variety and complexity of user devices and OSes have grown, the demand for more precise device and OS identification has significantly increased. Achieving this level of precision is challenging, as it often requires inferring subtle application-layer details from network traffic to identify specific device or OS models. To address these challenges, researchers have turned to FGTA, exploring whether specific traffic patterns can be correlated with particular OSes' or devices' application behaviors. This approach not only identifies broad device or OS types but also enables the pinpointing of specific device models or OS versions, particularly for diverse IoT and mobile devices. In this subsection, we introduce such FGTA approaches that deal with device/OS identification.

1) *OS identification*: Chen et al. [241] perform OS identification and detection of NAT and tethering (i.e., multiple devices sharing the Internet connection of a mobile device, which can lead to multiple OSes sharing a single IP address) by inspecting TCP/IP headers of packet traffic. They leverage a probability-based method by applying the Naïve Bayes classifier to effectively combine multiple features (e.g., TTL value, IP ID monotonicity, TCP timestamp, clock frequency, etc.), thereby fingerprinting and recognizing different OSes in different environments. Laštovička et al. [242] also proposed an OS identification method by constructing a decision tree with the TLS handshake, HTTP headers, and TCP/IP features. However, these approaches cannot distinguish between minor versions of the same OS. To tackle this problem, Ruffing et al. [243] identify different versions of smartphone OSes by using the frequency spectrum of packet timing from encrypted

traffic. By identification through correlations of the feature-extracted spectra, the authors demonstrate that even a network traffic input of 30 seconds can be enough for high-accuracy identification results.

2) *IoT device identification*: Compared with OS identification, IoT device identification can be more challenging due to the complexity of their network environments and the devices' wide variety. Lopez-Martin et al. [244] extract a time-series feature vectors from network traffic, where each element of the time-series vector contains the features of a packet in the flow. They then proposed a classifier that is based on both a RNN model and a CNN model to separate heterogeneous IoT traffic using the features. Meidan et al. [245] collected and labeled network traffic from nine distinct IoT devices (e.g., baby monitor, motion sensor, printer, security camera, etc.), PCs, and smartphones. They then utilized a multi-stage machine-learning-based classifier to classify traffic of IoT devices in two phases. In the first stage, the classifier can distinguish traffic between IoT and non-IoT devices. In the second stage, the classifier can further identify traffic from different IoT devices. The authors demonstrate that their approach is able to classify IoT traffic with an accuracy of 99.281%.

However, these two researches do not consider complicated network environments (e.g., smart homes, enterprises, and cities) of IoT devices. Sivanathan et al. [246] addressed this challenge by developing a robust framework for IoT device traffic classification with a multi-stage machine-learning-based algorithm. The authors instrumented a smart environment with 28 different IoT devices that consist of spanning cameras, lights, plugs, motion sensors, appliances, and health-monitors. They then collected and synthesized network traffic traces from this infrastructure for a period of six months. By extracting statistical features such as activity cycles, port numbers, signaling patterns, and cipher suites from the traffic and using Naïve Bayes and random forest as the identification models, they are able to classify heterogeneous IoT devices with an accuracy over 99%. Yao et al. [247] further proposed an end-to-end IoT traffic classification method that eliminates the multi-stage classification for high accuracy and efficiency. It relies on a deep-learning-aided capsule network to construct an efficient classification mechanism that integrates feature extraction, feature selection, and classification model. One drawback of these approaches is that their evaluations are all based on closed-world datasets, which may not be able to precisely reflect their true efficacy in the real world.

F. Application Identification

Using the network traffic from a device to identify the applications that are running on the device, even in the presence of traffic encryption, is one of the most classic use cases of TA. Decades ago, people have investigated using traditional TA approaches to coarsely classify traffic from different applications or application types. Before 2000, many researchers simply used traffic ports to identify some popular applications that have well-established ports (e.g., port 443 for HTTPS, port 110 for POP3). Port-based approaches fail for most emerging applications such as gaming, streaming, and messaging [248].

Later, Karagiannis et al. proposed BLINC [249], which not only looks at port-based features, but also inspects the host's social behavior and its community behavior to determine the applications. Bernaille et al. [250] observe the sizes of the first few packets of an SSL connection to identify the web application, which can achieve an accuracy of more than 85%. There are also many machine-learning-based traditional TA approaches [15], [251]–[253] that classify application traffic according to protocol-level traffic patterns.

However, application identification using traditional TA struggles to adapt to the modern network environment and distinguish between applications with similar purposes due to the following reasons:

- Traditional TA can primarily identify high-level protocols (e.g., HTTP, HTTPS, SMTP, POP3) and a limited set of commonly used applications with distinct traffic patterns (e.g., MySQL, BitTorrent, MSN). However, many modern applications, such as payment apps, share similar operational protocols and appear indistinguishable based on transport-layer features alone.
- Traditional TA lacks application-layer awareness, which is crucial for large-scale and fine-grained application identification.

Today's network environments are far more complex, and application ecosystems have become increasingly diverse and intricate. To address these challenges, researchers and developers have begun utilizing FGTA to analyze distinctions at or above the application layer, enabling precise identification of specific applications amidst diverse traffic from various devices. In this subsection, we collectively introduce FGTA approaches that serve the purpose of application identification (Table IX shows an overview).

1) *Application identification for general-purpose devices*: General-purpose devices, such as personal computers and servers, support the operation of countless web applications. Recently, FGTA-based application identification for general-purpose devices focuses on identifying more specific applications in more complicated network environments.

Chen et al. proposed Seq2img [254], an application traffic classification framework based on an online CNN model. Seq2img employs a data fusion method based on Reproducing Kernel Hilbert Space (RKHS) to convert flow sequences into images, which can fully capture the static and dynamic behaviors of different applications. Then, Seq2img utilizes a CNN model to recognize network traffic of popular applications, such as Facebook, Instagram, Wechat, etc.

Rezaei et al. [111] investigated using a few labeled, sampled packet-level datasets to train a comprehensive application identification model. They first pre-train a CNN-based model on a large unlabeled dataset, where the input is the time series features of a few sampled packets. Then, the learned weights are transferred to a new CNN model that is re-trained on a small labeled dataset. They demonstrated that this semi-supervised approach achieves almost the same accuracy as a fully-supervised method with a large labeled dataset. The proposed approach is able to identify applications like Google Drive, Google Doc, Google Search, Google Music, etc.

TABLE IX: Comparisons of selected FGTA approaches for application identification (○: not supported; ◐: partially supported; ●: supported).

Category	Approach	Year	Target Application	Traffic Feature	Method	System Overhead	Real-Time Analysis	Real-World Evaluation
App Identification for General-Purpose Devices	Chen et al. [254]	2017	Instagram, Skype, Facebook, Wechat, Youtube, etc.	Images converted from flow sequences	RKHS-based data fusion and CNN	High	◐	○
	Rezaei et al. [111]	2018	Google Drive, Youtube, Google Docs, Google Search, Google Music, etc.	Time series features extracted from sampled packets	Semi-supervised CNN	High	◐	○
	Lotfollahi et al. [120]	2020	Vimeo, YouTube, VoipBuster, Spotify, Netflix, Hangouts, Facebook, etc. (with or without VPN)	Normalized features extracted from packet headers	CNN and SAE	High	●	○
	Zhao et al. [255]	2024	Google Home, email, streaming, P2P, Tor, etc. (in complicated environments)	Protocol-agnostic per-packet feature sequences	Random convolution kernel transformations and meta-learning	High	● (few seconds)	○
Mobile App Identification	Wang et al. [256]	2015	Snapchat, Tencent QQ, Mint, Tinder, YouTube, etc.	Statistical features from packets (e.g., STD time, average size, STD size, etc.)	Random forest	Low	◐ (~300 secs)	◐
	Alan et al. [109]	2016	1595 applications on four different devices	Features from TCP/IP headers (e.g., packet size, timing, direction)	Jaccard's coefficient and Naïve Bayes	Medium	● (~2-16 secs)	◐
	Taylor et al. [257]	2016	110 most popular applications in Google Play Store	Two sets of features from flow-level traffic-flow vector and statistical features	SVM and random forest	Medium	● (2 secs-2 hours)	◐
	Aceto et al. [258]	2018	49 mobile applications (i.e., QQ, SayHi, eBay, 6Rooms, NetTalk, PureVPN, etc.)	Statistical features (e.g., packet length, percentiles, deviation, etc.) for incoming, outgoing, and bidirectional packets	A multi-classification (viz. fusion) model consists of Naïve Bayes, random forest, SVM, and decision tree	Medium	● (~5 secs)	○
	Aceto et al. [119]	2019	Facebook, Facebook Messenger, and other 49 apps on both Android and IOS	Automatically-extracted features using neural networks	Multiple machine learning models (e.g., CNN, LSTM, MLP, etc.)	High	● (few seconds)	○
	Van et al. [20]	2020	More than 1M apps from three datasets	Packet and flow-level features selected by adjusted mutual information	A semi-supervised fingerprinting with destination-based clustering, browser isolation, and pattern recognition	High	◐ (~300 secs)	○
Decentralized App Identification	Pham et al. [259]	2021	101 popular Android apps	63 features extracted from packets and flows, including aggregate, statistical, temporal, categorical features.	Deep graph convolution neural network	High	◐ (≤5 mins)	◐
	Shen et al. [104]	2019	Aragon, Bancor, Canwork, Chainy, Cryptopepes, Eth_town, Etheremon, etc.	57 features of packet lengths, 72 features of bursts, and 54 features of time series, fused by kernel functions	KNN, SVM, and random forest	Medium	◐ (few minutes)	◐
	Aioli et al. [260]	2019	BTC.com, BitPay, Bread, Wirex, Copay, etc.	Vectors of statistical features about the packet length from traffic flow	SVM and random forest	Medium	◐	◐
	Shen et al. [261]	2021	Closed-World setting: the top 40 DApps on Ethereum; Open-World setting: randomly selected 1,260 DApps on Ethereum.	Traffic Interaction Graph, which is capable of reserving information such as packet direction, length, ordering, and bursts.	Graph neural network (GNN) with MLP	High	◐	◐

In 2020, Lotfollahi et al. [120] proposed an application identification method that can work in both VPN and non-VPN networks. After extracting features from packet headers, they used both CNN and stacked autoencoder (SAE) to train the classification models. Evaluation results show that this approach can achieve a recall score of 0.98 in application identification tasks.

2) *Mobile application identification*: With the raising of mobile network, mobile application identification becomes an emerging research topic in recent years. Unlike general-purpose devices, mobile devices are less regularized in port usage. In addition, a wide variety of mobile applications may utilize some common libraries in communication, generating similar network traffic patterns. Thus, mobile application identification can be more challenging.

Wang et al. [256] use random forest algorithm to analyze packet-level traffic in wireless networks. Their approach is able to detect the usage of 13 selected popular mobile applications on IOS platform, such as Snapchat, Tencent QQ, Mint, Tinder,

YouTube, etc., with an accuracy of more than 87.23%. They demonstrate that by using the mobile applications the privacy of the user is more at risk compared to using online services through browsers on mobile devices.

Many researchers also studied application identification on Android platform. Inspired by some WFP approaches (Section V-C), Alan et al. [109] use Jaccard's coefficient and Naïve Bayes to analyze features (e.g., packet size, timing, direction) from TCP/IP headers to identify 1595 applications on four different devices. Taylor et al. [257] proposed AppScanner, a framework that can automatically fingerprint and identify Android applications from their encrypted network traffic. The authors extracted two sets of features (i.e., flow vector and statistical features) from flow-level network traffic and implemented this approach using both SVM and random forest algorithms. The evaluations show that AppScanner can identify the 110 most popular applications in Google Play Store with more than 99% accuracy. In the next year, Taylor et al. further extended AppScanner in a follow-up research [262].

They investigated how application fingerprints change over time, across different devices, and across different application versions.

Recently, many similar works (e.g., [20], [119], [258], [263]) have been proposed to enhance the efficacy, efficiency, and coverage of mobile application identifications.

3) *Application identification on other platforms:* A few researches have been focusing on identifying decentralized applications on blockchain systems. Shen et al. [104] proposed an encrypted traffic classification of decentralized applications (e.g., Cryptopepes, Matchpool, Lordless, etc.) on Ethereum with features like packet lengths, bursts, and time series. Aioli et al. [260] focused on identifying user activities on Bitcoin wallet applications (e.g., BTC.com, Bitcoin Wallet, Coinbase, etc.). The authors used SVM and random forest models to conduct the identification.

We also studied the application identification approaches for IoT devices. However, as each IoT device is usually bundled with an IoT application, the identification of IoT application is equal to the identification of IoT devices in most cases. Therefore, we introduce these approaches in Section V-E (IoT device identification).

G. Application Usage Inference

Application usage inference aims to analyze encrypted network traffic to identify certain application events, infer user behaviors, and measure specific service usage. It is one of the most challenging FGTA tasks, as it not only classifies the network traffic that is associated with different applications, device, or web pages, but also leverages the traffic patterns to recognize the application-layer activities that users conducted with the applications, devices, or web pages. Therefore, many application usage inference approaches may take extra steps (e.g., clustering, pre-filtering, etc.) to narrow down the scope before the final traffic classification. Besides, they need to perform traffic segmentation to locate different traffic bursts, where each burst represents a group of adjacent packets that support an application event.

In this subsection, we introduce representative application usage inference approaches, demonstrating their applicable scenarios and methodologies (Table X shows a comparison).

1) *Messenger/Online social network usage inference:* User activities on messaging or OSN applications are very private and sensitive. However, although being encrypted, a third party can still infer the rough messaging/OSN activities that users have performed only through content-agnostic network traffic data.

Back in 2009, Schneider et al. [270] investigated OSN usages from the perspective of network traffic for four different platforms—Facebook, LinkedIn, Hi5, and StudiVZ. The authors studied how users actually interact with OSNs by extracting clickstreams from passively monitored network traffic. They found that different OSN operations (e.g., login, open friend list, logout, select profile, etc.) will trigger statistically different network traffic. This research later led many researchers to dig deeper into using the traffic differences to classify different user actions on OSNs. Coull et al. [264]

analyzed the network traffic of encrypted messaging services such as Apple iMessage. The authors demonstrated that an eavesdropper can learn information about user actions (e.g., control, read, start, stop, image, and text), the language of messages, and even the length of those messages with greater than 96% accuracy simply by observing the sizes of encrypted packets. They used three algorithms to perform the inference—linear regression, Naïve Bayes, and rule lookup table. However, they only evaluated their approach in closed-world environments with a small dataset. Fu et al. [117] extended the inference to more messaging applications (i.e., Wechat and WhatsApp) and more activities (e.g., stream video call, news feed, location sharing, etc.). By segmenting Internet traffic into sessions with a number of dialogs, extracting discriminative features from the perspectives of packet length and time delay, and leveraging multiple machine learning models to conduct the classification, the proposed approach can achieve 96% and 97% accuracy in WeChat and WhatsApp respectively. Liu et al. [26] further extended the inference coverage to more OSN applications (e.g., Facebook, Wechat, and WhatsApp) and evaluated their approach in a real-world environment with real-time traffic data streaming. Real-world evaluation is essential to reveal the true performance and efficacy of application usage approaches, but many approaches were only evaluated through closed-world off-line cases, leaving the inference throughput and abilities to handle noise mysteries. Feng et al. [103], [271] developed and evaluated their OSN usage inference approach in a larger network environment—a campus network. Although their approach is mainly built for social bot detection, it can identify some commonly seen user activities (i.e., posting, reading, liking, etc.) on Twitter and Facebook.

2) *Streaming service usage inference:* There are a few works focusing on leveraging FGTA to extract behavioral information from network traffic of streaming service (e.g., VoIP, audio streaming, and video streaming). Researchers have demonstrated the feasibility of revealing voice information from encrypted VoIP conversations or identifying encrypted video streams [6].

Wright et al. [265] demonstrated that when the audio is encoded using variable bit rate codecs, the lengths of encrypted VoIP packets can be used to identify the phrases spoken within a call. By leveraging a HMM, the authors indicated that an eavesdropper can identify phrases from a standard speech corpus within encrypted calls with an average accuracy of 50%, and with accuracy greater than 90% for some phrases. Schuster et al. [22] demonstrated that many video streams are uniquely characterized by their burst patterns, and classifiers based on CNN models can accurately identify these patterns given very coarse network measurements. The authors only extracted features from flow attributes, such as inbound/outbound bytes per second, inbound/outbound packet per second, and inbound/outbound average packet length. They have examined this approach on Netflix, YouTube, Amazon, and Vimeo.

3) *General-purpose application usage inference:* The approaches discussed in this subsection aim at inferring all types of application-layer events rather than only recognizing certain event categories.

TABLE X: Comparisons of selected application usage inference approaches (○: not supported; ◐: partially supported; ●: supported).

Category	Approach	Year	Analysis Object	Feature	Method	System Overhead	Real-Time Analysis	Real-World Evaluation
Messenger/OSN	Coull et al. [264]	2014	Apple iMessage: language, control, read, start, stop, image, text, etc.	Payload length and the message length; a binary feature vector of packet length and direction pairs	Linear regression, Naïve Bayes, and rule lookup table	Medium	● (few seconds)	○
	Fu et al. [117]	2016	Wechat and WhatsApp: stream video call, news feed, location sharing, etc.	Discriminative features from the perspectives of packet length and time delay	Traffic segmentation with hierarchical clustering and thresholding heuristics; HMM-based classifier.	Medium	◐ (few minutes)	◐
	Liu et al. [26]	2017	Facebook, Wechat, and WhatsApp: short video, video call, text, picture, etc.	A selected feature set extracted from traffic packet sequences by a Maximizing Inner activity similarity and Minimizing Different activity similarity measurements.	A recursive time continuity constrained K-means clustering algorithm for traffic flow segmentation and a random forest classifier for segmented traffic classification.	Medium	◐	●
	Feng et al. [103]	2021	Facebook and Twitter: post, chat, read, etc.	Images converted from NetFlow records	Clustering-based traffic segmentation; CNN	High	◐ (~1-5 mins)	●
Streaming Service	Wright et al. [265]	2008	Identify the phrases spoken within a call from a standard speech corpus.	The lengths of encrypted VoIP packets	HMM	Medium	●	○
	Schuster et al. [22]	2017	Identify the videos streamed by YouTube, Netflix, Amazon, and Vimeo.	Time series data of the following flow attributes: down/up/all bytes per second, down/up/all packet per second, and down/up/all average packet length.	Time-based burst; CNN	High	◐ (few minutes)	○
General-Purpose	Conti et al. [266]	2015	User activities in Gmail, Facebook, Twitter, Tumblr, Dropbox, etc.	Features from TCP/IP packet fields (e.g., IP address, port number, packet size, direction, and timing)	Dynamic time warping, random forest, and a hierarchical clustering algorithm called agglomerative	Medium	● (~5 secs)	○
	Saltaformaggio et al. [267]	2016	User activities on Android and IOS platforms	Features extracted from IP packet headers, divided by behavior measurements (a small time window)	A K-means clustering model and an SVM model	Medium	● (0.25-1.5 secs)	◐
	Papadogiannaki et al. [19]	2018	User activities (e.g., voice call, video call, messaging, etc.) in popular Over-The-Top mobile applications (e.g., WhatsApp, Skype, Viber, etc.)	Customizable	A pattern language to identify application events, rule mining	Low	● (few seconds)	●
Others	Yan et al. [268]	2018	Red packet transactions and fund transfers in Wechat	Overall statistics, packet length, number of TCP handshakes, inbound and outbound statistics	Threshold-based traffic segmentation, random forest	Medium	● (~6 secs)	○
	Wang et al. [269]	2019	Classify specific actions (e.g., transfer payment, transfer receipt, QR code payment, etc.) on the mobile payment application, and then detect the detailed steps (e.g., click the button, receive the fund, open the red packet, etc.) within the action	Overall statistics of the packet length, range statistics of the packet length, flow statistics, incoming and outgoing statistics.	Threshold-based traffic segmentation, hierarchical identification with random forest, AdaBoost, GBDT, and XGBoost	Medium	◐ (~1 min)	○
	Jiang et al. [114]	2019	Application usage information (e.g., reading documents, surfing webs, editing documents, etc.) on remote desktop	Statistic features of flow burst	Threshold-based traffic segmentation, logistic regression, SVM, GBDT, random forest	Medium	◐	○
	Wang et al. [118]	2020	Identify DApp (e.g., Superrare, Editorial, John Orion Young, etc.) user behaviors (e.g., open DApps, open market, view detail, etc.)	Selected DApps features, behavior-sensitive features, and improved inter-arrival time series	Random forest, decision tree, and GBDT	Low	● (few seconds)	○

Conti et al. [266], [272] analyzed encrypted mobile traffic to infer user actions on Android devices, such as email exchange, posting a photo online, publishing a tweet, etc. They extracted features from TCP/IP packet fields (e.g., IP address, port number, packet size, direction, and timing) and use a random forest to perform the inference. They trained and evaluated their approach using a dataset that is associated with several Android applications with diverse functionalities, such as Gmail, Facebook, Twitter, Tumblr and Dropbox. The evaluation results demonstrate that it can achieve more than 95% of accuracy and precision for most of the actions

within the dataset. However, this approach was not evaluated in real-world environments. In 2016, Saltaformaggio et al. [267] proposed NetScope, a framework that can perform robust inferences of user activities for both Android and IOS devices by only inspecting IP packet headers. NetScope leverages a K-means model and an SVM model to learn and detect network traffic generated by different application behaviors. By testing the approach in a lab environment, the authors demonstrated that despite the widespread use of fully encrypted communication, NetScope can distinguish subtle traffic behavioral differences between user activities (e.g.,

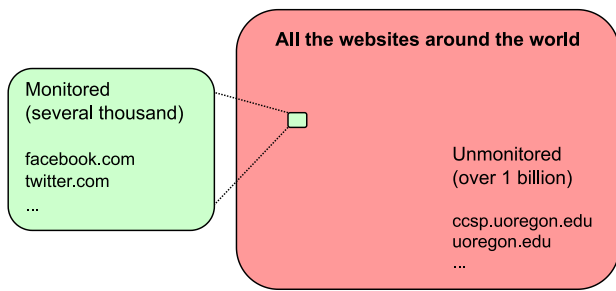


Fig. 11: Training data coverage for WFP.

Instagram browse versus post, Yelp browse versus search, Facebook feed versus post, etc.). Papadogiannaki et al. [19] further pushed application usage inference to a much larger scale. They proposed OTTer, a highly scalable engine that identifies fine-grained user actions (e.g., voice call, video call, messaging, etc.) in popular Over-The-Top mobile applications, such as WhatsApp, Skype, Viber, and Facebook Messenger with encrypted network traffic connections. By evaluating OTTer is a real-world test bed, the authors demonstrated that it can operate at traffic loads with an average of 109 Gbps.

4) *Others*: There are a few application usage inference approaches tackling different problems. For instance, Yan et al. [268] segmented the network traffic into several bursts and trained a random forest model to identify red packet transactions and fund transfers in Wechat; Wang et al. [269] proposed an approach to identify the mobile payment applications from traffic data, then classify specific actions (e.g., transfer payment, transfer receipt, QR code payment, etc.) on the mobile payment application, and finally, detect the detailed steps (e.g., click the button, receive the fund, open the red packet, etc.) within the action; Jiang et al. [114] studied encrypted remote desktop traffic and found that an eavesdropper can reveal application usage information (e.g., reading documents, surfing webs, editing documents, etc.) due to side-channel privacy leakage; Wang et al. [118] aimed at identifying DApp (e.g., Superrare, Editorial, John Orion Young, etc.) user behaviors (e.g., open DApps, open market, view detail, etc.) on Ethereum by using random forest, decision tree, and gradient boosting decision tree (GBDT).

VI. LIMITATIONS

While FGTA approaches appear potent in deducing diverse high-level, nuanced behaviors, it is important to acknowledge their limitations. In practice, many FGTA approaches often fall short of their theoretical promises, with their effectiveness contingent on a multitude of conditions. This section is dedicated to a thorough discussion of the inherent limitations of FGTA.

A. Coverage of Training Data

As most FGTA approaches are based on machine learning algorithms or prior knowledge about specific traffic, the efficacy of such approaches is highly dependent on the coverage of training datasets or rules learned beforehand. Unfortunately, existing datasets or rules can only represent a small fraction

of real-world scenarios. It is actually impossible to collect a dataset to cover all possible scenarios. Take the WFP attack discussed in Section V-C as an example, as shown in Figure 11, state-of-the-art datasets from public repositories can only cover less than 0.001% of all websites around the world. FGTA approaches built upon such datasets then have little effect in practice. Furthermore, network traffic of websites, applications, or OSes is dynamic. For instance, the layouts of Facebook websites have been changed for several times since its birth, and so has the network traffic associated with Facebook. Therefore, a FGTA approach that worked before may no longer be effective, if we do not update its classification model with the latest training datasets.

B. Uncertainties in Real-world Environments

As can be seen from our previous discussion (Section IV-D and Section V), many approaches were only evaluated in closed-world environments, which means they were only tested with a small amount of labeled traffic, with a little noise or without noise. Such closed-world evaluations cannot objectively reveal the efficacy of proposed approaches in the real world. Network traffic in real-world environments can be very quite different from traffic in laboratory environments:

- Real-world network configurations can be complicated, with traffic going through network address translation (NAT)s, Wi-Fi connections, or special middle boxes. All these factors can significantly change the original traffic characteristics.
- Edge users have different habits of using web applications. Some may send traffic with VPN, Shadowsocks, or Tor. Although many FGTA approaches claim to be effective even with traffic tunneling techniques, many researchers found their efficacy will actually be reduced under such circumstances [273].
- The ratio of different network traffic in the real world is different from that in the laboratory environment, making accuracies obtained from closed-world evaluations hardly representative.

Therefore, real-world evaluations or large-scale pilot studies are essential for developing and polishing a usable FGTA approach.

C. False Alarms

FGTA aims to identify specific types of user activities from network traffic. Usually, the analysis object only occupies a very tiny proportion of the whole traffic (e.g., less than 0.01%). Thus, a very small false positive rate can be amplified in deployment when facing massive traffic, making the proposed FGTA approach hardly usable. For example, if a packet-level FGTA approach has a false positive rate of 0.1%, then it will generate around 100 false alarms for every 100,000 negative packets. Even for a small ISP, its traffic volume can easily reach 100,000 packets in less than 1 second. Therefore, the FGTA approach will generate around 100 false alarms every second, which is unacceptable for most network administrators. In real-world scenarios, an excess of false alarms can

either cause too many collateral damages or force network administrators to ignore all alarms, which will make the FGTA approach useless.

D. Integrity of Network Traffic

As discussed in Section III, guaranteeing the integrity of real-world network traffic data presents significant challenges. Various factors contribute to this uncertainty. For instance, traffic routing often exhibits asymmetry, resulting in the collected data representing predominantly one-directional information. Furthermore, traffic data is frequently sampled, driven by performance optimization or storage constraints. Additionally, packet loss, which can occur due to network congestion or other factors, further compromises the reliability of the traffic data. This compromised data integrity inevitably diminishes the effectiveness of current FGTA methodologies. In many cases, it can render them ineffective, as they are typically trained and tested on the assumption of complete and unaltered traffic data. Such a limitation is especially severe for FGTA approaches that are based on deep learning algorithms as they are very sensitive to the integrity of training data.

E. Traffic Obfuscation

The previous subsection explored the impact of network traffic data integrity and correctness on the efficacy of FGTA approaches. This vulnerability can be exploited by adversaries to camouflage their network activities and circumvent FGTA screening. Adversaries might employ various obfuscation techniques, such as introducing noise (e.g., dummy packets) into their traffic, deliberately delaying packets, or manipulating packet aggregation. These tactics are not only straightforward for adversaries to implement, but they also tend to be highly effective in evading detection [205], [223], [274].

We delve into these traffic obfuscation strategies in greater detail in Section VII, offering a comprehensive analysis of their implementation and effectiveness.

F. Performance Overhead

In our exploration of various FGTA approaches, particularly those leveraging advanced techniques such as machine learning and high-dimensional clustering, we have identified significant computational overhead as a primary limitation. This overhead arises from the complexity and computational demands of these methods, which can be attributed to the following factors:

- The traffic capture engine introduces considerable overhead to the network infrastructure, particularly in systems lacking hardware acceleration support.
- Sophisticated analysis techniques, such as deep learning algorithms, impose substantial computational demands.
- The memory and storage requirements for storing, buffering, and processing large volumes of network traffic data are often significant.

To mitigate these performance overheads, the following optimization directions can be employed:

- Upgrading the network infrastructure to support high-speed traffic capture and processing. Employing hardware acceleration, such as GPUs, FPGAs, or programmable switches, to enhance the computational efficiency of the analysis engine.
- Optimizing the analysis algorithms to reduce computational complexity and memory requirements.
- Separating the traffic collection and analysis processes to distribute the computational load. For example, network service providers can outsource the traffic analysis to cloud-based services, such as AWS or Kentik, to deploy FGTA solutions without upgrading their infrastructure.

Additionally, the nature of the input traffic data, in terms of granularity or volume, plays a crucial role in the performance of FGTA approaches. For instance, analyzing packet-level input typically demands more computational resources compared to processing flow-level data, given the same throughput. This is because packet-level analysis entails a more detailed examination of each data packet. While adopting flow-level input for analysis can mitigate some of this computational burden, it comes with trade-offs. Primarily, it tends to diminish the accuracy of FGTA methodologies. Moreover, certain FGTA tasks are infeasible at the flow level due to the less granular nature of the data. This reduction in granularity potentially limits the scope of analysis and the depth of insights that can be derived. Therefore, the choice of input data granularity is a trade-off between computational overhead and analysis granularity.

G. Scalability

As network traffic continues to grow exponentially, scalability emerges as a significant challenge for FGTA methodologies. The task of analyzing and processing such immense volumes of data can overburden computational resources, thereby impeding the ability to conduct real-time analysis. This challenge is particularly acute for FGTA strategies reliant on deep learning algorithms, known for their intensive computational demands. Consequently, deploying FGTA at an internet-wide scale remains unfeasible. Presently, only organizations with substantial computational capabilities and budgets can afford to implement FGTA for analyzing segments of their network traffic, serving specific objectives. Looking ahead, it's an unresolved question whether the scalability of FGTA will be able to keep pace with the relentless growth of network traffic.

VII. COUNTERMEASURES

The growing capabilities of FGTA have brought about significant advancements in network monitoring, intrusion detection, and user behavior analysis. However, these same capabilities pose severe privacy risks if misused. Attackers can exploit FGTA to infer sensitive user activities, preferences, and even identities, undermining both individual privacy and organizational security. The increasing prevalence of encrypted traffic further exacerbates these concerns, as FGTA techniques can bypass traditional privacy-preserving measures by analyzing metadata and traffic patterns. Given these risks, it is

TABLE XI: Comparisons of selected well-known FGTA countermeasures (*None*: 0; *Low*: 0-30%; *Medium*: 30%-60%; *High*: more than 60%).

Category	Approach	Usage Scenarios	Time Overhead	Bandwidth Overhead	Additional Requirements
Network-Layer	Pinheiro et al. [275]	All web applications	None	Medium	Middlebox and SDN controller
	FRONT and GLUE [276]	Tor	None	Low	None
	Traffic morphing [277]	All web applications	None	Low	Knowledge about other traffic classes
	Walkie-Talkie [223]	Web browsing	Medium	Low	Knowledge about some web traffic
	WTF-PAD [224]	Web browsing	None	Low	None
	Liberatore et al. [202]	All web applications	None	High	None
	BuFLO [218]	Web browsing	High	High	Network Transfer with fixed rates
	TrafficSliver [216] (network-layer mode)	Tor	None	None	Multiple entry ORs in Tor network
Application-Layer	HTTPOS [278]	Web browsing	None	Low	None
	TrafficSliver [216] (L7 mode)	Tor	None	None	Multiple entry ORs in Tor network
	LLaMA and ALPaCA [279]	Tor	Server-side: Medium; Client-side: Low	Server-side: Medium; Client-side: Low	None

imperative to explore and implement effective countermeasures to prevent the misuse of FGTA. This section offers an in-depth analysis of such countermeasures, evaluating their effectiveness and exploring diverse application scenarios.

In Section VI, we delved into the limitations of FGTA approaches. It became evident that the integrity and accuracy of network traffic data play crucial roles in determining the effectiveness of FGTA strategies. Based on this idea, internet users, whether with legitimate or illegitimate intentions, can employ a variety of data modification tactics to evade the screening by FGTA systems. Illegitimate users might use these methods to remain undetected while engaging in malicious activities, whereas legitimate users might adopt them to disrupt FGTA and safeguard their privacy.

Naïve countermeasures send individual or aggregated traffic through encrypted channels to escape the inferences of traditional TA approaches, such as VPN, Shadowsocks, and Tor. However, these approaches are proven to be vulnerable to many FGTA approaches [280]–[283]. Therefore, people began to modify the features of traffic flows to perturb FGTA approaches' classification models. Such perturbations can be conducted from either network layer or application layer [216]. Table XI shows a comparison of some well-known countermeasure approaches, where the time overhead and bandwidth overhead are assessed based on the results reported in the original papers as well as our analysis of the methodological details of these approaches.

A. Network-layer Countermeasures

Network-layer FGTA countermeasures directly modifying the network traffic by adding padding packets, changing packet bytes, or delaying existing packets, thereby obfuscating specific features that FGTA approaches rely on, making the current traffic look like other activities', or regularizing the traffic patterns of different applications [276]. Such approaches usually come with some side effects. They might increase the overheads of the network system, including time overhead, bandwidth overhead, and potentially computational overhead.

Among all the network-layer countermeasures, traffic obfuscation is the most classic approach. Back in 2006, Liberatore et al. [202] leveraged per-packet padding (i.e., increasing the bytes of packets) in an attempt to defeat host profiling system. They found that per-packet padding is reasonably effective, which can lower predictive accuracy to less than 8% with a cost of increasing traffic volume by 145%. However, per-packet padding cannot defend against many WFP attacks [206], [218] because this approach still preserves some key traffic features that can help classify the traffic. To fix the drawbacks, WTF-PAD [224] extends per-packet padding to link-based padding to modify more traffic features. It detects large time gaps between packets and covers them by adding dummy packets. Further, to obscure traffic bursts, it also adds delays between packets to make them statistically different. Due to its low computational overhead and time overhead, WFP-PAD has been used in many real-world FGTA defense systems [284], [285]. Still, WTF-PAD leaks a portion of information in transmission and can be broken by some FGTA approaches [208], [286]. Gong et al. [276] proposed FRONT and GLUE. FRONT focuses on obfuscating the trace front with dummy packets. It also randomizes the number and distribution of dummy packets to impede the attacker's inferring process. GLUE adds dummy packets between separate traces so that they appear to the attacker as a long consecutive trace, making the attacker unable to find the start or end points.

Compared with traffic obfuscation that freely modifies traffic features, traffic confusion mimic other groups of traffic to let FGTA approaches generate wrong outputs, which is sometimes more effective, especially when defending against WFP attacks. Wright et al. proposed traffic morphing [277]. It can thwart statistical TA approaches by morphing one class of traffic to look like another class using convex optimizations. Although it cannot defend against some types of FGTA approaches [206], [218], this approach inspired many subsequent countermeasure approaches. For example, Glove [274] first leverages a clustering algorithm to group web pages with similar traffic, and then inserts only a small amount of dummy traffic to hide the web page traffic in a close

group; Supersequence [205] also clusters network traffic traces of different web pages and extracts the shortest common supersequence to cover current web traffic; Walkie-Talkie [223] modifies the browser to communicate in half-duplex mode (buffer traffic and send in bursts) rather than the usual full-duplex mode (immediately send available data). By combining with dummy packets, Walkie-Talkie can modify the traffic of monitored sensitive pages and benign non-sensitive pages, so that each page's packet sequences are exactly the same (each packet has the same timing, length, direction and sequence number). However, a traffic-confusion-based approach requires a priori knowledge about popular web pages' network traffic. It cannot tackle traffic of dynamic content or unpredictable activities. Moreover, such approaches can lead to noticeable computational overhead.

Another obfuscation direction is to regularize the network traffic, making different groups of traffic have relatively uniform patterns. For instance, Buffered Fixed-Length Obfuscation (BuFLO) [218] obfuscates page transmissions by sending packets of a fixed size at a fixed interval and using dummy packets to both fill in and potentially extend the transmission. Thus, the traffic generated by different websites has a similar continuous traffic flow. However, BuFLO can cause very high time and bandwidth overhead, sometimes can even bring congestion problems to the network [116]. To alleviate the problem, Congestion-Sensitive BuFLO (CS-BuFLO) [287] was proposed to vary the packet transmission rate. Tamaraw [222] achieves a better security/bandwidth trade-off by using smaller fixed packet sizes and treating incoming and outgoing packets differently to avoid unnecessary padding and dummy traffic. DynaFlow [288] morphs packets into fixed bursts, dynamically changes packet inter-arrival times to generate constant traffic flows, and pads the number of bursts. Theoretically, DynaFlow leads to less network overhead compared with BuFLO, CS-BuFLO, and Tamaraw.

The recent development of FGTA countermeasures mainly focuses on two aspects:

- 1) The countermeasure should lead to nearly zero overhead to both the data plane and the endpoints.
- 2) The countermeasure should be applicable to various web applications (e.g., web page visiting, video streaming, VoIP, etc.) and scenarios.

For instance, Henri et al. [289] split traffic exchanged between the user and Tor nodes over two different, unrelated network connections (e.g., DSL, Wi-Fi, or cellular networks) to protect against FGTA by a malicious ISP; TrafficSliver [216] limits the data a single observation point can observe and distorts repeatable traffic patterns exploited by FGTA with user-controlled splitting of traffic over multiple Tor entry nodes. TrafficSliver also offers an application-layer solution, which will be discussed in Section VII-B; Wang [290] points out that an attacker may only need to successfully identify a single web page (which they define as the one-page setting) in reality, and a WFP countermeasure must still thwart that attempt. Based on this assumption, Wang fortifies WFP countermeasures by exploring randomness and regularization options for several existing countermeasures. To protect IoT networks, Pinheiro

et al. [275] implement a middlebox to modify the outbound and inbound traffic's packet size. They also leverage an SDN application to obtain information of network traffic from both sides (source and destination) to manage the size-based padding mechanism.

B. Application-layer Countermeasures

Unlike network-layer countermeasures that directly modify network traffic to cover user activities, application-layer countermeasures use dummy applications to generate unnecessary traffic, thereby indirectly perturbing FGTA approaches. However, most application-layer countermeasures are limited in covering traffic of web pages being visited.

Panchenko et al. [204] proposed a browser plug-in that adds traffic noise by loading another random web page in parallel. However, it may fail to defend against some WFP attacks if users lower the page loading frequency to decrease the bandwidth overhead [205]. Another Tor-based countermeasures approach [291] randomizes the order of requests for embedded website content and the pipeline size (i.e., the number of requests processed in parallel) to perturb WFPs. Cherubin et al. [279] propose LLaMA and ALPaCA, defenses for client side and server side. LLaMA reorders outgoing HTTP requests by randomly delaying them and adding dummy HTTP requests. On the server side, ALPaCA conducts traffic morphing by padding web objects of a page and inserting invisible dummy web objects. The three methods above only work in Tor environments.

HTTP Obfuscation (HTTPOS) [278] is countermeasure that can be used in environments other than Tor. By modifying HTTP requests and basic TCP features, it manipulates four fundamental network flow features, including packet size, web object size, flow size, and timing of packets. It can also modify and reorder HTTP headers and insert dummy HTTP requests. Another general countermeasure is TrafficSliver's application-layer defense [216]. This approach is on the client side. By sending single HTTP requests for different web objects over distinct Tor entry nodes, this application-layer defense can reduce the detection rate of WFP classifiers by almost 50 percentage points.

VIII. FUTURE RESEARCH DIRECTION

Despite decades of development, FGTA continues to offer substantial opportunities for further advancement, enhancement, and exploration. In this section, we outline potential research directions based on our analysis of recent trends, existing literature, industry implementations, and key challenges that remain unaddressed in this domain.

A. Improvement of Analysis Efficacy and Coverage

FGTA has been used in many subfields of computer network, including attack detection, traffic measurement, side-channel attack, network management, etc. Researchers have constructed myriad analysis models and collected plenty of datasets specifically for different categories of tasks. But there are still many use cases or scenarios that have not been

comprehensively covered by existing approaches. For instance, with the rise of Unmanned Aerial Vehicle (UAV) and autonomous vehicles, there have been initiatives to adapt FGTA for unique applications like UAV anomaly detection [292], [293] and the protection of autonomous vehicles [294], [295]. However, this area of research is still in its infancy, characterized by a limited number of studies. Furthermore, as new applications, attack vectors, and communication protocols continue to emerge, the capacity of existing FGTA methodologies to effectively manage contemporary traffic challenges may be limited. Therefore, researchers can gather more updated traffic datasets to enhance the coverage of existing FGTA approaches, so that they can be used in more types of tasks and scenarios.

In addition, the efficacy of many current FGTA approaches are not ideal for real-world deployments. Depending on the observation points, FGTA approaches may easily see millions of traffic flows over a short time period in the real world. Under such circumstances, an FGTA approach could generate large numbers of false positives or false negatives, even if it achieves more than 95% accuracies in closed-world evaluations. Thus, increasing the efficacy of FGTA is a timeless topic for researchers and developers.

B. Evaluation Enhancement

As we elaborate in Section VI, current closed-world evaluation methods are far away from revealing an FGTA approach's real capability and many open-world evaluations are not very standardized and effective [273]. It is therefore suitable to propose a new, operable, and effective evaluation paradigm for FGTA. Such an evaluation paradigm should contain a testing dataset similar to a real-world test case in terms of volume, environment, and data distribution. Simultaneously, the dataset should have comprehensive labels for almost all traffic flows, not only for analysis targets. This can be achieved by either constructing a large scale sandbox to simulate and collect all types of traffic from a white box view, or collect a large-scale, real-world traffic dataset and carefully label it using knowledge of endpoints from all perspectives. Besides, the testing data portion that is visible to the observation point should be consistent with the real-world deployment conditions.

C. Dealing with Complex Network Environments

In real-world deployments, the network environments and configurations can be different from researchers' assumptions. The following factors were not widely discussed in previous papers, but can be common for network service providers.

- Many observation points can only see asymmetric network traffic, which can challenge most FGTA approaches.
- Some networks are composed of multiple subnets, including but not limited to wireless network, optical network, or radio frequency network. Traffic flows collected from such a network can have different delays and congestion control mechanisms. Tackling this type of traffic can be challenging.
- Due to deployments of modern traffic engineering approaches, traffic captured from some observation points is dynamic [52], posing difficulties to many FGTA methods.

We believe designing and implementing new FGTA approaches that can work under these circumstances are directions worthy of future research.

D. Integrating FGTA into Other Analytical Systems

Information contained in network traffic is essentially limited. Even though FGTA can already reveal considerable amount of information, the detailed behavior models of endpoints are still hidden behind the curtain. To more comprehensively investigate the network situation, researchers can try to combine FGTA with information from other dimensions (e.g., application-layer activities, server specifics, hardware conditions, etc.), which can provide a better situational awareness. So far, there are a few researches that combine TA with information from other layers for more accurate attack/anomaly detection and timely threat response (e.g., [296]–[298]). Researchers can push this idea forward by further integrating FGTA into this idea.

Furthermore, cyber threat intelligence (CTI) [299], allowing entities to share attack/anomaly information with trusted partners and peers, is becoming a powerful tool to quickly and accurately tackle intractable attacks. By embedding results from FGTA into CTI systems, participating entities can raise awareness of the current situation, thereby more quickly responding to incoming attacks. Designing attack defense systems with both FGTA and CTI is thus a promising research direction.

E. Cutting Edge Technologies for FGTA

With advancements in cutting-edge technologies, FGTA has the potential for significant enhancements across various dimensions. For instance, ET-BERT [137] utilizes concepts from natural language processing (NLP) to process TA. This approach involves pre-training deeply contextualized datagram-level representations using extensive unlabeled data sets. Subsequently, the pre-trained model can be fine-tuned with a minimal set of task-specific labeled data, catering to specific FGTA tasks. The recent surge in popularity of large language models, exemplified by ChatGPT [300], has opened up new avenues for enhancing FGTA. By fine-tuning these advanced models with specialized TA datasets, we can systematically decode and understand the narratives embedded within network traffic data.

In the future, we anticipate the emergence of more sophisticated technologies geared towards understanding, processing, and integrating knowledge. Researchers are encouraged to apply these advancements to FGTA, with the aim of broadening the scope of this domain. Such integration is expected to not only enhance performance but also significantly improve the explainability and adaptability of FGTA methodologies.

F. Enhancing the Explainability of FGTA

Explainability plays a pivotal role in the practical implementation of FGTA [301]. An FGTA method that offers clear explainability greatly assists network administrators by enabling them to: (1) grasp the underlying analysis procedure

and its logic; (2) efficiently verify the analysis outcomes, which aids in reducing false alarms and bolstering confidence in the results; and (3) gain a deeper understanding of the network's situation, leading to more informed and effective network management decisions.

Although some rule-based and statistical FGTA approaches demonstrate satisfactory performance in terms of explainability [19], [114], the majority of FGTA techniques heavily rely on machine learning, with only a limited number emphasizing explainability [152], [164], [302]. On the other hand, a range of general explainable machine learning techniques have been proposed, as documented in several studies [303]–[307]. In theory, these techniques could be adapted for use in various machine learning-based solutions. However, the specific explainability requirements in FGTA often differ from those in other fields, which can make the direct application of these existing techniques challenging. Additionally, their adoption in FGTA has been limited, possibly due to issues related to algorithmic suitability, performance, scalability, among other factors.

It is therefore essential to develop novel explainable machine learning techniques that are specifically tailored for FGTA. Such techniques should be designed to address the unique challenges and requirements of FGTA, while also offering satisfactory performance in terms of accuracy, efficiency, and scalability.

IX. CONCLUSION

With the increasing complexity of network transmission technology, FGTA is becoming a crucial tool to gain a finer granularity of visibility on or above the application layer from the network traffic. From the perspective of attackers, FGTA approaches can be used to analyze the content-agnostic metadata and statistical information of network traffic to infer the website visited by users, estimate locations of traffic sender, or decode the video content streamed in the link. As for the network administrators, FGTA approaches can be used to detect application-layer threats even with layer 3 or layer 4 data, investigate quality of experience without collecting sensitive user data, or perform fine-grained traffic measurement to better configure the network.

In this paper, we analyze literature that deal with FGTA to help researchers and developers learn the latest developments in this area. After comparing different FGTA approaches by their methodologies and use cases, we found that most existing approaches are based on deep learning or high-dimensional clustering. They are effective in capturing the subtle differences between network traffic generated by different activities. However, many FGTA approaches still come with limitations related to training data coverage, traffic data availability, high false positive rates, real-world usability, etc. In addition, edge users of the network can adopt a variety of countermeasures to defend against FGTA, with some overheads regarding network bandwidth and delay. Researchers can further research and develop this domain to increase the coverage of FGTA, make FGTA more practical in complex real-world network environments, enhance the robustness of FGTA, and reduce the overheads of FGTA approaches.

ACRONYMS

ABR	Adaptive Bitrate
ACC	accuracy
APT	advanced persistent threat
AUC	area under the curve
BTS	base transceiver station
CCPA	California Consumer Privacy Act
CDN	content delivery network
CNN	convolutional neural network
CTI	cyber threat intelligence
DNN	deep neural network
DPI	deep packet inspection
F1	F1 score
FDR	false discovery rate
FGTA	fine-grained traffic analysis
FNR	false negative rate
FOR	false omission rate
FPR	false positive rate
GBDT	gradient boosting decision tree
GDPR	General Data Protection Regulation
GNN	graph neural network
HMM	hidden Markov model
IoT	Internet of things
ISP	internet service provider
KNN	k-nearest neighbor
LBA	location-based applications
LBS	location-based services
LSTM	long short-term memory
MLP	multi-layer perceptron
NAT	network address translation
NDAE	nonsymmetric deep autoencoder
NLP	natural language processing
NPV	negative predictive value
OS	operating system
OSN	online social network
PCA	principal component analysis
PII	personally identifiable information
PPV	positive predictive value
QoE	quality of experience
QoS	quality of service
QUIC	Quick UDP Internet Connection
RKHS	Reproducing Kernel Hilbert Space
RNN	recurrent neural network
ROC	receiver operating characteristic
SAE	stacked autoencoder
SMO	Sequential Minimal Optimization
SMOT	Synthetic Minority Oversampling Technique
SVM	Support Vector Machine
TA	traffic analysis
TNR	true negative rate
TPR	true positive rate
TTL	time to live
UAV	Unmanned Aerial Vehicle
WFP	website fingerprinting

ACKNOWLEDGMENTS

The authors would like to thank Zhangxiang Hu, Lei Jiao, Yingjiu Li, and Thanh Nguyen from the University of Oregon for their constructive suggestions on this work.

This research / project is partially supported by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

This work is also partially supported by the Natural Science Foundation of Sichuan Province (Grant No. 2024NS-FSC1450).

This material is also based upon work partially supported by Ripple under the University Blockchain Research Initiative (UBRI) [308]. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Ripple.

REFERENCES

- [1] C. A. Sunshine, *Computer network architectures and protocols*. Springer Science & Business Media, 2013.
- [2] Wikipedia, "Internet protocol suite," https://en.wikipedia.org/wiki/Internet_protocol_suite, date of visit: 2021-10-05.
- [3] J.-F. Raymond, "Traffic analysis: Protocols, attacks, design issues, and open problems," in *Designing Privacy Enhancing Technologies*. Springer, 2001, pp. 10–29.
- [4] C. So-In, "A survey of network traffic monitoring and analysis tools," *Cse 576m computer system analysis project*, Washington University in St. Louis, 2009.
- [5] A. D'Alconzo, I. Drago, A. Morichetta, M. Mellia, and P. Casas, "A survey on big data for network traffic monitoring and analysis," *IEEE Transactions on Network and Service Management*, vol. 16, no. 3, pp. 800–813, 2019.
- [6] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021.
- [7] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2018.
- [8] V. Paxson, "Bro: a system for detecting network intruders in real-time," *Computer networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.
- [9] H. Kim, K. C. Claffy, M. Fomenkov, D. Barman, M. Faloutsos, and K. Lee, "Internet traffic classification demystified: myths, caveats, and the best practices," in *Proceedings of the 2008 ACM CoNEXT conference*, 2008, pp. 1–12.
- [10] J. C. Guevara, R. da S. Torres, and N. L. da Fonseca, "On the classification of fog computing applications: A machine learning perspective," *Journal of Network and Computer Applications*, vol. 159, p. 102596, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804520300709>
- [11] A. Sang and S.-q. Li, "A predictability analysis of network traffic," *Computer networks*, vol. 39, no. 4, pp. 329–345, 2002.
- [12] A. Lakhina, K. Papagiannaki, M. Crovella, C. Diot, E. D. Kolaczyk, and N. Taft, "Structural analysis of network traffic flows," in *Proceedings of the joint international conference on Measurement and modeling of computer systems*, 2004, pp. 61–72.
- [13] J. Mirkovic, Y. Feng, and J. Li, "Measuring changes in regional network traffic due to covid-19 stay-at-home measures," *arXiv preprint arXiv:2203.00742*, 2022.
- [14] P. Barford, J. Kline, D. Plonka, and A. Ron, "A signal analysis of network traffic anomalies," in *Proceedings of the 2nd ACM SIGCOMM Workshop on Internet measurement*, 2002, pp. 71–82.
- [15] J. Eрман, M. Arlitt, and A. Mahanti, "Traffic classification using clustering algorithms," in *Proceedings of the 2006 SIGCOMM workshop on Mining network data*, 2006, pp. 281–286.
- [16] S. Li, L. Luo, D. Guo, Q. Zhang, and P. Fu, "A survey of sketches in traffic measurement: Design, optimization, application and implementation," *arXiv preprint arXiv:2012.07214*, 2020.
- [17] Z. Zeng, L. Cui, M. Qian, Z. Zhang, and K. Wei, "A survey on sliding window sketch for network measurement," *Computer Networks*, vol. 226, p. 109696, 2023.
- [18] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2022.
- [19] E. Papadogiannaki, C. Halevidis, P. Akritidis, and L. Koromilas, "Otter: A scalable high-resolution encrypted traffic identification engine," in *International Symposium on Research in Attacks, Intrusions, and Defenses*. Springer, 2018, pp. 315–334.
- [20] T. van Ede, R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Hoffnes, M. van Steen, and A. Peter, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Network and Distributed System Security Symposium (NDSS)*, vol. 27, 2020.
- [21] Y. Feng, J. Li, L. Jiao, and X. Wu, "BotFlowMon: Learning-based, content-agnostic identification of social bot traffic flows," in *2019 IEEE Conference on Communications and Network Security (CNS)*, June 2019, pp. 169–177.
- [22] R. Schuster, V. Shmatikov, and E. Tromer, "Beauty and the burst: Remote identification of encrypted video streams," in *26th {USENIX} Security Symposium ({USENIX} Security 17)*, 2017, pp. 1357–1374.
- [23] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: accounting for noisy labels and non-stationarity," in *Proceedings of the 23rd ACM SIGKDD International Conference on knowledge discovery and data mining*, 2017, pp. 1723–1732.
- [24] S. Xu, S. Sen, and Z. M. Mao, "Csi: Inferring mobile abr video adaptation behavior under https and quic," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–16.
- [25] Q. Yin, Z. Liu, Q. Li, T. Wang, Q. Wang, C. Shen, and Y. Xu, "Automated multi-tab website fingerprinting attack," *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [26] J. Liu, Y. Fu, J. Ming, Y. Ren, L. Sun, and H. Xiong, "Effective and real-time in-app activity analysis in encrypted internet traffic streams," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 335–344.
- [27] G. Ateniese, B. Hitaj, L. V. Mancini, N. V. Verde, and A. Villani, "No place to hide that bytes won't reveal: Sniffing location-based encrypted traffic to track a user's position," in *International Conference on Network and System Security*. Springer, 2015, pp. 46–59.
- [28] G. O. Ferreira, C. Ravazzi, F. Dabbene, G. C. Calafiore, and M. Fiore, "Forecasting network traffic: A survey and tutorial with open-source comparative evaluation," *IEEE Access*, vol. 11, pp. 6018–6044, 2023.
- [29] H. Jmila, G. Blanc, M. R. Shahid, and M. Lazrag, "A survey of smart home iot device classification using machine learning-based network traffic analysis," *IEEE Access*, vol. 10, pp. 97 117–97 141, 2022.
- [30] D. A. Tedjopurmomo, Z. Bao, B. Zheng, F. M. Choudhury, and A. K. Qin, "A survey on modern deep neural network for traffic prediction: Trends, methods and challenges," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 4, pp. 1544–1561, 2022.
- [31] T. Wu, F. Breitinger, and S. Niemann, "Iot network traffic analysis: Opportunities and challenges for forensic investigators?" *Forensic Science International: Digital Investigation*, vol. 38, p. 301123, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666281721000214>
- [32] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in iot networks: A survey," *Journal of Network and Computer Applications*, vol. 154, p. 102538, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804520300126>
- [33] M. Conti, Q. Q. Li, A. Maragno, and R. Spolaor, "The dark side(-channel) of mobile devices: A survey on network traffic analysis," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 2658–2713, 2018.
- [34] D. Naboulsi, M. Fiore, S. Ribot, and R. Stanica, "Large-scale mobile traffic analysis: A survey," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 124–161, 2016.
- [35] P. Velan, M. Čermák, P. Čeleda, and M. Drašar, "A survey of methods for encrypted traffic classification and analysis," *International Journal of Network Management*, vol. 25, no. 5, pp. 355–374, 2015.
- [36] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Muller, and K. Hanssgen, "A survey of payload-based traffic classification approaches," *IEEE*

- Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 1135–1156, 2014.
- [37] J. a. V. Gomes, P. R. M. Inácio, M. Pereira, M. M. Freire, and P. P. Monteiro, "Detection and classification of peer-to-peer traffic: A survey," *ACM Comput. Surv.*, vol. 45, no. 3, jul 2013. [Online]. Available: <https://doi.org/10.1145/2480741.2480747>
 - [38] A. Callado, C. Kamiński, G. Szabo, B. P. Gero, J. Kelner, S. Fernandes, and D. Sadok, "A survey on internet traffic identification," *IEEE Communications Surveys & Tutorials*, vol. 11, no. 3, pp. 37–52, 2009.
 - [39] B. Chandrasekaran, "Survey of network traffic models," *Washington University in St. Louis CSE*, vol. 567, 2009.
 - [40] A. Erramilli, M. Roughan, D. Veitch, and W. Willinger, "Self-similar traffic and network dynamics," *Proceedings of the IEEE*, vol. 90, no. 5, pp. 800–819, 2002.
 - [41] A. Cecil, "A summary of network traffic monitoring and analysis techniques," *Computer systems analysis*, pp. 4–7, 2006.
 - [42] V. Mohan, Y. J. Reddy, K. Kalpana *et al.*, "Active and passive network measurements: a survey," *International Journal of Computer Science and Information Technologies*, vol. 2, no. 4, pp. 1372–1385, 2011.
 - [43] M. Abbasi, A. Shahraiki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (ntma): A survey," *Computer Communications*, vol. 170, pp. 19–41, 2021.
 - [44] A. Bates, B. Mood, J. Pletcher, H. Pruse, M. Valafar, and K. Butler, "Detecting co-residency with active traffic analysis techniques," in *Proceedings of the 2012 ACM Workshop on Cloud computing security workshop*, 2012, pp. 1–12.
 - [45] A. Iacovazzi, S. Sarda, and Y. Elovici, "Inflow: Inverse network flow watermarking for detecting hidden servers," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 747–755.
 - [46] M. Gregorczyk, P. Żórawski, P. Nowakowski, K. Cabaj, and W. Mazurczyk, "Sniffing detection based on network traffic probing and machine learning," *IEEE Access*, vol. 8, pp. 149 255–149 269, 2020.
 - [47] X. Wang, S. Chen, and S. Jajodia, "Network flow watermarking attack on low-latency anonymous communication systems," in *2007 IEEE Symposium on Security and Privacy (SP'07)*. IEEE, 2007, pp. 116–130.
 - [48] A. Iacovazzi and Y. Elovici, "Network flow watermarking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 1, pp. 512–530, 2016.
 - [49] L. Zhang, Y. Kong, Y. Guo, J. Yan, and Z. Wang, "Survey on network flow watermarking: model, interferences, applications, technologies and security," *IET Communications*, vol. 12, no. 14, pp. 1639–1648, 2018.
 - [50] X. Fu, B. Graham, R. Bettati, and W. Zhao, "Active traffic analysis attacks and countermeasures," in *2003 International Conference on Computer Networks and Mobile Computing, 2003. ICCNMC 2003*. IEEE, 2003, pp. 31–39.
 - [51] Y. He, M. Faloutsos, S. Krishnamurthy, and B. Huffaker, "On routing asymmetry in the internet," in *GLOBECOM'05. IEEE Global Telecommunications Conference, 2005.*, vol. 2. IEEE, 2005, pp. 6–pp.
 - [52] N. Wang, K. H. Ho, G. Pavlou, and M. Howarth, "An overview of routing optimization for internet traffic engineering," *IEEE Communications Surveys & Tutorials*, vol. 10, no. 1, pp. 36–56, 2008.
 - [53] I. F. Akylidiz, A. Lee, P. Wang, M. Luo, and W. Chou, "A roadmap for traffic engineering in sdn-openflow networks," *Computer Networks*, vol. 71, pp. 1–30, 2014.
 - [54] B. Trammell and E. Boschi, "An introduction to ip flow information export (ipfix)," *IEEE Communications Magazine*, vol. 49, no. 4, pp. 89–95, 2011.
 - [55] Y. Yu, C. Qian, and X. Li, "Distributed and collaborative traffic monitoring in software defined networks," in *Proceedings of the third workshop on Hot topics in software defined networking*, 2014, pp. 85–90.
 - [56] D. Fernández, X. M. da Silva, F. J. Nóvoa, F. Cacheda, and V. Carneiro, "Using collaborative filtering in a new domain: traffic analysis," in *Proceedings of the 4th Spanish Conference on Information Retrieval*, 2016, pp. 1–8.
 - [57] L. Lu, Z. Huan, X. Zhang, L. Qi, S. Chen, and Y. Wu, "Collaborative network traffic analysis via alternating direction method of multipliers," in *2018 IEEE 22nd International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2018, pp. 547–552.
 - [58] M. Burkhart, *Enabling collaborative network security with privacy-preserving data aggregation*. ETH Zurich, 2011, vol. 125.
 - [59] Y. Zhao, J. Chen, D. Wu, J. Teng, and S. Yu, "Multi-task network anomaly detection using federated learning," in *Proceedings of the 10th international symposium on information and communication technology*, 2019, pp. 273–279.
 - [60] Y. Liu, J. James, J. Kang, D. Niyato, and S. Zhang, "Privacy-preserving traffic flow prediction: A federated learning approach," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7751–7763, 2020.
 - [61] J. Pei, K. Zhong, M. A. Jan, and J. Li, "Retracted: Personalized federated learning framework for network traffic anomaly detection," 2022.
 - [62] V. Perifanis, N. Pavlidis, R.-A. Koutsiamanis, and P. S. Efraimidis, "Federated learning for 5g base station traffic forecasting," *Computer Networks*, vol. 235, p. 109950, 2023.
 - [63] "Solaris snoop packet sniffer," <http://www.softpanorama.org/Net/Sniffers/snoop.shtml>, accessed: 2022-02-10.
 - [64] A. Orebaugh, G. Ramirez, and J. Beale, *Wireshark & Ethereal network protocol analyzer toolkit*. Elsevier, 2006.
 - [65] "Data sheet of sniffer infinistream," <http://mavin.com/pictures/InfiniStream.pdf>, accessed: 2022-02-10.
 - [66] M. Bagnulo, P. Matthews, and I. van Beijnum, "Stateful nat64: Network address and protocol translation from ipv6 clients to ipv4 servers," Tech. Rep., 2011.
 - [67] N. Brownlee, C. Mills, and G. Ruth, "Traffic flow measurement: Architecture," RFC 2722, Tech. Rep., 1999.
 - [68] J. Rajahalme, A. Conta, B. Carpenter, and S. Deering, "Ipv6 flow label specification," RFC3697, 2004.
 - [69] J. Quittek, T. Zseby, B. Claise, and S. Zander, "Requirements for ip flow information export (ipfix)," RFC 3917 (informational), Tech. Rep., 2004.
 - [70] D. W. McRobb, "Cflowd design," CAIDA, Sept, 1998.
 - [71] R. Hofstede, P. Čeleda, B. Trammell, I. Drago, R. Sadre, A. Sperotto, and A. Pras, "Flow monitoring explained: From packet capture to data analysis with netflow and ipfix," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 2037–2064, 2014.
 - [72] J. Lee, "Calculating netflow volume," <https://www.plixer.com/blog/calculating-netflow-volume/>, posted: 2013-03-26.
 - [73] B.-Y. Choi and S. Bhattacharyya, "On the accuracy and overhead of cisco sampled netflow," in *Proceedings of ACM SIGMETRICS Workshop on Large Scale Network Inference (LSNI)*, 2005, pp. 1–6.
 - [74] C. Hare, "Simple network management protocol (snmp)." 2011.
 - [75] B. Claise, B. Trammell, and P. Aitken, "Specification of the ip flow information export (ipfix) protocol for the exchange of flow information," pp. 2070–1721, 2013.
 - [76] B. Claise, G. Sadasivan, V. Valluri, and M. Djernaes, "Cisco systems netflow services export version 9," 2004.
 - [77] "Netflow v5 formats," <https://www.ibm.com/docs/en/npi/1.3.0?topic=versions-netflow-v5-formats>, accessed: 2022-03-14.
 - [78] "Argus project," <https://openargus.org/>, date of visit: 2021-11-25.
 - [79] P. Phaal, S. Panchen, and N. McKee, "Rfc3176: Inmon corporation's sflow: A method for monitoring traffic in switched and routed networks," 2001.
 - [80] "Tcpdump & libpcap," <https://www.tcpdump.org/>, accessed: 2022-02-10.
 - [81] ntop, "Pf_ring documentation," https://www.ntop.org/guides/pf_ring/, accessed: 2022-02-12.
 - [82] L. Rizzo, "netmap: a novel framework for fast packet i/o," in *21st USENIX Security Symposium (USENIX Security 12)*, 2012, pp. 101–112.
 - [83] J. Rasley, B. Stephens, C. Dixon, E. Rozner, W. Felter, K. Agarwal, J. Carter, and R. Fonseca, "Planck: Millisecond-scale monitoring and control for commodity networks," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 407–418, 2014.
 - [84] J. Svoboda, I. Ghafir, V. Prenosil *et al.*, "Network monitoring approaches: An overview," *Int J Adv Comput Netw Secur*, vol. 5, no. 2, pp. 88–93, 2015.
 - [85] L.-M. Wang, T. Miskell, J. Morgan, and E. Verplanke, "Design of a real-time traffic mirroring system," in *2021 IFIP/IEEE International Symposium on Integrated Network Management (IM)*. IEEE, 2021, pp. 793–796.
 - [86] "ProfiShark Network TAPs," <https://www.profitap.com/profishark-network-taps/>, 2019, accessed: 2022-02-11.
 - [87] Wikipedia, "Argus – audit record generation and utilization system," https://en.wikipedia.org/wiki/Argus_%E2%80%93_Audit_Record_Generation_and_Utilization_System, accessed: 2022-02-11.
 - [88] Í. Cunha, F. Silveira, R. Oliveira, R. Teixeira, and C. Diot, "Uncovering artifacts of flow measurement tools," in *International Conference on Passive and Active Network Measurement*. Springer, 2009, pp. 187–196.
 - [89] Huawei, "Configuration guide - network management and monitoring," <https://support.huawei.com/enterprise/en/doc/EDOC1000178174/986bf11e/overview-of-netstream>, accessed: 2022-02-11.

- [90] S. Waldbusser, R. Cole, C. Kalbfleisch, and D. Romascanu, "Introduction to the remote monitoring (rmon) family of mib modules," *RFC3577, Network Working Group*, 2003.
- [91] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: Fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th international conference on emerging networking experiments and technologies*, 2018, pp. 54–66.
- [92] M. S. Brunella, G. Belocchi, M. Bonola, S. Pontarelli, G. Siracusano, G. Bianchi, A. Cammarano, A. Palumbo, L. Petrucci, and R. Bifulco, "{hXDP}: Efficient software packet processing on {FPGA}{NICs}," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 973–990.
- [93] Y. Zhou, C. Sun, H. H. Liu, R. Miao, S. Bai, B. Li, Z. Zheng, L. Zhu, Z. Shen, Y. Xi *et al.*, "Flow event telemetry on programmable data plane," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 76–89.
- [94] D. Comer, "Ubiquitous b-tree," *ACM Computing Surveys (CSUR)*, vol. 11, no. 2, pp. 121–137, 1979.
- [95] P. Voigt and A. Von dem Bussche, "The eu general data protection regulation (gdpr)," *A Practical Guide, 1st Ed., Cham: Springer International Publishing*, vol. 10, no. 3152676, pp. 10–5555, 2017.
- [96] Q. Huang, X. Jin, P. P. Lee, R. Li, L. Tang, Y.-C. Chen, and G. Zhang, "Sketchvisor: Robust network measurement for software packet processing," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 113–126.
- [97] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, and S. Uhlig, "Elastic sketch: Adaptive and fast network-wide measurements," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 561–575.
- [98] Y. Zhou, Y. Zhang, C. Ma, S. Chen, and O. O. Odegbile, "Generalized sketch families for network traffic measurement," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 3, no. 3, pp. 1–34, 2019.
- [99] K. Yang, S. Long, Q. Shi, Y. Li, Z. Liu, Y. Wu, T. Yang, and Z. Jia, "Sketchint: Empowering int with towersketch for per-flow per-switch measurement," *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [100] R. Miao, Y. Zhang, Z. Zheng, R. Wang, R. Zhang, T. Yang, Z. Liu, and J. Jiang, "Cocosketch: High-performance sketch-based measurement over arbitrary partial key query," *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 2653–2668, 2023.
- [101] H. Namkung, Z. Liu, D. Kim, V. Sekar, and P. Steenkiste, "Sketchovsky: Enabling ensembles of sketches on programmable switches," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1273–1292.
- [102] Y. Feng, J. Luo, C. Ma, T. Li, and L. Hui, "I can still observe you: Flow-level behavior fingerprinting for online social network," in *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 2022, pp. 6427–6432.
- [103] Y. Feng, J. Li, L. Jiao, and X. Wu, "Towards learning-based, content-agnostic detection of social bot traffic," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2149–2163, 2021.
- [104] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, and Y. Liu, "Encrypted traffic classification of decentralized applications on ethereum using feature fusion," in *2019 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS)*. IEEE, 2019, pp. 1–10.
- [105] I. Guyon, S. Gunn, M. Nikravesh, and L. A. Zadeh, *Feature extraction: foundations and applications*. Springer, 2008, vol. 207.
- [106] S. Khalid, T. Khalil, and S. Nasreen, "A survey of feature selection and feature extraction techniques in machine learning," in *2014 science and information conference*. IEEE, 2014, pp. 372–378.
- [107] R. Zebari, A. Abdulazeez, D. Zeebaree, D. Zebari, and J. Saeed, "A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction," *Journal of Applied Science and Technology Trends*, vol. 1, no. 2, pp. 56–70, 2020.
- [108] P. Dhal and C. Azad, "A comprehensive survey on feature selection in the various fields of machine learning," *Applied Intelligence*, pp. 1–39, 2022.
- [109] H. F. Alan and J. Kaur, "Can android applications be identified using only tcp/ip headers of their launch time traffic?" in *Proceedings of the 9th ACM conference on security & privacy in wireless and mobile networks*, 2016, pp. 61–66.
- [110] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep learning approach for network intrusion detection in software defined networking," in *2016 international conference on wireless networks and mobile communications (WINCOM)*. IEEE, 2016, pp. 258–263.
- [111] S. Rezaei and X. Liu, "How to achieve high classification accuracy with just a few labels: A semi-supervised approach using sampled packets," *arXiv preprint arXiv:1812.09761*, 2018.
- [112] F. Gringoli, L. Salgarelli, M. Dusi, N. Cascarano, F. Risso, and K. Claffy, "Gt: picking up the truth from the ground for internet traffic," *ACM SIGCOMM Computer Communication Review*, vol. 39, no. 5, pp. 12–18, 2009.
- [113] Y. Feng, J. Li, and D. Sisodia, "Cj-sniffer: Measurement and content-agnostic detection of cryptojacking traffic," in *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses*, 2022, pp. 482–494.
- [114] M. Jiang, G. Gou, J. Shi, and G. Xiong, "I know what you are doing with remote desktop," in *2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2019, pp. 1–7.
- [115] G. D. Bissias, M. Liberatore, D. Jensen, and B. N. Levine, "Privacy vulnerabilities in encrypted http streams," in *International Workshop on Privacy Enhancing Technologies*. Springer, 2005, pp. 1–11.
- [116] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 605–616.
- [117] Y. Fu, H. Xiong, X. Lu, J. Yang, and C. Chen, "Service usage classification with encrypted internet traffic in mobile messaging apps," *IEEE Transactions on Mobile Computing*, vol. 15, no. 11, pp. 2851–2864, 2016.
- [118] Y. Wang, Z. Li, G. Gou, G. Xiong, C. Wang, and Z. Li, "Identifying dapps and user behaviors on ethereum via encrypted traffic," in *International Conference on Security and Privacy in Communication Systems*. Springer, 2020, pp. 62–83.
- [119] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapè, "Mimetic: Mobile encrypted traffic classification using multimodal deep learning," *Computer networks*, vol. 165, p. 106944, 2019.
- [120] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [121] P. V. Amoli, T. Hamalainen, G. David, M. Zolotukhin, and M. Mirzamohammad, "Unsupervised network intrusion detection systems for zero-day fast-spreading attacks and botnets," *JDCTA (International Journal of Digital Content Technology and its Applications)*, vol. 10, no. 2, pp. 1–13, 2016.
- [122] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE transactions on emerging topics in computational intelligence*, vol. 2, no. 1, pp. 41–50, 2018.
- [123] K. Park and W. Willinger, "Self-similar network traffic: An overview," *Self-Similar Network Traffic and Performance Evaluation*, pp. 1–38, 2000.
- [124] W. Duch, R. Setiono, and J. M. Zurada, "Computational intelligence methods for rule-based data understanding," *Proceedings of the IEEE*, vol. 92, no. 5, pp. 771–805, 2004.
- [125] N. Alon and J. H. Spencer, *The probabilistic method*. John Wiley & Sons, 2016.
- [126] I. Rish *et al.*, "An empirical study of the naive bayes classifier," in *IJCAI 2001 workshop on empirical methods in artificial intelligence*, vol. 3, no. 22, 2001, pp. 41–46.
- [127] M. H. Davis, *Markov models & optimization*. CRC Press, 1993, vol. 49.
- [128] L. Rabiner and B. Juang, "An introduction to hidden markov models," *ieee assp magazine*, vol. 3, no. 1, pp. 4–16, 1986.
- [129] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [130] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [131] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [132] Q. Zhou, L. Wang, H. Zhu, and T. Lu, "Few-shot website fingerprinting attack with cluster adaptation," *Computer Networks*, vol. 229, p. 109780, 2023.
- [133] H. Zou, J. Su, Z. Wei, S. Chen, and B. Zhao, "An efficient cross-domain few-shot website fingerprinting attack with brownian distance covariance," *Computer Networks*, vol. 219, p. 109461, 2022.

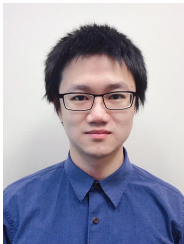
- [134] M. Chen, Y. Wang, and X. Zhu, "Few-shot website fingerprinting attack with meta-bias learning," *Pattern Recognition*, vol. 130, p. 108739, 2022.
- [135] P. Liu, L. He, and Z. Li, "A survey on deep learning for website fingerprinting attacks and defenses," *IEEE Access*, vol. 11, pp. 26 033–26 047, 2023.
- [136] M. Li, D. Han, D. Li, H. Liu, and C.-C. Chang, "Mfv: an anomaly traffic detection method merging feature fusion network and vision transformer architecture," *EURASIP Journal on Wireless Communications and Networking*, vol. 2022, no. 1, p. 39, 2022.
- [137] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, ser. WWW '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 633–642. [Online]. Available: <https://doi.org/10.1145/3485447.3512217>
- [138] D. A. Bierbrauer, M. J. De Lucia, K. Reddy, P. Maxwell, and N. D. Bastian, "Transfer learning for raw network traffic detection," *Expert Systems with Applications*, vol. 211, p. 118641, 2023.
- [139] C. Zhang, H. Zhang, J. Qiao, D. Yuan, and M. Zhang, "Deep transfer learning for intelligent cellular traffic prediction based on cross-domain big data," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1389–1401, 2019.
- [140] A. Shahraki, M. Abbasi, A. Taherkordi, and A. D. Jurcut, "A comparative study on online machine learning techniques for network traffic streams analysis," *Computer Networks*, vol. 207, p. 108836, 2022.
- [141] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "Tensorflow: a system for large-scale machine learning," in *Osdi*, vol. 16, no. 2016. Savannah, GA, USA, 2016, pp. 265–283.
- [142] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshine, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [143] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proceedings of the 22nd ACM international conference on Multimedia*, 2014, pp. 675–678.
- [144] M. Alloghani, D. Al-Jumeily, J. Mustafina, A. Hussain, and A. J. Aljaaf, "A systematic review on supervised and unsupervised machine learning algorithms for data science," *Supervised and unsupervised learning for data science*, pp. 3–21, 2020.
- [145] M. Usama, J. Qadir, A. Raza, H. Arif, K.-L. A. Yau, Y. Elkhatib, A. Hussain, and A. Al-Fuqaha, "Unsupervised machine learning for networking: Techniques, applications and research challenges," *IEEE access*, vol. 7, pp. 65 579–65 615, 2019.
- [146] L. Van Der Maaten, E. Postma, J. Van den Herik *et al.*, "Dimensionality reduction: a comparative," *J Mach Learn Res*, vol. 10, no. 66-71, p. 13, 2009.
- [147] J. A. Hartigan and M. A. Wong, "Algorithm as 136: A k-means clustering algorithm," *Journal of the royal statistical society. series c (applied statistics)*, vol. 28, no. 1, pp. 100–108, 1979.
- [148] G. H. Duntelman, *Principal components analysis*. Sage, 1989, no. 69.
- [149] F. Murtagh and P. Contreras, "Algorithms for hierarchical clustering: an overview," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, pp. 86–97, 2012.
- [150] J. Lever, "Classification evaluation: It is important to understand both what a classification metric expresses and what it hides," *Nature methods*, vol. 13, no. 8, pp. 603–605, 2016.
- [151] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. Ramos, and A. Madeira, "Flowlens: Enabling efficient flow classification for ml-based network security applications," in *Proceedings of the 28th Network and Distributed System Security Symposium (San Diego, CA, USA, 2021)*.
- [152] Y. Feng, J. Li, D. Sisodia, and P. Reiher, "On explainable and adaptable detection of distributed denial-of-service traffic," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 2211–2226, 2024.
- [153] P. Bukaty, *The california consumer privacy act (ccpa): An implementation guide*. IT Governance Ltd, 2019.
- [154] A. K. Das, P. H. Pathak, C.-N. Chuah, and P. Mohapatra, "Privacy-aware contextual localization using network traffic analysis," *Computer Networks*, vol. 118, pp. 24–36, 2017.
- [155] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed Systems Security (NDSS) Symposium*, 2018.
- [156] N. Rosner, I. B. Kadron, L. Bang, and T. Bultan, "Profit: Detecting and quantifying side channels in networked applications," in *NDSS*, 2019.
- [157] M. J. Khokhar, T. Ehlinger, and C. Barakat, "From network traffic measurements to qoe for internet video," in *2019 IFIP Networking Conference (IFIP Networking)*. IEEE, 2019, pp. 1–9.
- [158] Kentik, "Kentik—the network observability platform," <https://www.kentik.com/>, accessed: 2023-12-05.
- [159] P. A. Networks, "Hunt down and stop tomorrow's threats, today—analyze network traffic with best-in-class machine learning and analytics," <https://www.paloaltonetworks.com/cortex/network-traffic-analysis>, accessed: 2023-12-05.
- [160] Z. M. Fadlullah, T. Taleb, N. Ansari, K. Hashimoto, Y. Miyake, Y. Nemoto, and N. Kato, "Combating against attacks on encrypted protocols," in *2007 IEEE International Conference on Communications*. IEEE, 2007, pp. 1211–1216.
- [161] T. Taleb, Z. M. Fadlullah, K. Hashimoto, Y. Nemoto, and N. Kato, "Tracing back attacks against encrypted protocols," in *Proceedings of the 2007 international conference on Wireless communications and mobile computing*, 2007, pp. 121–126.
- [162] Y. Feng and J. Li, "Toward explainable and adaptable detection and classification of distributed denial-of-service attacks," in *International Workshop on Deployable Machine Learning for Security Defense*. Springer, 2020, pp. 105–121.
- [163] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.
- [164] D. Han, Z. Wang, W. Chen, K. Wang, R. Yu, S. Wang, H. Zhang, Z. Wang, M. Jin, J. Yang *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *30th Annual Network and Distributed System Security Symposium (NDSS)*, 2023.
- [165] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "Ids-int: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic," *Digital Communications and Networks*, vol. 10, no. 1, pp. 190–204, 2024.
- [166] A. Shabtai, L. Tenenboim-Chekina, D. Mimran, L. Rokach, B. Shapira, and Y. Elovici, "Mobile malware detection through analysis of deviations in application network behavior," *Computers & Security*, vol. 43, pp. 1–18, 2014.
- [167] S. Wang, Z. Chen, L. Zhang, Q. Yan, B. Yang, L. Peng, and Z. Jia, "Trafficav: An effective and explainable detection of mobile malware behavior using network traffic," in *2016 IEEE/ACM 24th International Symposium on Quality of Service (IWQoS)*. IEEE, 2016, pp. 1–6.
- [168] A. H. Lashkari, A. F. A. Kadir, H. Gonzalez, K. F. Mbah, and A. A. Ghorbani, "Towards a network-based framework for android malware detection and characterization," in *2017 15th Annual conference on privacy, security and trust (PST)*. IEEE, 2017, pp. 233–23309.
- [169] M. Piskozub, F. De Gaspari, F. Barr-Smith, L. Mancini, and I. Martinovic, "Malphase: Fine-grained malware detection using network flow data," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 774–786. [Online]. Available: <https://doi.org/10.1145/3433210.3453101>
- [170] J. Ren, A. Rao, M. Lindorfer, A. Legout, and D. Choffnes, "Recon: Revealing and controlling pii leaks in mobile network traffic," in *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*, 2016, pp. 361–374.
- [171] A. Continella, Y. Fratantonio, M. Lindorfer, A. Puccetti, A. Zand, C. Kruegel, and G. Vigna, "Obfuscation-resilient privacy leak detection for mobile apps through differential analysis," in *NDSS*, 2017.
- [172] D. Willems, K. Kohls, B. van der Kamp, and H. Vranken, "Data exfiltration detection on network metadata with autoencoders," *Electronics*, vol. 12, no. 12, 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/12/2584>
- [173] R. Coulter, Q.-L. Han, L. Pan, J. Zhang, and Y. Xiang, "Data-driven cyber security in perspective—intelligent traffic analysis," *IEEE transactions on cybernetics*, vol. 50, no. 7, pp. 3081–3093, 2019.
- [174] E. Papadogiannaki and S. Ioannidis, "Acceleration of intrusion detection in encrypted network traffic using heterogeneous hardware," *Sensors*, vol. 21, no. 4, p. 1140, 2021.
- [175] R. Moustafa and J. Slay, "A comprehensive data set for network intrusion detection systems," *School of Engineering and Information Technology University of New South Wales at the Australian Defense Force Academy Canberra, Australia, UNSW-NB15*, 2015.
- [176] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos, "Profildroid: Multi-layer profiling of android applications," in *Proceedings of the 18th*

- annual international conference on Mobile computing and networking, 2012, pp. 137–148.
- [177] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, “Taintdroid: an information-flow tracking system for realtime privacy monitoring on smartphones,” *ACM Transactions on Computer Systems (TOCS)*, vol. 32, no. 2, pp. 1–29, 2014.
- [178] A. Razaghpanah, N. Vallina-Rodriguez, S. Sundaresan, C. Kreibich, P. Gill, M. Allman, and V. Paxson, “Haystack: In situ mobile traffic analysis in user space,” *arXiv preprint arXiv:1510.01419*, pp. 1–13, 2015.
- [179] Y. Song and U. Hengartner, “Privacyguard: A vpn-based platform to detect information leakage on android devices,” in *Proceedings of the 5th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices*, 2015, pp. 15–26.
- [180] A. Le, J. Varmarken, S. Langhoff, A. Shuba, M. Gjoka, and A. Markopoulou, “Antmonitor: A system for monitoring from mobile devices,” in *Proceedings of the 2015 ACM SIGCOMM Workshop on Crowdsourcing and Crowdsourcing of Big (Internet) Data*, 2015, pp. 15–20.
- [181] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, “The weka data mining software: an update,” *ACM SIGKDD explorations newsletter*, vol. 11, no. 1, pp. 10–18, 2009.
- [182] Y. Feng, D. Sisodia, and J. Li, “Poster: Content-agnostic identification of cryptojacking in network traffic,” in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020, pp. 907–909.
- [183] S. Khirman and P. Henriksen, “Relationship between quality-of-service and quality-of-experience for public internet service,” in *In Proc. of the 3rd Workshop on Passive and Active Measurement*, vol. 1, 2002.
- [184] X. Xiao and L. M. Ni, “Internet qos: A big picture,” *IEEE network*, vol. 13, no. 2, pp. 8–18, 1999.
- [185] M. Karakus and A. Duresi, “Quality of service (qos) in software defined networking (sdn): A survey,” *Journal of Network and Computer Applications*, vol. 80, pp. 200–218, 2017.
- [186] S. K. Keshari, V. Kansal, and S. Kumar, “A systematic review of quality of services (qos) in software defined networking (sdn),” *Wireless Personal Communications*, vol. 116, no. 3, pp. 2593–2614, 2021.
- [187] J. Bi, X. Zhang, H. Yuan, J. Zhang, and M. Zhou, “A hybrid prediction method for realistic network traffic with temporal convolutional network and lstm,” *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 3, pp. 1869–1879, 2021.
- [188] A. Montieri, G. Bovenzi, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapè, “Packet-level prediction of mobile-app traffic using multitask deep learning,” *Computer Networks*, vol. 200, p. 108529, 2021.
- [189] D. Andreoletti, S. Troia, F. Musumeci, S. Giordano, G. Maier, and M. Tornatore, “Network traffic prediction based on diffusion convolutional recurrent neural networks,” in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2019, pp. 246–251.
- [190] I. Lohrasbinasab, A. Shahraki, A. Taherkordi, and A. Delia Jurcut, “From statistical-to machine learning-based network traffic prediction,” *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 4, p. e4394, 2022.
- [191] L. Tang, J. Li, H. Du, L. Li, J. Wu, and S. Wang, “Big data in forecasting research: a literature review,” *Big Data Research*, vol. 27, p. 100289, 2022.
- [192] T. Hoßfeld and A. Binzenhöfer, “Analysis of skype voip traffic in umts: End-to-end qos and qoe measurements,” *Computer Networks*, vol. 52, no. 3, pp. 650–666, 2008.
- [193] F. Agboma, M. Smy, and A. Liotta, “Qoe analysis of a peer-to-peer television system,” in *Proceedings of IADISInt. Conf. on Telecommunications, Networks and Systems*, 2008, pp. 365–382.
- [194] G. Dimopoulos, I. Leontiadis, P. Barlet-Ros, and K. Papagiannaki, “Measuring video qoe from encrypted traffic,” in *Proceedings of the 2016 Internet Measurement Conference*, 2016, pp. 513–526.
- [195] I. Orsolic, D. Pevec, M. Suznjevic, and L. Skorin-Kapov, “Youtube qoe estimation based on the analysis of encrypted network traffic using machine learning,” in *2016 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 2016, pp. 1–6.
- [196] M. H. Mazhar and Z. Shafiq, “Real-time video quality of experience monitoring for https and quic,” in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 1331–1339.
- [197] D. McCoy, K. Bauer, D. Grunwald, T. Kohno, and D. Sicker, “Shining light in dark places: Understanding the tor network,” in *International symposium on privacy enhancing technologies symposium*. Springer, 2008, pp. 63–76.
- [198] M. Perry, “Experimental defense for website traffic fingerprinting,” <https://blog.torproject.org/experimental-defense-website-traffic-fingerprinting/>, posted: 2011-09-05.
- [199] “Shadowsocks - a fast tunnel proxy that helps you bypass firewalls,” <https://shadowsocks.org/>, accessed: 2022-02-11.
- [200] S. Mistry and B. Raman, “Quantifying traffic analysis of encrypted web-browsing,” 1998, project paper.
- [201] Q. Sun, D. R. Simon, Y.-M. Wang, W. Russell, V. N. Padmanabhan, and L. Qiu, “Statistical identification of encrypted web browsing traffic,” in *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 19–30.
- [202] M. Liberatore and B. N. Levine, “Inferring the source of encrypted http connections,” in *Proceedings of the 13th ACM conference on Computer and communications security*, 2006, pp. 255–263.
- [203] D. Herrmann, R. Wendolsky, and H. Federrath, “Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier,” in *Proceedings of the 2009 ACM workshop on Cloud computing security*, 2009, pp. 31–42.
- [204] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, “Website fingerprinting in onion routing based anonymization networks,” in *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*, 2011, pp. 103–114.
- [205] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, “Effective attacks and provable defenses for website fingerprinting,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 143–157.
- [206] J. Hayes and G. Danezis, “k-fingerprinting: A robust scalable website fingerprinting technique,” in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1187–1203.
- [207] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, “Automated website fingerprinting through deep learning,” 2017.
- [208] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1928–1943.
- [209] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, “Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1131–1148.
- [210] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, “Snwf: website fingerprinting attack by ensembling the snapshot of deep learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1214–1226, 2022.
- [211] X. Deng, Q. Yin, Z. Liu, X. Zhao, Q. Li, M. Xu, K. Xu, and J. Wu, “Robust multi-tab website fingerprinting attacks in the wild,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1005–1022.
- [212] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, “Rfc2616: Hypertext transfer protocol-http/1.1,” 1999.
- [213] A. Hintz, “Fingerprinting websites using traffic analysis,” in *International workshop on privacy enhancing technologies*. Springer, 2002, pp. 171–178.
- [214] L. Lu, E.-C. Chang, and M. C. Chan, “Website fingerprinting and identification using ordered feature sequences,” in *European Symposium on Research in Computer Security*. Springer, 2010, pp. 199–214.
- [215] S. J. Murdoch and R. N. Watson, “Metrics for security and performance in low-latency anonymity systems,” in *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2008, pp. 115–132.
- [216] W. De la Cadena, A. Mitseva, J. Hiller, J. Pennekamp, S. Reuter, J. Filter, T. Engel, K. Wehrle, and A. Panchenko, “Trafficsliver: Fighting website fingerprinting attacks with traffic splitting,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1971–1985.
- [217] R. Dingledine and N. Mathewson, “Tor protocol specification,” <https://gitweb.torproject.org/torspec.git/tree/tor-spec.txt>, date of visit: 2021-10-05.
- [218] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, “Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail,” in *2012 IEEE symposium on security and privacy*. IEEE, 2012, pp. 332–346.
- [219] T. Wang and I. Goldberg, “Improved website fingerprinting on tor,” in *Proceedings of the 12th ACM workshop on Workshop on privacy in the electronic society*, 2013, pp. 201–212.
- [220] M. Juarez, S. Afroz, G. Acar, C. Diaz, and R. Greenstadt, “A critical evaluation of website fingerprinting attacks,” in *Proceedings of the 2014*

- ACM SIGSAC Conference on Computer and Communications Security, 2014, pp. 263–274.
- [221] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, “Website fingerprinting at internet scale,” in *NDSS*, 2016.
- [222] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, “A systematic approach to developing and evaluating website fingerprinting defenses,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 227–238.
- [223] T. Wang and I. Goldberg, “Walkie-talkie: An efficient defense against passive website fingerprinting attacks,” in *26th {USENIX} Security Symposium ({USENIX} Security 17)*, 2017, pp. 1375–1390.
- [224] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, “Toward an efficient website fingerprinting defense,” in *European Symposium on Research in Computer Security*. Springer, 2016, pp. 27–46.
- [225] K. Abe and S. Goto, “Fingerprinting attack on tor anonymity using deep learning,” *Proceedings of the Asia-Pacific Advanced Network*, vol. 42, pp. 15–20, 2016.
- [226] S. Bhat, D. Lu, A. Kwon, and S. Devadas, “Var-cnn: A data-efficient website fingerprinting attack based on deep learning,” *Proceedings on Privacy Enhancing Technologies*, vol. 1, p. 19.
- [227] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [228] S. E. Oh, S. Sunkam, and N. Hopper, “-fp: Extraction, classification, and prediction of website fingerprints with deep learning,” *Proceedings on Privacy Enhancing Technologies*, vol. 2019, no. 3, pp. 191–209, 2019.
- [229] C. Wang, J. Dani, X. Li, X. Jia, and B. Wang, “Adaptive fingerprinting: website fingerprinting over few encrypted traffic,” in *Proceedings of the Eleventh ACM Conference on Data and Application Security and Privacy*, 2021, pp. 149–160.
- [230] N. P. Hoang, A. A. Niaki, P. Gill, and M. Polychronakis, “Domain name encryption is not enough: privacy leakage via ip-based website fingerprinting,” *Proceedings on Privacy Enhancing Technologies*, vol. 2021, no. 4, pp. 420–440, 2021.
- [231] O. Ajao, J. Hong, and W. Liu, “A survey of location inference techniques on twitter,” *Journal of Information Science*, vol. 41, no. 6, pp. 855–864, 2015.
- [232] Y. Ikawa, M. Enoki, and M. Tatsubori, “Location inference using microblog messages,” in *Proceedings of the 21st international conference on world wide web*, 2012, pp. 687–690.
- [233] J. Han, E. Owusu, L. T. Nguyen, A. Perrig, and J. Zhang, “Accomplice: Location inference using accelerometers on smartphones,” in *2012 Fourth International Conference on Communication Systems and Networks (COMSNETS 2012)*. IEEE, 2012, pp. 1–9.
- [234] A. Gallagher, D. Joshi, J. Yu, and J. Luo, “Geo-location inference from image content and user tags,” in *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, 2009, pp. 55–62.
- [235] I. Trestian, S. Ranjan, A. Kuzmanovic, and A. Nucci, “Measuring serendipity: connecting people, locations and interests in a mobile 3g network,” in *Proceedings of the 9th ACM SIGCOMM conference on Internet measurement*, 2009, pp. 267–279.
- [236] A. K. Das, P. H. Pathak, C.-N. Chuah, and P. Mohapatra, “Contextual localization through network traffic analysis,” in *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 2014, pp. 925–933.
- [237] F. Xu, Y. Li, H. Wang, P. Zhang, and D. Jin, “Understanding mobile traffic patterns of large scale cellular towers in urban environment,” *IEEE/ACM transactions on networking*, vol. 25, no. 2, pp. 1147–1161, 2016.
- [238] H. Wang, F. Xu, Y. Li, P. Zhang, and D. Jin, “Understanding mobile traffic patterns of large scale cellular towers in urban environment,” in *Proceedings of the 2015 Internet Measurement Conference*, 2015, pp. 225–238.
- [239] F. Xu, Y. Lin, J. Huang, D. Wu, H. Shi, J. Song, and Y. Li, “Big data driven mobile traffic understanding and forecasting: A time series approach,” *IEEE transactions on services computing*, vol. 9, no. 5, pp. 796–805, 2016.
- [240] R. Lippmann, D. Fried, K. Piwowarski, and W. Streilein, “Passive operating system identification from tcp/ip packet headers,” in *Workshop on Data Mining for Computer Security*, vol. 40, 2003.
- [241] Y.-C. Chen, Y. Liao, M. Baldi, S.-J. Lee, and L. Qiu, “Os fingerprinting and tethering detection in mobile networks,” in *Proceedings of the 2014 Conference on Internet Measurement Conference*, 2014, pp. 173–180.
- [242] M. Laštovička, S. Špaček, P. Velan, and P. Čeleda, “Using tls fingerprints for os identification in encrypted traffic,” in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–6.
- [243] N. Ruffing, Y. Zhu, R. Libertini, Y. Guan, and R. Bettati, “Smart-phone reconnaissance: Operating system identification,” in *2016 13th IEEE Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 2016, pp. 1086–1091.
- [244] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, “Network traffic classifier with convolutional and recurrent neural networks for internet of things,” *IEEE access*, vol. 5, pp. 18 042–18 050, 2017.
- [245] Y. Meidan, M. Bohadana, A. Shabtai, J. D. Guarnizo, M. Ochoa, N. O. Tippenhauer, and Y. Elovici, “Profilot: a machine learning approach for iot device identification based on network traffic analysis,” in *Proceedings of the symposium on applied computing*, 2017, pp. 506–509.
- [246] A. Sivanathan, H. H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, and V. Sivaraman, “Classifying iot devices in smart environments using network traffic characteristics,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 8, pp. 1745–1759, 2018.
- [247] H. Yao, P. Gao, J. Wang, P. Zhang, C. Jiang, and Z. Han, “Capsule network assisted iot traffic classification mechanism for smart cities,” *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 7515–7525, 2019.
- [248] L. Bernaille, R. Teixeira, I. Akodkenou, A. Soule, and K. Salamatian, “Traffic classification on the fly,” *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 2, pp. 23–26, 2006.
- [249] T. Karagiannis, K. Papagiannaki, and M. Faloutsos, “Blink: multilevel traffic classification in the dark,” in *Proceedings of the 2005 conference on Applications, technologies, architectures, and protocols for computer communications*, 2005, pp. 229–240.
- [250] L. Bernaille and R. Teixeira, “Early recognition of encrypted applications,” in *International Conference on Passive and Active Network Measurement*. Springer, 2007, pp. 165–175.
- [251] L. Bernaille, R. Teixeira, and K. Salamatian, “Early application identification,” in *Proceedings of the 2006 ACM CoNEXT conference*, 2006, pp. 1–12.
- [252] A. McGregor, M. Hall, P. Lorier, and J. Brunskill, “Flow clustering using machine learning techniques,” in *International workshop on passive and active network measurement*. Springer, 2004, pp. 205–214.
- [253] A. W. Moore and D. Zuev, “Internet traffic classification using bayesian analysis techniques,” in *Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, 2005, pp. 50–60.
- [254] Z. Chen, K. He, J. Li, and Y. Geng, “Seq2img: A sequence-to-image based approach towards ip traffic classification using convolutional neural networks,” in *2017 IEEE International conference on big data (big data)*. IEEE, 2017, pp. 1271–1276.
- [255] J. Zhao, Q. Li, Y. Hong, and M. Shen, “Metarocketc: Adaptive encrypted traffic classification in complex network environments via time series analysis and meta-learning,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2460–2476, 2024.
- [256] Q. Wang, A. Yahyavi, B. Kemme, and W. He, “I know what you did on your smartphone: Inferring app usage over encrypted data traffic,” in *2015 IEEE conference on communications and network security (CNS)*. IEEE, 2015, pp. 433–441.
- [257] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, “Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic,” in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 439–454.
- [258] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, “Multi-classification approaches for classifying mobile app traffic,” *Journal of Network and Computer Applications*, vol. 103, pp. 131–145, 2018.
- [259] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, “Mappgraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks,” in *Proceedings of the 37th Annual Computer Security Applications Conference*, 2021, pp. 1025–1038.
- [260] F. Airolli, M. Conti, A. Gangwal, and M. Polato, “Mind your wallet’s privacy: identifying bitcoin wallet apps and user’s actions through network traffic analysis,” in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 2019, pp. 1484–1491.
- [261] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, “Accurate decentralized application identification via encrypted traffic analysis using graph neural networks,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2367–2380, 2021.

- [262] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 1, pp. 63–78, 2017.
- [263] G. Aceto, D. Ciunzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 445–458, 2019.
- [264] S. E. Coull and K. P. Dyer, "Traffic analysis of encrypted messaging services: Apple imessage and beyond," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 5–11, 2014.
- [265] C. V. Wright, L. Ballard, S. E. Coull, F. Monrose, and G. M. Masson, "Spot me if you can: Uncovering spoken phrases in encrypted voip conversations," in *2008 IEEE Symposium on Security and Privacy (sp 2008)*. IEEE, 2008, pp. 35–49.
- [266] M. Conti, L. V. Mancini, R. Spolaor, and N. V. Verde, "Analyzing android encrypted network traffic to identify user actions," *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 1, pp. 114–125, 2015.
- [267] B. Saltaformaggio, H. Choi, K. Johnson, Y. Kwon, Q. Zhang, X. Zhang, D. Xu, and J. Qian, "Eavesdropping on fine-grained user activities within smartphone apps over encrypted network traffic," in *10th {USENIX} Workshop on Offensive Technologies ({WOOT} 16)*, 2016.
- [268] F. Yan, M. Xu, T. Qiao, T. Wu, X. Yang, N. Zheng, and K.-K. R. Choo, "Identifying wechat red packets and fund transfers via analyzing encrypted network traffic," in *2018 17th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/12th IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE)*. IEEE, 2018, pp. 1426–1432.
- [269] Y. Wang, N. Zheng, M. Xu, T. Qiao, Q. Zhang, F. Yan, and J. Xu, "Hierarchical identifier: Application to user privacy eavesdropping on mobile payment app," *Sensors*, vol. 19, no. 14, p. 3052, 2019.
- [270] F. Schneider, A. Feldmann, B. Krishnamurthy, and W. Willinger, "Understanding online social network usage from a network perspective," in *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*, 2009, pp. 35–48.
- [271] Y. Feng, "Botflowmon: Identify social bot traffic with netflow and machine learning," 2018.
- [272] M. Conti, L. V. Mancini, R. Spolaor, and N. V. Verde, "Can't you hear me knocking: Identification of user actions on android apps via traffic analysis," in *Proceedings of the 5th ACM Conference on Data and Application Security and Privacy*, 2015, pp. 297–304.
- [273] Y. Zhao, X. Ma, J. Li, S. Yu, and W. Li, "Revisiting website fingerprinting attacks in real-world scenarios: A case study of shadowsocks," in *International Conference on Network and System Security*. Springer, 2018, pp. 319–336.
- [274] R. Nithyanand, X. Cai, and R. Johnson, "Glove: A bespoke website fingerprinting defense," in *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, 2014, pp. 131–134.
- [275] A. J. Pinheiro, P. Freitas de Araujo-Filho, J. de M. Bezerra, and D. R. Campelo, "Adaptive packet padding approach for smart home networks: A tradeoff between privacy and performance," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3930–3938, 2021.
- [276] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 717–734.
- [277] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *NDSS*, vol. 9. Citeseer, 2009.
- [278] X. Luo, P. Zhou, E. W. Chan, W. Lee, R. K. Chang, R. Perdisci *et al.*, "Httpos: Sealing information leaks with browser-side obfuscation of encrypted flows," in *NDSS*, vol. 11, 2011.
- [279] G. Cherubin, J. Hayes, and M. Juárez, "Website fingerprinting defenses at the application layer," *Proc. Priv. Enhancing Technol.*, vol. 2017, no. 2, pp. 186–203, 2017.
- [280] Z. Deng, Z. Liu, Z. Chen, and Y. Guo, "The random forest based detection of shadowsocks's traffic," in *2017 9th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC)*, vol. 2. IEEE, 2017, pp. 75–78.
- [281] J. Beznazwy and A. Houmansadr, "How china detects and blocks shadowsocks," in *Proceedings of the ACM Internet Measurement Conference*, 2020, pp. 111–124.
- [282] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related," in *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*. sn, 2016, pp. 407–414.
- [283] P. Choorod and G. Weir, "Tor traffic classification based on encrypted payload characteristics," in *2021 National Computing Colleges Conference (NCCC)*. IEEE, 2021, pp. 1–6.
- [284] I. Goldberg and C. A. Wood, "Network-based website fingerprinting," <https://datatracker.ietf.org/doc/html/draft-wood-privsec-wfattacks-00>, 2019.
- [285] "New tor release: Tor 0.4.0.5," <https://blog.torproject.org/new-release-tor-0405/>, 2019, accessed: 2022-02-15.
- [286] S. Li, H. Guo, and N. Hopper, "Measuring information leakage in website fingerprinting attacks and defenses," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1977–1992.
- [287] X. Cai, R. Nithyanand, and R. Johnson, "Cs-bufflo: A congestion sensitive website fingerprinting defense," in *Proceedings of the 13th Workshop on Privacy in the Electronic Society*, 2014, pp. 121–130.
- [288] D. Lu, S. Bhat, A. Kwon, and S. Devadas, "Dynaflow: An efficient website fingerprinting defense based on dynamically-adjusting flows," in *Proceedings of the 2018 Workshop on Privacy in the Electronic Society*, 2018, pp. 109–113.
- [289] S. Henri, G. Garcia-Aviles, P. Serrano, A. Banchs, and P. Thiran, "Protecting against website fingerprinting with multihoming," *Proceedings on Privacy Enhancing Technologies*, vol. 2020, no. 2, pp. 89–110, 2020.
- [290] T. Wang, "The one-page setting: A higher standard for evaluating website fingerprinting defenses," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2794–2806.
- [291] mikeperry, "Experimental defense for website traffic fingerprinting," <https://blog.torproject.org/experimental-defense-website-traffic-fingerprinting/>, 2011, accessed: 2022-02-11.
- [292] A. Fotouhi, H. Qiang, M. Ding, M. Hassan, L. G. Giordano, A. Garcia-Rodriguez, and J. Yuan, "Survey on uav cellular communications: Practical aspects, standardization advancements, regulation, and security challenges," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3417–3442, 2019.
- [293] J. Whelan, A. Almelhadi, and K. El-Khatib, "Artificial intelligence for intrusion detection systems in unmanned aerial vehicles," *Computers and Electrical Engineering*, vol. 99, p. 107784, 2022.
- [294] K. Kim, J. S. Kim, S. Jeong, J.-H. Park, and H. K. Kim, "Cybersecurity for autonomous vehicles: Review of attacks and defense," *Computers & Security*, vol. 103, p. 102150, 2021.
- [295] K. M. Ali Alheeti and K. McDonald-Maier, "Intelligent intrusion detection in external communication systems for autonomous vehicles," *Systems Science & Control Engineering*, vol. 6, no. 1, pp. 48–56, 2018.
- [296] Y. Feng, J. Li, and T. Nguyen, "Application-layer ddos defense with reinforcement learning," in *2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*. IEEE, 2020, pp. 1–10.
- [297] W. Wang, Y. Shang, Y. He, Y. Li, and J. Liu, "Botmark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors," *Information Sciences*, vol. 511, pp. 284–296, 2020.
- [298] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: methods, systems and tools," *Ieee communications surveys & tutorials*, vol. 16, no. 1, pp. 303–336, 2013.
- [299] S. Barnum, "Standardizing cyber threat intelligence information with the structured threat information expression (stix)," *Mitre Corporation*, vol. 11, pp. 1–22, 2012.
- [300] OpenAI, "Chatgpt," 2023, [Online]. Available: <https://openai.com/chatgpt>.
- [301] A. Nascita, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "A survey on explainable artificial intelligence for internet traffic classification and prediction, and intrusion detection," *IEEE Communications Surveys & Tutorials*, 2024.
- [302] D. Han, Z. Wang, W. Chen, Y. Zhong, S. Wang, H. Zhang, J. Yang, X. Shi, and X. Yin, "Deepaid: Interpreting and improving deep learning-based anomaly detection in security applications," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3197–3217.
- [303] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.
- [304] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.

- [305] G. Plumb, D. Molitor, and A. S. Talwalkar, "Model agnostic supervised local explanations," *Advances in neural information processing systems*, vol. 31, 2018.
- [306] V. Belle and I. Papantonis, "Principles and practice of explainable machine learning," *Frontiers in big Data*, p. 39, 2021.
- [307] U. Bhatt, A. Xiang, S. Sharma, A. Weller, A. Taly, Y. Jia, J. Ghosh, R. Puri, J. M. Moura, and P. Eckersley, "Explainable machine learning in deployment," in *Proceedings of the 2020 conference on fairness, accountability, and transparency*, 2020, pp. 648–657.
- [308] Y. Feng, J. Xu, and L. Weymouth, "University blockchain research initiative (ubri): Boosting blockchain education and research," *IEEE Potentials*, vol. 41, no. 6, pp. 19–25, 2022.



Yebo Feng is a research fellow in the College of Computing and Data Science (CCDS) at Nanyang Technological University (NTU). He received his Ph.D. degree in Computer Science from the University of Oregon (UO) in 2023. His research interests include network security, blockchain security, and anomaly detection. He is the recipient of the Best Paper Award of 2019 IEEE CNS, Gurdeep Pall Graduate Student Fellowship of UO, and Ripple Research Fellowship. He has served as the reviewer of IEEE TDSC, IEEE TIFS, ACM TKDD, IEEE

JSAC, IEEE COMST, etc. Furthermore, he has been a member of the program committees for international conferences including SDM, CIKM, and CYBER, and has also served on the Artifact Evaluation (AE) committees for USENIX OSDI and USENIX ATC.



Cong Wu is currently a research fellow at School of Computer Science and Engineering, Nanyang Technological University, Singapore. He received Ph.D. degree at School of Cyber Science and Engineering, Wuhan University in 2022. His research interests include AI system security and Web3 security. His research outcomes have appeared in USENIX Security, ACM CCS, IEEE TDSC, TIFS.



Chong Wang received the bachelor's and PhD degrees from Fudan University, in 2018. He is currently a research fellow at Cyber Security Laboratory, Nanyang Technological University. His research interests lie in the crossroads between artificial intelligence and software engineering. His overarching research mission revolves around enhancing both productivity and security within the realm of software development. He has published multiple papers in international journals and conferences, such as the IEEE Transactions on Software Engineering (TSE), ACM Transactions on Software Engineering and Methodology (TOSEM), ACM Symposium on the Foundations of Software Engineering (FSE), IEEE/ACM International Conference on Automated Software Engineering (ASE) and IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER).

neering (TSE), ACM Transactions on Software Engineering and Methodology (TOSEM), ACM Symposium on the Foundations of Software Engineering (FSE), IEEE/ACM International Conference on Automated Software Engineering (ASE) and IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER).



Jun Li is a professor in the Department of Computer Science and director of the Network and Security Research Laboratory at the University of Oregon. He was also a Ripple Fellow, Narus Research Fellow, and founding director of the Center for Cyber Security and Privacy at the University of Oregon. He has received the CAREER Award from the US National Science Foundation, the Faculty Excellence Award from the University of Oregon, and the Recognition of Service Award from ACM. He received his Ph.D. with honors from UCLA in 2002. His research interests include networking, distributed systems, cybersecurity, and blockchain.

He has published more than 100 peer-reviewed papers, including several Best Paper awards.



Hao Ren was a Research Fellow at Nanyang Technological University, Singapore. He received his Ph.D. degree in Dec. 2020 from the University of Electronic Science and Technology of China. He was a visiting Ph.D. student at the University of Waterloo from Dec. 2018 to Jan. 2020. He has published papers in major conferences/journals, including ACM ASIACCS, ACSAC, IEEE TCC, and IEEE Network. He won the Best Paper Award from IEEE BigDataSecurity 2023. His research interests include applied cryptography and privacy-preserving

machine learning.



Jelena Mirkovic is Research Team Leader at USC Information Sciences Institute and Research Associate Professor of Computer Science at USC. During her professional career she published more than 100 conference and journal papers, and the first book on the denial-of-service attacks. She also pioneered use of testbeds in cybersecurity education. Her research interests span cybersecurity, networking and education.



Jiahua Xu is Associate Professor in Financial Computing, and Programme Director of the MSc Emerging Digital Technologies at UCL. She is also affiliated to the UCL Centre for Blockchain Technologies. Her research focuses on blockchain economics and decentralized finance. She has published in Usenix Security, ACM IMC, FC, IEEE ICDCS and IEEE ICBC. She has reviewed for Advances in Complex Systems, Computer Networks, Transactions on the Web and Cities.



Yang Liu is currently a full professor and the director of the cyber security lab in Nanyang Technological University, Singapore. He specializes in software security, verification, software engineering and artificial intelligence. His research has bridged the gap between the theory and practical usage of formal methods and program analysis to evaluate the design and implementation of software for high assurance and security. His work led to the development of state-of-the-art model checker, Process Analysis Toolkit (PAT). By now, he has more than

200 publications and 6 best paper awards in top tier conferences and journals. With more than 50 million Singapore dollar funding support, he is leading a large research team working on state-of-the-art software engineering and cyber security problems and currently serving as an associated editor of TIFS.